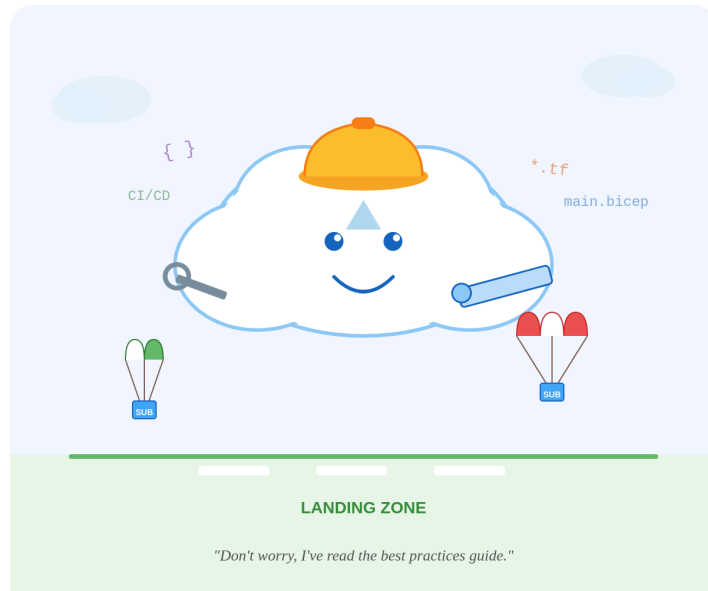




Azure Landing Zones

Infrastructure-as-Code Repository Best Practices

Repository: erjosito/alz-repo-best-practices

Version: b662f42 · 26 May 2026

A field guide for DevOps architects and platform engineers

Azure Landing Zones - Infrastructure-as-Code Repository Best Practices

A field guide for DevOps architects and platform engineers designing the Git repositories, pipelines, and operating model that will deliver an Azure Landing Zone (ALZ) implementation at enterprise scale.

This guide is **opinionated but balanced**: every chapter focuses on the *design decisions* you have to make, the *tradeoffs* of each option, and a *recommendation* for the typical enterprise scenario. Anti-patterns are called out explicitly.

Why this guide exists

When the first Azure Landing Zone accelerators appeared in 2020-2021, most enterprise teams focused on the *content* of the landing zone: which policies to apply, how to structure management groups, which connectivity model to adopt. The *repository* that would host and maintain that content was treated as an afterthought -- a place to put ARM templates or Terraform files before the real work began. Years later, those same teams found the infrastructure code was solid but the operational model around it had accumulated technical debt that was genuinely hard to pay down: long-lived pipeline secrets that could not be rotated without downtime, state files covering entire environments with no blast-radius control, naming conventions that existed only in a Confluence page nobody had updated since 2022, and modules copy-pasted so many times that fixing a bug meant finding every copy first.

This guide exists to address the repository and operating model -- not the policy library or the network topology, but the decisions that determine whether your ALZ implementation remains manageable at year three as much as at day one. It draws on official Microsoft guidance, community patterns from the IaC ecosystem, and the lessons of platform engineering teams that have operated Azure estates at enterprise scale. The "How we got here" narratives at the start of each chapter trace how the industry's tooling and thinking evolved to produce the current best practices -- so you understand not just *what* to do, but *why* the field landed there.

It is written for platform engineers and DevOps architects who are either building a new ALZ implementation or inheriting one that needs to be brought up to a higher standard. If you are starting fresh, read the chapters in order. If you are inheriting an existing estate, start with the summary below and Chapter 14 -- the anti-pattern checklist is the fastest way to identify where the most pressing technical debt lives.

How to read this guide

The chapters are loosely ordered from "high-level strategy" to "day-2 operations". You can read them in order, or jump straight to the decision you are wrestling with - every page links to its neighbours.

#	Topic	Decision in one sentence
01	<u>Repository topology - monorepo vs multi-repo</u>	How many Git repositories, and what goes in each?
02	<u>IaC tooling - Bicep, Terraform, ARM, Pulumi</u>	Which language/engine for which layer of the platform?
03	<u>Module & registry strategy (AVM, ALZ modules)</u>	Where do reusable modules live and how are they versioned?
04	<u>Branching, environments & promotion</u>	How does a change flow from a developer laptop to prod?
05	<u>Authentication & identity for pipelines</u>	OIDC, federated credentials, MIs, service principals.
06	<u>Secrets & supply-chain security</u>	Secrets handling, signed commits, SBOMs, dependency hygiene.
07	<u>State management (Terraform & deployment stacks)</u>	Backends, locking, blast-radius, drift.
08	<u>CI/CD pipeline patterns</u>	GitHub Actions vs Azure DevOps; PR vs deploy workflows.
09	<u>Testing, validation & policy-as-code</u>	what-if, plan, PSRule, Conftest, OPA, Checkov.
10	<u>Code quality, linting & developer experience</u>	Pre-commit hooks, formatters, devcontainers.
11	<u>Manageability & day-2 operations</u>	Drift, blast radius, ownership (CODEOWNERS), runbooks.
12	<u>Naming, tagging & metadata conventions</u>	The boring stuff that breaks everything if you skip it.
13	<u>Documentation & onboarding</u>	ADRs, READMEs, diagrams-as-code.
14	<u>Anti-patterns & common pitfalls</u>	A checklist of things you will regret.

A consolidated list of external references is in [docs/references.md](#).

Scope and assumptions

- **Target audience:** platform engineering teams shipping an ALZ implementation to one or more Azure tenants for an enterprise IT estate.
- **Definition of "ALZ":** the Microsoft Azure Landing Zones conceptual architecture (management groups, policies, platform subscriptions, application landing zones). See the [Cloud Adoption Framework - Landing Zone docs](#).
- **Out of scope:** application-level CI/CD, AKS cluster patterns, Azure Policy authoring (referenced but not deep-dived), and the underlying networking topology (covered only where it affects repository design).

A quick summary for the impatient

If you take nothing else from this guide:

1. **Separate "platform" from "application landing zones"** at the repository level. Mixing them is the single biggest source of long-term pain.

2. **Use OIDC federated credentials** for all pipeline auth. Never store an Azure secret in a CI/CD variable in 2026.
3. **Pin every module by version**, never by `main`. Use semantic versioning and a private registry (or git tags) as the source of truth.
4. **Treat policy as the API contract** between the platform team and application teams - not as a guard-rail bolted on later.
5. **Keep the blast radius of any single apply small**. One state file per environment per workload, not one giant state file for the whole tenant.

The rest of this repo explains *why*.

PDF version

A print-ready PDF of the full guide is available at [dist/ALZ-IaC-Best-Practices.pdf](#) -- optimised for e-readers (reMarkable, Kindle) with rendered diagrams, tables, and a cover page showing the version it was built from.

To regenerate after editing docs:

```
node .github/skills/generate-pdf/scripts/build-pdf.mjs \  
  && npx md-to-pdf dist/alz-combined.md --config-file .github/skills/generate-  
pdf/scripts/pdf-config.json \  
  && mv dist/alz-combined.pdf dist/ALZ-IaC-Best-Practices.pdf
```

Requires Node.js 20+. Dependencies (`md-to-pdf` , `@mermaid-js/mermaid-cli`) are fetched automatically via `npx`.

01 · Repository topology -- monorepo vs multi-repo

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [The three layers of an ALZ estate](#)
- [Option A -- Pure monorepo](#)
- [Option B -- Pure multi-repo](#)
- [Option C -- Layered "few-repo" \(recommended default\)](#)
- [When ownership boundaries blur -- cross-team resources](#)
- [Anti-patterns](#)
- [Reference layouts](#)
- [References](#)

Decision: *how many Git repositories will host your ALZ IaC, and what belongs in each?*

[← Back to index](#) · Next → [02 IaC tooling](#)

For roughly 80 % of enterprise ALZ implementations, the right answer is a layered "**few-repo**" topology: three to five Git repositories aligned to the natural ownership boundaries of foundation, platform, modules, landing zones, and (optionally) policies. A pure monorepo is correct only for small estates with a single platform team, and a pure multi-repo is correct only when you have a self-service developer portal and the platform investment to keep dozens of repositories aligned with each other. The recurring mistake is treating this as a tooling question when it is really an ownership question -- repo boundaries should mirror who is on call, who approves changes, and who is held accountable when something breaks. The cost of getting it wrong is unusually high because repository topology is one of the most expensive things to refactor after the fact: get it wrong on day one and you will spend years either drowning in cross-repo pull requests or watching a single broken pipeline freeze the entire estate.

How we got here

In the early 2010s, when "infrastructure as code" still mostly meant Chef cookbooks and Puppet manifests, the natural home was a single SVN or Git repo per project. Then the **microservices wave** of 2014-2017 cargo-culted the same fragmentation onto infrastructure: every service got its own repo, its own Terraform state, its own pipeline. Within a couple of years, "how do I bump the VPC module across 80 repos?" became a recurring LinkedIn post. By the late 2010s the pendulum swung the other way as Google, Facebook, and Microsoft published essays on the productivity benefits of monorepos backed by purpose-built tooling (Bazel, Buck, Source Depot). For IaC specifically, the Terraform community discovered that **Git submodules don't fix the problem** and that **module registries do** (Terraform Registry, then private

registries, then OCI). Today most mature ALZ implementations live in a *small number* of carefully chosen repos -- neither a single behemoth nor an unmanageable swarm -- and that's the option this chapter recommends.

Key terms

ALZ (Azure Landing Zone) -- a pre-configured Azure environment (management groups, subscriptions, policies, networking) that follows Microsoft's Cloud Adoption Framework and is ready to host workloads securely at scale.

IaC (Infrastructure as Code) -- the practice of defining cloud infrastructure in declarative or imperative code files (Terraform, Bicep, Pulumi) rather than manual portal clicks, enabling version control, review, and automation.

SVN (Subversion) -- a centralised version-control system that predated Git's dominance.

Cargo-culting -- blindly copying a practice from another context without understanding why it worked there.

Git submodules -- a Git feature that embeds one repository inside another; notoriously difficult to manage and a common source of checkout confusion.

Module registries -- hosted catalogues (e.g. Terraform Registry, Azure Container Registry) that distribute versioned IaC modules, replacing error-prone Git-source references.

OCI (Open Container Initiative) -- a standard for container image formats and registries, increasingly used to distribute non-container artefacts such as IaC modules.

Decision framework

Pick the topology by answering these questions in order. The first answer that points decisively in one direction usually wins; later questions are tiebreakers. The three option names below (monorepo, layered few-repo, multi-repo) are explained in the dedicated sections further down.

1. How many teams will commit IaC?

- 1 team → **monorepo** is fine.
- 2-10 teams → **layered few-repo** (recommended default).
- 10+ teams → layered few-repo with one landing-zone repo per business unit, or full **multi-repo** if you have a self-service developer portal.

2. What is your regulatory posture?

- Auditors require separation of duties between *who changes policy* and *who deploys workloads* → at minimum split foundation/policy from workloads (rules out the pure monorepo).

3. Do you have a self-service developer portal? (Backstage, internal CLI, scaffolding tool)

- Yes → multi-repo becomes viable.
- No → stay layered; the discoverability cost of multi-repo is prohibitive.

4. How different are the deployment cadences across layers?

- Wildly different (foundation quarterly, workloads daily) → split repos. Putting a quarterly-changing foundation in the same pipeline trigger as daily workloads is constant friction.
- Similar cadence everywhere → topology matters less; choose on team/ownership grounds.

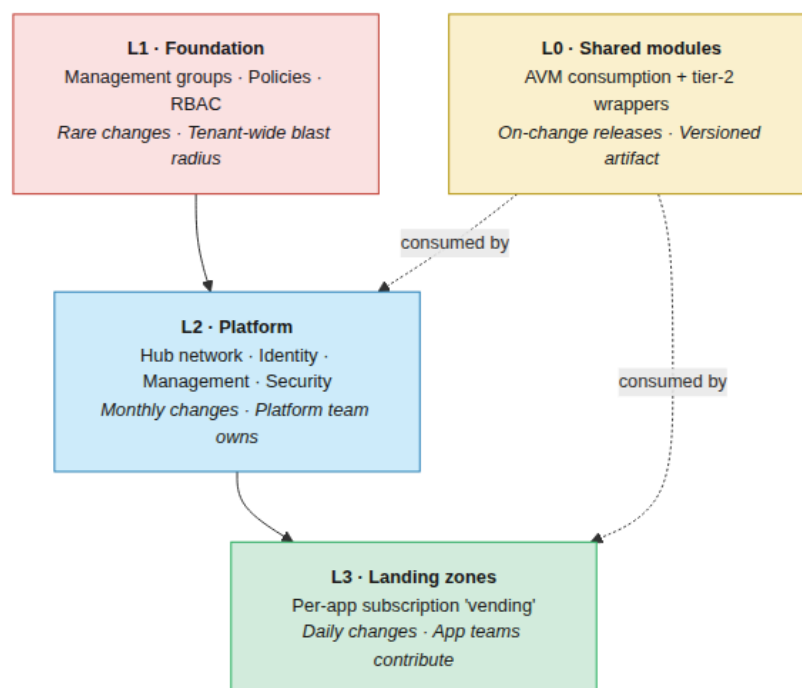
Quick reference of the three options:

Option	One-liner	Fits when
<u>Pure monorepo</u>	One repo for foundation, platform, modules, and every landing zone.	<10 landing zones, one platform team, no separation-of-duties requirement.
<u>Layered "few-repo"</u> (recommended)	3-5 repos aligned to ownership boundaries.	2-10 teams, mixed cadences, normal enterprise governance.
<u>Pure multi-repo</u>	One repo per workload, per module, per policy set.	100+ landing zones with a self-service portal and platform investment to keep them aligned.

The full analysis -- pros, cons, when each option breaks, and the tradeoffs of each choice -- follows below.

The three layers of an ALZ estate

Before diving into the options, a shared vocabulary: almost every ALZ implementation has the same natural layers. Whether each layer is a *folder* or a *repo* is precisely what the topology question decides.



L0 sits to the side: it's not a deployment layer at all but a *library* that L2 and L3 consume by version.

With the layers named, here is how each topology distributes them -- and where each one breaks down in practice.

Option A -- Pure monorepo

A single repository holds the foundation, the platform, every reusable module, and every application landing zone. This is the simplest topology to start with -- and for small estates with a single platform team and no separation-of-duties requirement, it is also the right one. Beyond roughly ten landing zones, however, the operational tax begins to compound quickly: permissions get coarser than your governance model wants, pipelines get slower than your developers will tolerate, and the shared blast radius starts to make even routine commits feel risky. A typical layout looks like this:

```
alz/
├── foundation/      # mgmt groups, policy assignments
├── platform/
│   ├── connectivity/ # hub VNets, ER, firewall
│   ├── identity/
│   └── management/   # log analytics, automation
├── landingzones/
│   ├── corp-app01/
│   └── online-app42/
├── modules/         # local module library
└── .github/workflows/
```

✓ Pros

- **Atomic cross-layer changes.** "Add a new region" can be one PR that touches policy, hub, and a workload simultaneously.
- **Single source of truth** for naming conventions, tags, module versions.
- **Easy refactoring** with codebase-wide search/replace and IDE rename.
- **One pipeline framework** to maintain.

Key terms used below

CODEOWNERS -- a file (`.github/CODEOWNERS`) that maps file-path patterns to GitHub teams or users who must approve PRs touching those paths. Only enforced when branch protection requires CODEOWNERS review.

Blast radius -- the scope of damage a single failure can cause. In IaC, this is typically defined by what resources a single `terraform apply` or deployment stack can touch. Smaller blast radius means a mistake affects fewer resources.

Path filters & matrix builds -- path filters trigger a pipeline only when files in specific directories change, avoiding unnecessary runs. Matrix builds run multiple jobs in parallel (e.g. one per environment or workload), allowing a single workflow to plan/apply many targets concurrently.

✗ Cons

- **Permissions are coarse.** GitHub repo roles apply to the whole repo; fine-grained access requires CODEOWNERS + branch protection gymnastics.

- **Pipeline scaling.** A naive workflow that runs `terraform plan` on the whole repo on every PR becomes unusable past ~50 workloads. You **must** invest in path filters and matrix builds early.
- **Blast radius psychology.** Engineers see the foundation code next to their app code and either get scared to commit or, worse, don't.
- **Single CODEOWNERS becomes a 500-line file** that nobody reviews.

The monorepo cracks start to show the moment a team that has been comfortably shipping a handful of workloads receives the polite request to "please add just twenty more this quarter". When that happens, the natural next move is the opposite extreme, where every ownership boundary is made explicit by giving each one its own repository.

Option B -- Pure multi-repo

The opposite extreme is to give every workload, every module, and every policy set its own repository. The discipline this enforces is obvious -- each repo maps cleanly to a single team, a single on-call rotation, and a single release cadence -- but the coordination tax is just as obvious. This topology is correct only at very large scale (100+ landing zones) where you already have a self-service developer portal and the platform investment to keep dozens of repos honest with each other. Without that scaffolding, the cost of cross-cutting changes swamps the benefits quickly. A layout in this style does not need a diagram: it looks like sixty repositories with broadly similar contents and slightly different names.

✓ Pros

- **Crystal-clear ownership** -- repo = team = on-call rotation.
- **Independent release cadence.** Workload A deploys 10× a day; the foundation deploys quarterly. Each gets the cadence it needs.
- **Smallest possible blast radius** per pipeline run.
- **Simple permissions** -- GitHub team → repo mapping is 1:1.

✗ Cons

- **Cross-cutting changes are nightmarish.** Bumping a module version across 60 repos = 60 PRs, 60 reviews, 60 pipeline runs (Renovate/Dependabot helps, doesn't eliminate).
- **Drift between repos.** Conventions, pipeline templates, even tool versions diverge. You end up needing a "templates" repo + scaffolding tool (cookiecutter, Backstage, `gh repo template`) just to keep them aligned.
- **Discoverability suffers.** New engineers ask "where is X?" constantly.

Most organisations don't fit neatly into either of these two extremes, which is precisely why a third option exists. It trades the conceptual cleanness of the poles for something more pragmatic and, in practice, more durable.

Option C -- Layered "few-repo" (recommended default)

The pragmatic compromise sits between the two extremes, and it is the right answer for roughly 80 % of enterprise ALZ implementations: three to five repositories aligned to the natural ownership boundaries, each with a coherent `CODEOWNERS`, its own approval policy, and its own pipeline cadence, with a module registry that keeps consumers honest about which version they depend on. The split below is the one most enterprises converge on, although the exact naming and whether to peel off `alz-policies` as a fifth repo are matters of local preference rather than principle:

Repo	Owner	Cadence	Typical blast radius
<code>alz-foundation</code>	Cloud governance	Quarterly	Tenant
<code>alz-platform</code>	Platform engineering	Weekly	Platform subs
<code>alz-modules</code>	Platform engineering	On change (semver)	None (artifact only)
<code>alz-landingzones</code> or one repo per business unit	App teams + platform reviewers	Daily	One subscription
<code>alz-policies</code> (optional)	Security/governance	Weekly	Tenant policy

✓ Pros

- Ownership boundaries match real-world team boundaries.
- Each repo has a *coherent* `CODEOWNERS`, branch protection, and approval policy.
- Module repo can publish to a registry (ACR / Terraform Cloud / private registry) and consumers pin versions -- see [03 modules & registries](#).
- Foundation repo can require *much* stricter controls (mandatory pair review, manual approval, change-advisory ticket reference) without slowing daily workload deployments.

✗ Cons

- Cross-repo refactors that genuinely span layers -- bumping a module that foundation, platform, and landing zones all consume -- still need coordination. They are rare by construction (foundation rarely changes), but you should plan for them rather than wish them away.
- You need a shared **pipeline template repo** (`alz-pipeline-templates`) or reusable workflows so the four or five repos don't drift in their CI definitions.

Even with these caveats, the few-repo split is the default the rest of this book assumes when a chapter needs to talk about "the platform repo" or "the modules repo". It is not, however, the only defensible choice, and the next callout summarises the live debate among practitioners.

The debate -- monorepo vs few-repo for IaC

The recommendation above ("layered few-repo for ~80 % of enterprises") reflects the mainstream ALZ guidance, but informed practitioners disagree.

The monorepo case: Major tech companies (Google, Meta) operate massive monorepos successfully. Their argument: a single source of truth eliminates version-skew between layers, makes cross-cutting refactors atomic, and improves discoverability ("everything is `grep`-able in one place"). With CODEOWNERS, path-scoped branch rules, and mature CI (path filters + matrix builds), monorepos can enforce ownership boundaries nearly as well as separate repos -- without the coordination overhead of 60 PRs for a module bump. Terragrunt and Nx/Turborepo make monorepos practical even at scale.

The few-repo case: In regulated enterprises, repo-level access control is simpler to audit than path-level CODEOWNERS. Independent release cadences (foundation = quarterly, workloads = daily) are naturally expressed by separate repos with separate pipelines. The psychological effect matters too: an app engineer who can see and accidentally edit foundation code is a risk no CODEOWNERS file fully mitigates.

Where the industry stands (2026): Most Azure-focused guidance (CAF, ALZ accelerators) defaults to few-repo. Most Terraform-community guidance defaults to either monorepo + Terragrunt or multi-repo + registry. Neither is wrong -- the choice hinges more on your team's Git maturity and regulatory posture than on any inherent technical superiority.

When ownership boundaries blur -- cross-team resources

Option C assumes clean ownership boundaries -- the platform team owns the platform repo, app teams own their landing zones -- but reality is messier. Some Azure resources sit at the boundary between platform and application, and no repo split eliminates the tension entirely. The way through it is to separate **ownership of intent** (the team that knows *what* the rule needs to do) from **ownership of implementation** (the team that owns the deployable code), and then to enforce the boundary with Azure Policy, CODEOWNERS, or both. Two recurring examples illustrate the pattern, followed by a decision heuristic and the most common variants (DNS, subscription vending, multi-region).

NSGs -- app-owned resource, platform-mandated rules

Network Security Groups live in the application landing zone subscription. The app team knows which ports their workload needs. But the central security team mandates baseline rules: no `*-to-*` inbound, required deny-all at the bottom, specific allowed sources for management traffic.

The pattern that survives contact with reality combines **app ownership with policy guardrails**, so that the app team retains day-to-day control of the rules while the security team retains the veto. Concretely:

- The **app team owns the NSG code** in their landing zone folder and submits PRs for rule changes.
- **Azure Policy** (owned by the security team in the policies repo) denies non-compliant rules at deploy time -- e.g. a `deny` policy that rejects NSG rules with `sourceAddressPrefix: *` and `access: Allow`.

- The NSG *module* (in `alz-modules`) bakes in mandatory baseline rules by default and exposes only constrained inputs for app-specific additions.

This keeps the blast radius narrow (app team can only affect their own subscription) while the security team governs the guardrails, not the individual rules.

 *For legitimate exceptions (a legacy app that genuinely needs a broad rule temporarily), use an **Azure Policy exemption** with an expiry date, linked to a reviewed and approved security ticket -- never an ad-hoc NSG edit.*

Azure Firewall rules -- platform-owned resource, app-sourced knowledge

The Azure Firewall sits in the connectivity subscription. The platform team owns the firewall configuration. But the *knowledge* of which TCP/UDP flows each application needs lives with the app teams -- and platform engineers can't write rules they don't understand.

The workable pattern here is a **PR-based request flow**, with optional data-driven automation layered on top once the volume of requests justifies it. Most teams should start at level 1 and only graduate to level 2 when the manual review queue becomes a bottleneck.

Level 1 -- PR-based requests (works for any team size):

- App teams submit a PR to the **platform repo** adding their required firewall rules (a new `.tfvars` block, a YAML manifest, or a Terraform `locals` entry).
- `CODEOWNERS` requires platform team review before merge.
- The platform pipeline applies the change to the firewall.
- The app team describes *intent* ("app01 needs HTTPS egress to `api.contoso.com`"); the platform team translates to firewall syntax and validates against policy.

Level 2 -- data-driven automation (scales to many teams):

- Each app team maintains a **firewall request manifest** (YAML/JSON) in their *own* repo, with a versioned schema:

```
# alz-landingzones/corp/app01/firewall-requests.yaml
schema_version: "1"
rules:
  - name: app01-to-api
    direction: outbound
    protocol: TCP
    destination: api.contoso.com
    port: 443
    justification: "REST API dependency -- see ADR-012"
```

- A **platform pipeline** reads manifests from all app repos, validates against policy (no overly broad rules, no blocked destinations), generates firewall rule collections, and applies them.
- The app team owns the intent; the platform team owns the implementation and remains the final enforcement point.

Decision heuristic

Scenario	Who owns the code	Who governs
NSGs / UDRs in app subscriptions	App team	Policy (security team)
Shared modules with safe defaults	Platform team (module)	Module API contract
Central firewall / WAF rules	Platform team (via PR or automation)	CODEOWNERS review
DNS records in central DNS zone	Platform team (via PR) or app team (with DNS sharding)	CODEOWNERS review / zone-level RBAC

The common thread: **separate ownership of intent from ownership of implementation**, and enforce the boundary with policy, CODEOWNERS, or both.

DNS sharding -- reducing blast radius for central DNS

Private DNS zones in a hub subscription are a classic ownership bottleneck: every app team needs records, but a single zone like `privatelink.blob.core.windows.net` is a shared, high-blast-radius resource. One bad PR can break name resolution for every workload in the estate.

DNS sharding splits the problem by delegating ownership at the zone or sub-zone level:

- **Zone-per-workload for private endpoints.** Instead of one monolithic `privatelink.*.core.windows.net` zone, create per-workload zones (or use Azure policy to auto-register into the correct zone). Each zone has its own RBAC, its own state/stack, and its own blast radius.
- **Sub-zone delegation for app-owned records.** Delegate `app01.internal.contoso.com` to the app team's subscription. The platform team owns `internal.contoso.com` and the NS delegation; the app team manages records in their delegated zone autonomously.
- **Separate state/stack per DNS zone.** Even without delegation, putting each zone in its own Terraform state or Deployment Stack means a bad `apply` affects only one zone, not all of DNS.

The tradeoff is more zones to manage and more VNet links to maintain. For estates with fewer than ~10 workloads, a single zone with PR-based governance is simpler. Beyond that, sharding pays for itself in reduced coordination cost and smaller blast radius.

Private link DNS zones -- centralised is usually right

For **private link** DNS zones specifically (`privatelink.*.core.windows.net` , `privatelink.database.windows.net` , etc.), the community consensus is increasingly that a **single centralised set** is the right default:

- **Decentralised private link zones** (one per subscription or region) lead to **NXDOMAIN failures** in hybrid scenarios -- on-premises DNS forwarders can only point to one zone, and cross-subscription zone linking creates operational complexity that few teams sustain.
- **Centralised zones** with platform-team ownership and automated record creation (via Azure Policy `DeployIfNotExists` or the private endpoint's built-in DNS integration) are simpler to operate and

debug.

- For **disaster recovery**, rather than replicating zones cross-region, consider a DR runbook that redeploys the private link zones in the secondary region. Some organisations accept public endpoint fallback during DR as a pragmatic compromise.

DNS sharding remains the right approach for **non-private-link zones** (custom domains, internal app records) where team-level ownership is valuable.

 *From the ALZ Weekly Questions -- Private DNS Zones: Centralized vs Decentralized Most customers who start with decentralised private link DNS zones eventually migrate back to centralised. The NXDOMAIN debugging alone costs more than the perceived flexibility is worth.*

See also [11 manageability](#) for the CODEOWNERS and governance mechanics.

Subscription vending -- repo structure at scale

When your ALZ "vends" (provisions) application landing zone subscriptions, the repo structure for subscription vending deserves its own attention:

One state file per vended subscription, not one giant state. A single Terraform state managing 200 subscriptions means every `plan` touches all 200, every apply locks all 200, and a corruption loses all 200. Instead:

- **Three-layer state architecture:**
 1. **Platform state** -- management groups, policies, hub networking (in the `alz-platform` repo).
 2. **Vending state** -- one state file per vended subscription, each in its own storage account or container. Contains the subscription scaffold: resource groups, RBAC, networking peering, baseline tags.
 3. **Workload state** -- owned by the application team, in their own repo and storage account.
- **Config file per subscription:** Each vended subscription gets a `.tfvars` or `.bicepparam` file in the repo. The pipeline uses `git diff` to build a **matrix** of changed subscriptions and deploys only those in parallel -- not a sequential mega-apply.
- **Protecting vended resources** from workload teams:
 - **Deny assignments via Deployment Stacks** -- the platform team's stack owns baseline resources with `denyDelete` settings.
 - **RBAC at the resource group scope, not subscription scope** -- give app teams Contributor on *their* resource groups, not the subscription.
 - **Deny-action policies with tag-based exclusions** -- prevent deletion of platform-managed resources while allowing app teams to manage their own.

 *From the ALZ Weekly Questions -- Subscription Vending: Repo Structure, Security & Multi-Tenant The git-diff matrix pattern lets you scale to hundreds of subscriptions without pipeline timeouts. Each subscription deploys independently and in parallel.*

Subscription democratisation -- why more subscriptions is usually right

A common early mistake is cramming multiple workloads into a single subscription "to save money." Subscriptions are **free** -- the resources inside them cost money, not the subscription itself. Using fewer subscriptions than needed creates:

- **Larger blast radius** -- a misconfigured RBAC or policy affects all co-hosted workloads.
- **Noisy-neighbour effects** -- one workload's resource locks, API throttling, or quota consumption impacts others.
- **Harder cleanup** -- decommissioning a workload means surgically removing resources rather than deleting a subscription.

The recommended default: **one subscription per workload per environment** (e.g. `app01-prod` , `app01-nonprod`). This gives each workload its own blast radius, quota, and RBAC boundary.

 *From the ALZ Weekly Questions -- [Community ALZ Projects + Right-Sized ALZ](#) Moving resources between subscriptions after the fact is painful and sometimes impossible. Start with the right subscription structure from Day 1.*

Multi-region expansion

When expanding an ALZ estate to a second (or third) region:

- **Start with a minimum of two regions** for your platform landing zone even if workloads are initially single-region. This establishes the pattern early.
- **Multi-region expansion is primarily a networking exercise** -- management groups and policies need minimal changes (possibly a region policy adjustment to allow the new location).
- **Lightweight expansion option:** for non-critical workloads, use **global VNet peering** back to the existing hub rather than deploying a full hub-and-spoke in the new region. This avoids the cost of a second firewall, VPN gateway, and ExpressRoute circuit.
- **Deploy a full hub** only when justified by reliability/resiliency requirements and cost -- not as the default.

 *From the ALZ Weekly Questions -- [Multi-Region ALZ Expansion](#) The cheapest multi-region expansion is global VNet peering to the existing hub. Save the full hub deployment for workloads that genuinely need local egress.*

Anti-patterns

-  **One mega-repo with one giant Terraform state.** A single `terraform apply` that touches every subscription. The blast radius of a bad commit is the entire estate. See [07 state management](#).
-  **One repo per resource group.** Granularity gone mad -- you'll spend more time on repo plumbing than on infrastructure.
-  **"Modules" folder duplicated across 40 repos.** Promote to a shared module repo + registry the moment you have *any* duplication.

- **✗ Mixing application code (Helm charts, app source) in the IaC repo.** Different release cadence, different reviewers, different security model.

Reference layouts

Recommended starter layout (`alz-platform`)

```

alz-platform/
├─ README.md
├─ CODEOWNERS
├─ docs/                # ADRs, diagrams
├─ envs/
│   └─ prod/
│       ├── connectivity/
│       ├── identity/
│       └─ management/
│   └─ nonprod/
│       └─ ...
├─ modules/             # thin wrappers around AVM
├─ policies/            # custom policy definitions (if any)
├─ scripts/
└─ .github/
    ├── workflows/
    └─ CODEOWNERS

```

Recommended starter layout (`alz-landingzones`)

```

alz-landingzones/
├─ README.md
├─ CODEOWNERS           # per-folder ownership
├─ _shared/             # naming, tagging modules
├─ corp/
│   ├── app01-prod/
│   ├── app01-nonprod/
│   └─ app02-prod/
├─ online/
│   └─ ...
└─ .github/workflows/

```

`CODEOWNERS` uses path globs so each landing zone has its own approvers:

```

/corp/app01-* @org/team-app01
/corp/app02-* @org/team-app02
/_shared/    @org/platform-engineering

```

With topology decided and the reference layouts in hand, you have the structural skeleton of your ALZ estate. The remaining chapters layer in the details -- what language to write in those repos, how to version the modules, how to promote code between environments, and how to keep it all secure. The very next question,

tackled in Chapter 02, is which IaC engine those repos will use: a deceptively straightforward choice that carries long-term operational consequences worth thinking through carefully.

References

- Microsoft, *CAF -- Landing zones implementation options*: <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/implementation-options>
 - Microsoft, *ALZ Bicep -- recommended repo structure*: <https://github.com/Azure/ALZ-Bicep/wiki>
 - Hashicorp, *Recommended Terraform code structure*: <https://developer.hashicorp.com/terraform/cloud-docs/recommended-practices>
 - GitHub, *Monorepo vs polyrepo*: <https://github.blog/engineering/architecture-optimization/monorepo-vs-polyrepo/>
 -  ALZ Weekly -- *Subscription Vending: Repo Structure, Security & Multi-Tenant*: <https://www.youtube.com/watch?v=11PmT0t6TUI>
 -  ALZ Weekly -- *Community ALZ Projects + Right-Sized ALZ*: <https://www.youtube.com/watch?v=MznCQbT-EZw>
 -  ALZ Weekly -- *Multi-Region ALZ Expansion*: https://www.youtube.com/watch?v=nii_LF65zyE
 -  ALZ Weekly -- *Private DNS Zones: Centralized vs Decentralized*: <https://www.youtube.com/watch?v=WnrKlflIiwo>
-

[← Back to index](#) · Next → [02 IaC tooling](#)

02 · IaC tooling -- Bicep, Terraform, ARM, Pulumi

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [Summary recommendation](#)
- [The contenders](#)
- [Mixing engines -- when, and how to survive it](#)
- [Example -- same resource in both engines](#)
- [Anti-patterns](#)
- [References](#)

***Decision:** which IaC engine for which layer of the platform? A monolingual estate is simpler; a bilingual estate is sometimes correct.*

[← 01 Repository topology · Index · 03 Modules & registries →](#)

Pick one primary Infrastructure as Code (IaC) engine per Azure Landing Zone (ALZ) layer, and default to the tool your team can operate confidently under pressure. For Azure-only greenfield estates, **Bicep + Deployment Stacks** is now a first-class default; for brownfield, multi-cloud, or Terraform-skilled teams, **Terraform with AzureRM/AzAPI** remains the safer operational bet. Keep platform and landing-zone code on the same engine unless you have a documented boundary and a specific reason to go bilingual. Do not choose on syntax aesthetics alone -- choose on ownership, state tolerance, review quality, and who will maintain the estate at 2 a.m. Read on if you need to justify Bicep versus Terraform, decide whether Pulumi fits your team, or survive a mixed-engine ALZ without drift wars.

How we got here

Bicep and Terraform are both first-class Azure choices now, and the history matters because it explains why state, plan quality, and team familiarity still dominate the decision. The first wave of "Azure as code" was bash scripts wrapping `azure-cli` (then `az`), often committed alongside README sentences like *"run these in order, don't forget to set the subscription"*. Microsoft launched **Azure Resource Manager (ARM) JSON templates** in 2014 -- a genuine declarative model, but the syntax made grown engineers cry. The same year, HashiCorp shipped Terraform; the **AzureRM provider** (2016) gave multi-cloud teams a sane authoring experience and a real `plan` diff, and quickly became the default in enterprises.

That early Terraform lead explains why this decision still cannot be reduced to syntax. Microsoft responded with **Bicep** (public preview 2020, GA 2021) -- essentially "ARM JSON, but lovable" -- and then with **Deployment Stacks** (GA 2024) to close the state-management gap that pushed many teams to Terraform in the first place. **Pulumi** (2018) bet on real programming languages and found a loyal niche but never

displaced Terraform for ops-led teams. Today the honest answer to "Bicep or Terraform?" is "whichever your team will still be operating well at 2 a.m." -- both are first-class on Azure in 2026, so the next step is to decide which lifecycle your team can actually own.

Decision framework

That history leads to a practical rule: use the engine your operators can own for the full lifecycle, and treat mixed engines as an exception that needs an explicit boundary. Answer these questions in order before debating syntax or ecosystem preferences.

1. **Start with skills already on the team.** The single best predictor of long-term success is whether the team already operates the tool well; a team fluent in Terraform will ship a better Terraform ALZ than a Bicep one even if Bicep is "theoretically" simpler for Azure-only.
2. **Choose Bicep for pure Azure with no SaaS configuration.** Pure Azure, no SaaS configuration → **Bicep** is hard to argue against.
3. **Choose Terraform when the estate crosses Azure's boundary.** Any non-Azure resources (GitHub, Entra B2B, Datadog, AWS DR site) → **Terraform**.
4. **Choose Bicep + Deployment Stacks when separate state is unacceptable.** If your operating model cannot accept a separate state store (security review, operational burden, blob/Cosmos availability concerns), Bicep + Deployment Stacks removes that burden entirely.
5. **Use AzAPI when Terraform needs same-day Azure features.** Bicep reaches new Azure features the same day as the ARM REST API; Terraform AzureRM can lag by weeks or months; Terraform AzAPI is same day, at the cost of writing ARM-shaped HCL.
6. **Prefer Terraform when PR review depends on the strongest plan output.** `terraform plan` is more accurate than `az deployment what-if` today, although the gap has narrowed considerably with Bicep's `what-if` improvements.

Quick reference by operating context:

Context	Lead recommendation	Why
Azure-only greenfield	Bicep + Deployment Stacks	Native Azure, no separate state store, same-day ARM API access.
Existing Terraform or multi-cloud estate	Terraform AzureRM/AzAPI	Existing skills, reviewable <code>plan</code> , providers for Azure and non-Azure systems.
Strong software-engineering culture with cross-cloud needs	Pulumi	Real programming languages and normal unit-test frameworks.
Legacy ARM templates	Decompile to Bicep	ARM JSON remains a transport format, not a 2026 authoring target.

If the framework still leaves you genuinely split -- often because different platform layers are owned by teams with different skill sets -- read the tool analysis first, then treat a bilingual estate as an explicit

exception rather than the default.

Summary recommendation

Because the framework is intentionally concise, the chapter needs a summary baseline before it goes deep: use Bicep or Terraform for the core ALZ layers, keep the choice consistent within a layer, and reserve ARM JSON and Pulumi for narrow cases. The short version below sets that baseline before the chapter goes deeper into each tool's strengths and failure modes:

Layer	Recommended	Acceptable alternative
Foundation (mgmt groups, policy assignments)	Bicep + Deployment Stacks or Terraform AzureRM/AzAPI	ARM templates (legacy)
Platform (hub, identity, mgmt)	Bicep or Terraform -- pick one and stick to it	--
Landing zones (workloads)	Same engine as platform	Mixed only with strong justification
Multi-cloud workloads	Terraform or Pulumi	--

The table deliberately puts ownership ahead of aesthetics, because the biggest mistake is choosing a tool for how pleasant it looks in a pull request rather than for *who maintains what tomorrow*. The right answer is whichever your engineers can operate confidently at 2 a.m.

The debate -- is "team skill" really the deciding factor?

The "2 a.m." framing is popular advice, but critics argue it understates real structural differences between the tools.

The Bicep case beyond skills: *Bicep + Deployment Stacks eliminates an entire class of operational problems -- state corruption, state surgery, backend lock contention, `terraform_remote_state` coupling -- that affect even skilled Terraform operators. A mediocre team on Bicep may produce safer infrastructure than a good team on Terraform, simply because fewer things can go catastrophically wrong at 2 a.m.*

The Terraform case beyond skills: *`terraform plan` output is genuinely more reliable for code review than `az deployment what-if` (which still has known gaps). Terraform's ecosystem of third-party providers means you manage GitHub repos, Datadog monitors, PagerDuty schedules, and Azure resources in one workflow -- Bicep can't. And for teams already running Terraform, migration cost is real and rarely justified by Bicep's simplicity alone.*

Where the industry stands (2026): *Azure-only greenfield projects increasingly default to Bicep; brownfield and multi-cloud estates stay on Terraform. The "skills" heuristic is a useful tiebreaker, but pretending the tools are interchangeable undersells the architectural implications of each choice.*

That disagreement is useful rather than academic, because it frames the tool-by-tool analysis that follows: each engine has a different failure mode, and your choice should make that failure mode explicit before you standardise on it.

The contenders

With the recommendation baseline established, the real defaults for ALZ are Bicep and Terraform; ARM is now legacy authoring, Pulumi remains a specialist choice, and accelerators are scaffolding that help you start without deciding the engine for you. Looking at the contenders in that order makes the tradeoffs easier to compare, because the first two are tools you can run an estate on, while the latter cases are constraints around migration, team culture, and bootstrap.

Bicep

Bicep is the strongest fit for Azure-only estates that value native API coverage and do not want to manage a separate state file. It is Microsoft's first-party domain-specific language (DSL), and it transpiles (compiles from one high-level language to another) to ARM JSON.

Strengths

- Native Azure -- every resource and API version is available *day one*.
- No state file to manage. Azure Resource Manager is the source of truth.
- **Deployment Stacks** (GA) give a Terraform-like "what is managed by this deployment" model with `denySettings` for drift protection.
- Tight integration with `az deployment` and AVM (Azure Verified Modules).
- Excellent VS Code experience: type checking, intellisense, decompile from ARM.

Weaknesses

- Azure-only. Useless for AWS/GCP, third-party SaaS, GitHub, Datadog, etc.
- Looping/conditionals are less expressive than HCL (HashiCorp Configuration Language) or general-purpose languages.
- Smaller community ecosystem of modules (though AVM closes the gap).
- `what-if` is powerful but has rough edges with complex resources (RBAC role assignments, child resources). Validate empirically.

Terraform (with AzureRM and/or AzAPI providers)

Where Bicep optimises for Azure-native simplicity, Terraform is the safer choice when you need multi-cloud reach, third-party providers, or the strongest reviewable plan workflow. It remains the de-facto multi-cloud standard, which is why many brownfield ALZ estates stay with it even when Bicep would be technically sufficient for the Azure resources alone.

Strengths

- Huge ecosystem -- providers for everything (Azure, AWS, GCP, GitHub, Azure DevOps, AAD/Entra, Datadog, Cloudflare...).
- `plan` produces a clear, reviewable diff -- gold standard for PR workflows.
- HCL is mature and expressive (functions, dynamic blocks, `for_each`).
- Strong testing story (terratest, native `terraform test`).

- **AzAPI provider** closes the "new Azure feature" gap by calling the ARM REST API directly when AzureRM lags.

Weaknesses

- **State management is your problem.** Backends, locking, secrets in state, state surgery -- all real operational concerns. See [07 state management](#).
- AzureRM provider sometimes lags new Azure features (mitigated by AzAPI).
- OpenTofu fork (post-BSL licence change) is now production-viable; choosing between Terraform and OpenTofu is a fresh decision in 2026.

 **BSL (Business Source Licence)** -- in August 2023 HashiCorp relicensed Terraform from the permissive MPL 2.0 to the BSL 1.1, which restricts competitive commercial use. **OpenTofu** is the community fork that continues under the original open-source licence.

ARM (JSON)

ARM JSON is the original Azure declarative format, but in a modern ALZ it should remain a transport or migration format rather than hand-authored platform code. You still benefit from its universality because Bicep compiles to it and existing deployments can be decompiled from it, yet that does not make it a pleasant or defensible authoring target in 2026.

Strengths

- Universally supported. Always works, no tooling required.
- Useful as a *transport format* -- Bicep compiles to it; you can decompile existing deployments.

Weaknesses

- Authoring ARM JSON by hand in 2026 is an anti-pattern. Use Bicep.

Pulumi

Pulumi belongs in the conversation only when your platform team has the software-engineering discipline to review infrastructure as program code. It gives you IaC in real programming languages such as TypeScript, Python, Go, and C#, which is powerful when the team can keep infrastructure intent visible through ordinary code review and dangerous when the code becomes an application in disguise.

Strengths

- Real language → unit tests with normal frameworks, complex logic, IDE.
- Cross-cloud with one mental model.

Weaknesses

- Smaller community than Terraform.
- Conflates infrastructure and program logic -- easy to write code that's hard to review as a "diff".
- Best suited to teams with a strong software-engineering culture; less ideal for Ops-background platform teams.

ALZ accelerators

ALZ accelerators are best treated as a bootstrap mechanism for the repo and pipeline, after which you take ownership of the generated codebase. They are not an engine choice by themselves; instead, they give you a production-grade CI/CD pipeline, state storage or Deployment Stack parameters, managed identities, and an opinionated folder structure so you do not start from a blank repo. Understanding what each accelerator does -- and what it *doesn't* do -- matters because the accelerator's output becomes the codebase you will maintain for years.

Key terms

ALZ accelerator -- an automation package that provisions the scaffolding (repos, pipelines, identities, state storage) needed to deploy an Azure Landing Zone via IaC. The accelerator is a bootstrap; the AVM modules it references are the implementation.

Bootstrap phase -- the one-time setup that creates the version control repo, CI/CD pipelines, managed identities (with federated credentials for OIDC), and Terraform state storage accounts. All accelerators use a PowerShell module (`ALZ-PowerShell-Module`) for this step, regardless of whether the resulting IaC is Bicep or Terraform.

ALZ Bicep accelerator (AVM-based)

The Bicep accelerator replaces the classic `Azure/ALZ-Bicep` repo, which is entering extended support and will be archived in February 2027. That matters because new Bicep ALZ implementations should start from the AVM-based path rather than from the classic repository.

Aspect	Detail
Input	Interactive questionnaire via ALZ PowerShell module: target management-group hierarchy, connectivity model (hub-spoke or vWAN), regions, policy defaults, VCS choice (GitHub / Azure DevOps).
Bootstrap	Terraform provisions: a Git repo, CI/CD pipelines (GitHub Actions or ADO Pipelines), managed identities with OIDC federation, and storage accounts for Deployment Stack parameters.
Output	A ready-to-run repo containing AVM Bicep pattern modules (≈16 resource + 3 pattern modules), YAML-driven configuration for management groups, policies, and connectivity, plus pipeline definitions that deploy via Deployment Stacks.
Day-2 workflow	Edit YAML config or Bicep parameters → PR → pipeline runs <code>what-if</code> → merge → pipeline deploys via <code>az stack sub create</code> .

ALZ Terraform accelerator (AVM-based)

The Terraform accelerator follows the same pattern for Terraform estates: it replaces the classic `Azure/terraform-azurerm-caf-enterprise-scale` module, which is entering extended support and will be archived in August 2026. If you are starting today, the AVM-based accelerator is the path you should standardise on.

Aspect	Detail
Input	Same PowerShell questionnaire: hierarchy, connectivity model, regions, VCS. Additionally: backend choice (azurerm storage), Terraform/OpenTofu version preference.
Bootstrap	Terraform provisions: a Git repo, CI/CD pipelines, managed identities with OIDC, a storage account for state, and a starter Terraform root module consuming AVM modules.
Output	A repo with a root module referencing AVM Terraform resource and pattern modules, <code>.tfvars</code> per environment, and pipeline definitions that run <code>terraform plan</code> on PR and <code>terraform apply</code> on merge.
Day-2 workflow	Edit <code>.tf</code> / <code>.tfvars</code> → PR → pipeline runs <code>plan</code> + policy checks → merge → pipeline runs <code>apply</code> .

ALZ Portal accelerator

Aspect	Detail
Input	Browser-based wizard at aka.ms/alz/portal : click-through selections for management groups, policies, connectivity, and identity.
Bootstrap	ARM deployment directly from the portal -- no local tooling required.
Output	Deployed Azure resources (management groups, policies, hub network) in your tenant. No repo, no pipeline, no IaC artifacts.
Day-2 workflow	None -- portal-deployed resources are not under source control.

The portal accelerator is useful for **demos, proof-of-concept, and learning** the ALZ architecture. It is **not suitable for production GitOps** because there is no IaC to review, version, or roll back. If you start here, plan to re-deploy via Bicep or Terraform before going live.

ALZ PowerShell module

The PowerShell module (`Azure/ALZ-PowerShell-Module`) is not a standalone accelerator -- it is the **bootstrap engine** used by both the Bicep and Terraform accelerators. It collects configuration via an interactive questionnaire, then invokes Terraform to provision the VCS repo, pipelines, identities, and state storage. You can also run it non-interactively with a parameter file for repeatable bootstraps.

What all accelerators share

- **Opinionated defaults aligned to CAF** -- management-group hierarchy, default policy assignments, hub network topology.
- **AVM modules under the hood** -- the accelerators don't invent their own resource definitions; they compose AVM resource and pattern modules.
- **You own the output.** The accelerator is a one-time scaffold. Once the repo exists, you maintain it -- updating AVM module versions, customising policies, adding landing zones. The accelerator doesn't "phone home" or auto-update.

- **Interactive mode** -- Running `Deploy-Accelerator` with no parameters launches a guided, interactive questionnaire that walks you through every decision. No need to craft a parameter file upfront.

Because that output becomes your long-lived platform code, you should **fork or wrap** the accelerator output rather than consume it raw. The module ownership and registry patterns covered in [03 modules & registries](#) are what let you keep that scaffold maintainable after the bootstrap phase is over.

 *From the ALZ Weekly Questions -- [How to use AVM in ALZ, Bicep or Terraform?](#) As of early 2025, AVM is the **only** recommended module set for new ALZ deployments. The Bicep AVM accelerator now uses Deployment Stacks natively, giving Bicep parity with Terraform's state-based lifecycle management.*

SMB and right-sized scenarios

Not every organisation needs the full three-subscription, multi-region ALZ layout. The accelerator now ships **SMB (small and medium business) scenarios** that drastically reduce Day-1 cost and complexity:

Setting	Full ALZ	SMB ALZ
Subscriptions	3+ (management, connectivity, identity)	2 (management + connectivity combined, plus workload)
Firewall SKU	Standard or Premium	Basic
DDoS Protection	On by default	Off by default
Regions	Multi-region recommended	Single region start

The SMB scenario uses the **same modules and codebase** -- only the `.tfvars` / `.bicepparam` defaults differ. This means you can grow in-place: upgrading from Basic to Standard firewall, enabling DDoS, or adding a second region is a parameter change, not a repo migration.

 *From the ALZ Weekly Questions -- [ALZ for SMB](#) The SMB scenario was purpose-built for organisations that need governance guardrails without enterprise-level cost. Start small, grow in-place.*

The ALZ library tool (`alzlibtool`)

The accelerator story also depends on the ALZ library, which is a **data layer** separated from business logic -- a set of JSON/YAML files that define management-group archetypes, policy sets, and role assignments. The `alzlibtool` Go module exposes three key commands:

Command	Purpose
<code>alzlib check</code>	Validates the library files against the schema -- catches errors before deployment.
<code>alzlib generate</code>	Produces the JSON artefacts consumed by the Terraform provider or Bicep modules.
<code>alzlib document</code>	Auto-generates human-readable documentation of the entire policy landscape.

The same Go module powers both the Terraform ALZ provider and the Bicep generation pipeline. The library is **composable**: you can layer ALZ base + Sovereign Landing Zone (SLZ) + your own local overrides.

 *From the ALZ Weekly Questions -- [When to use SLZ over ALZ? + alzlibtool](#) The `alzlibtool` is how both Bicep and Terraform stay in sync with the canonical ALZ policy set. Treat the library as data, the tool as compiler, and your IaC as consumer.*

Understanding the Bicep file structure

When you move from the library data into the generated Bicep code, the file structure can look more fragmented than you might expect: the accelerator output contains multiple `.bicep` / `.bicepparam` file pairs rather than a single monolithic template. This split is not accidental, because ARM has a **4 MB deployment payload limit**, and a full ALZ deployment exceeds it. The accelerator therefore separates the deployment into files for management groups, policies, connectivity, and other domains, while a future Bicep feature -- **extendable parameters** -- should eventually allow a single parameter file to feed multiple Bicep files and reduce today's parameter repetition.

 *From the ALZ Weekly Questions -- [Understanding ALZ Bicep File Structure](#) The Bicep modules can also be consumed **standalone**, without the accelerator. If you only need the connectivity module, reference it directly from the AVM registry.*

Migrating from CAF-Enterprise-Scale

That same archive timeline becomes more concrete if you are already running the classic `Azure/terraform-azurerm-caf-enterprise-scale` module, which reaches its archive date on **1 August 2026**. A purpose-built **Golang state migration tool** helps you move in two phases:

1. **Phase 1 -- Connectivity and management resources** (VNets, firewalls, Log Analytics). The tool reads your existing state, maps resources from CAF-ES module addresses to AVM module addresses, and generates Terraform `import` blocks.
2. **Phase 2 -- Management groups and policies**. These are trickier because the policy structure changed between CAF-ES and AVM. The tool produces an issues CSV listing resources that need manual attention.

The migration tool works from **any CAF-ES version** -- you don't need to be on the latest before migrating. However, older versions produce more entries in the issues CSV, which is why the most pragmatic ALZ-team guidance is to import only resources you cannot easily delete and recreate, such as ExpressRoute circuits, firewalls with BGP sessions, or DNS zones with live records. For management groups and policies, a clean re-deploy is often easier to reason about, especially when Deployment Stacks can handle the cutover.

 *From the ALZ Weekly Questions -- [Migrating from CAF-Enterprise-Scale to AVM](#) The migration tool is also useful outside ALZ -- it can recover or restructure any Terraform state file by mapping old module addresses to new ones.*

Azure Migrate agent for platform landing zones

After the migration path, the newest accelerator-adjacent feature is the preview **Azure Migrate agent**, which builds on top of the accelerator rather than replacing it. Available in the Azure portal and VS Code, it lets you describe your desired landing zone in natural language and generates the accelerator configuration:

- Grounded on CAF and ALZ documentation -- recommendations are not hallucinated.
- Uses an **MCP server** for customisation -- you can extend it with your own policies or naming conventions.
- Includes **cost estimation** so you can see the monthly impact before deploying.
- Currently **Terraform-only**; Bicep support is planned.

 *From the ALZ Weekly Questions -- [Azure Migrate Agent Preview](#) and [Azure Migrate Agent Deep Dive](#) The Migrate agent is ideal for initial exploration and configuration generation. For production deployments, always review the generated output before applying.*

With all the options on the table, the practical question is which combination actually fits your team and organisation. That question becomes sharper when the answer appears to involve more than one engine, because mixed estates need boundaries as much as they need tooling.

Mixing engines -- when, and how to survive it

Avoid bilingual estates unless one of a few specific situations applies, and if you do mix engines, enforce hard ownership boundaries from the start. The previous section showed that Bicep and Terraform are both defensible defaults, but that does not mean you should let them manage the same estate casually.

Bilingual estates usually appear for a practical reason rather than as an architectural ideal:

- Foundation/policy is in **Bicep** (Microsoft accelerator), workloads are in **Terraform** because app teams already know it.
- Platform is in **Terraform** (multi-cloud DR), but a specific workload uses **Bicep** because it needs a brand-new Azure preview feature.

When one of those reasons is strong enough to justify the extra operating model, the rules are simple and non-negotiable:

1. **One engine per resource.** Never let two engines manage the same resource -- drift wars guaranteed.
2. **Clear boundary at the resource group or subscription level.** Make it impossible for the boundary to be ambiguous.
3. **Shared naming/tagging convention.** Both engines must produce identical outputs.
4. **Document it loudly** in the top-level README -- every new joiner asks "why?".

Those rules are easier to understand when you compare the authoring and deployment workflow side by side, which is why the next section shows the same small resource in both engines without pretending the example alone should decide the estate standard.

Example -- same resource in both engines

Use the example as a syntax and workflow comparison, not as a reason to switch engines by itself. The same simple deployment -- a virtual network (VNet) with two subnets -- looks familiar in both tools, but the

operational questions around state, plan output, and API coverage are different enough that the code sample is only a starting point.

Bicep

```
// modules/network/main.bicep
param location string = resourceGroup().location
param vnetName string
param addressSpace string
param subnets array

resource vnet 'Microsoft.Network/virtualNetworks@2024-05-01' = {
  name: vnetName
  location: location
  properties: {
    addressSpace: { addressPrefixes: [ addressSpace ] }
    subnets: [for s in subnets: {
      name: s.name
      properties: { addressPrefix: s.prefix }
    }]
  }
}

output vnetId string = vnet.id
```

Deploy with:

```
az stack sub create \
  --name plat-net-prod \
  --location swedencentral \
  --template-file main.bicep \
  --deny-settings-mode denyDelete \
  --action-on-unmanage deleteAll
```

Terraform

```
# modules/network/main.tf
variable "location"      { type = string }
variable "vnet_name"     { type = string }
variable "address_space" { type = string }
variable "subnets"      { type = list(object({ name = string, prefix = string })) }

resource "azurerm_virtual_network" "this" {
  name            = var.vnet_name
  location        = var.location
  resource_group_name = var.rg_name
  address_space   = [var.address_space]
}

resource "azurerm_subnet" "this" {
  for_each      = { for s in var.subnets : s.name => s }
  name          = each.value.name
  resource_group_name = var.rg_name
  virtual_network_name = azurerm_virtual_network.this.name
  address_prefixes   = [each.value.prefix]
}

output "vnet_id" { value = azurerm_virtual_network.this.id }
```

Deploy with:

```
terraform init -backend-config=...
terraform plan -out=tfplan
terraform apply tfplan
```

The Bicep version is shorter, while the Terraform version gives you a richer plan diff and works against AWS tomorrow. Neither is "better" in isolation; they answer different questions, and the failures in the next section usually happen when teams forget which question they were trying to answer.

Anti-patterns

The failures to avoid are choosing tools for isolated features, hand-authoring legacy formats, or letting multiple engines manage the same resources. By this point in the chapter, those mistakes should look familiar: each one breaks the ownership, lifecycle, or boundary rule that made the tool choice defensible in the first place. The recurring offenders:

- **✗ Authoring ARM JSON by hand.** Use Bicep and `az bicep decompile` to migrate any inherited templates.
- **✗ Two engines managing overlapping resources.** Drift wars; pick a side.
- **✗ Choosing a tool for a single feature.** "I need Terraform because of one Datadog dashboard" -- use the Datadog provider in a *separate* repo rather than rewriting your platform.
- **✗ Pulumi without a programming culture.** It will rot into spaghetti.

-  **Adopting community ALZ alternatives without due diligence.** Several "vibe-coded" open-source ALZ implementations look polished but lack testing, issue triage, and long-term support. Before adopting any community project, check: contributor count, issue history (triaged regularly?), end-to-end test coverage, and whether the project has survived more than one Azure API breaking change.

 *From the ALZ Weekly Questions -- Community ALZ Projects + Right-Sized ALZ The official ALZ repository has handled ~3 000 issues, triages twice weekly, and runs end-to-end tests. Evaluate any alternative against that baseline.*

With the toolchain chosen, the estate has a shape from 01 repository topology and a language from this chapter. The missing link is reuse: how do you avoid writing the same VNet module for each team that needs one, and how do you update it across 40 consumers without a week of coordinated PRs? Chapter 03 covers the module and registry architecture that turns "you have IaC" into "you have a maintainable IaC estate". The take-home from this chapter is simple: pick the engine your team owns confidently, keep it consistent within a layer, and put any deviation in writing.

References

These are the source materials behind the tool recommendations and accelerator notes in this chapter.

- Microsoft, *Bicep documentation*: <https://learn.microsoft.com/azure/azure-resource-manager/bicep/>
- Microsoft, *Deployment stacks*: <https://learn.microsoft.com/azure/azure-resource-manager/bicep/deployment-stacks>
- Hashicorp, *AzureRM provider*: <https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs>
- Hashicorp, *AzAPI provider*: <https://registry.terraform.io/providers/Azure/azapi/latest/docs>
- Azure, *Verified Modules*: <https://aka.ms/avm>
- Azure, *ALZ-Bicep (Classic -- entering extended support)*: <https://github.com/Azure/ALZ-Bicep>
- Azure, *ALZ Terraform (Classic -- entering extended support)*: <https://github.com/Azure/terraform-azurerm-caf-enterprise-scale>
- Azure, *ALZ accelerator (AVM-based, current)*: <https://azure.github.io/Azure-Landing-Zones/accelerator/>
- Azure, *ALZ Terraform migration guide*: <https://aka.ms/alz/tf/migrate>
- OpenTofu: <https://opentofu.org/>
-  ALZ Weekly -- *How to use AVM in ALZ, Bicep or Terraform?*: https://www.youtube.com/watch?v=ry39tWr_SXc
-  ALZ Weekly -- *When to use SLZ over ALZ? + alzlibtool*: <https://www.youtube.com/watch?v=r8h7F6JIQw>
-  ALZ Weekly -- *Understanding ALZ Bicep File Structure*: <https://www.youtube.com/watch?v=sPA3YWkQ-4s>
-  ALZ Weekly -- *Migrating from CAF-Enterprise-Scale to AVM*: <https://www.youtube.com/watch?v=DSBWjQIVpSs>

-  ALZ Weekly -- *Community ALZ Projects + Right-Sized ALZ*: <https://www.youtube.com/watch?v=MznCQbT-EZw>
 -  ALZ Weekly -- *ALZ for SMB*: <https://www.youtube.com/watch?v=cyLhLJEYIkU>
 -  ALZ Weekly -- *Azure Migrate Agent Preview*: <https://www.youtube.com/watch?v=kFS9lNPuXxM>
 -  ALZ Weekly -- *Azure Migrate Agent Deep Dive*: <https://www.youtube.com/watch?v=ODaFlsja308>
-

[← 01 Repository topology](#) · [Index](#) · [03 Modules & registries](#) [→](#)

03 · Modules & registries -- composition, versioning, distribution

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [The three-tier module model](#)
- [Azure Verified Modules \(AVM\)](#)
- [Where do *your* \(tier-2\) modules live?](#)
- [Versioning policy](#)
- [Module design checklist](#)
- [Naming and namespacing](#)
- [Anti-patterns](#)
- [References](#)

Decision: *where do reusable modules live, how are they versioned, and how do consumers pin to a known-good revision?*

[← 02 IaC tooling](#) · [Index](#) · [04 Branching & environments](#) →

For most Azure Landing Zone (ALZ) Infrastructure as Code (IaC) estates, the safest module model is to use Azure Verified Modules (AVM) resource modules as tier 1, publish your own tier-2 pattern modules with Semantic Versioning (SemVer), and require every consumer to pin a known-good version. That combination gives you one place to enforce tags, diagnostics, security defaults, and naming while keeping landing-zone code mostly composition glue. Prefer a dedicated module repo and private registry once more than a few teams consume the modules; Git tags are acceptable for simple Terraform-only estates, but floating branches are never acceptable. Treat AVM as the default, not a mandate: raise upstream pull requests for gaps and keep mature custom tier-1 modules only when they are already tested and maintained. If you own platform modules, review workload compositions, or need a versioning model that makes ALZ changes reproducible across environments, the rest of the chapter shows how those choices fit together.

How we got here

Module registries exist because copy/paste reuse and Git-source shortcuts failed at enterprise scale, and AVM now provides the tier-1 baseline this chapter builds on. The first Terraform projects had no concept of modules at all, so engineers copy-pasted `.tf` files between repos and patched the differences by hand. When that became unbearable, the community tried **Git submodules** (universally hated for confusing

checkouts), then `git::` sources pinned to commits (better, but no SemVer), then the **public Terraform Registry** in 2017 for open-source modules.

As reuse became more formal, private registries followed, either via Terraform Cloud or self-hosted Open Container Initiative (OCI) and HTTP backends. Bicep launched without modules in 2020, added them in 2021, then added the **br: registry protocol** so you could `bicep publish` to Azure Container Registry (ACR). The breakthrough came in 2023 with AVM: a single, Microsoft-curated catalogue of Bicep *and* Terraform modules with consistent inputs, baked-in Well-Architected Framework (WAF) defaults, and a real maintenance commitment. Before AVM, every consultancy shipped its own subtly broken "vnet module"; after AVM, that duplication is a smell, which is why the decision framework starts by asking how much module structure you actually need.

Decision framework

With that history in mind, answer these questions in order before you add another tier or registry. If a choice would make ownership, versioning, or release automation ambiguous, simplify the module model first; the deeper sections below are there to explain the defaults, not to justify unnecessary complexity.

- 1. How many tiers of modules do you actually need?** Use AVM or equivalent resource modules as tier 1, add tier-2 wrappers only where enterprise conventions need to be enforced, and keep tier 3 as workload composition glue.
- 2. Where do your tier-2 modules live?** Prefer a dedicated `alz-modules` repo for shared enterprise patterns; keep modules beside platform code only while one small platform team owns all consumers.
- 3. How are modules distributed and versioned?** Git tags are fine for simple Terraform consumption; use ACR/OCI or a private Terraform registry once Bicep or multiple teams need the same registry UX as AVM.
- 4. How do consumers pin module versions?** Pin foundation and platform repos to exact versions, let workload repos use minor ranges for patches, and never point any environment at `main`, `latest`, or a branch.
- 5. When do you fork an AVM module instead of raising a PR upstream?** Do not fork by default; raise upstream PRs for gaps, and keep custom tier-1 modules only where they are already mature, tested, and actively maintained.

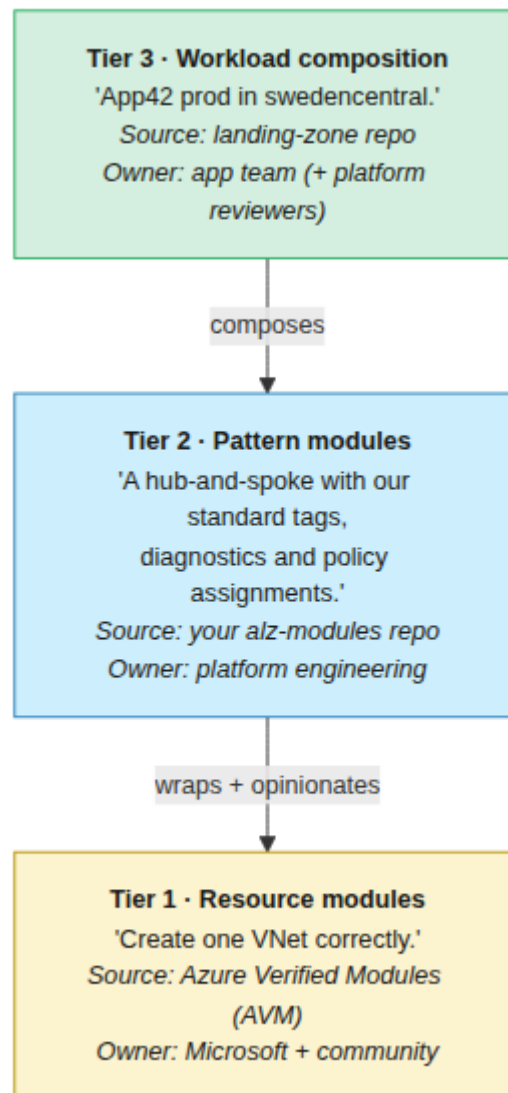
Quick reference:

Question	Recommended default
Module tiers	Tier 1 = AVM/resource modules; tier 2 = enterprise patterns; tier 3 = workload composition.
Tier-2 location	Dedicated <code>alz-modules</code> repo once modules are shared beyond one platform team.
Distribution	Git tags for simple Terraform estates; ACR/OCI or private registry for broader consumption.
Pinning	Exact for foundation/platform, minor range for workloads, never floating branches.
AVM gaps	Contribute upstream unless a mature custom tier-1 module already exists.

The full analysis follows the same order: first you separate the tiers, then you decide how AVM, repositories, registries, versioning rules, and design standards make those tiers safe to consume.

The three-tier module model

Keep resource modules, enterprise pattern modules, and workload composition separate, because conflating them is the fastest way to lose governance consistency. A mature ALZ implementation has **three tiers** of modules, and each tier exists to protect a different ownership boundary.



- **Tier 1** is *consumed*, never modified. If AVM is missing something, contribute upstream.
- **Tier 2** is the *opinionation layer*. This is where your enterprise conventions (tags, log analytics workspace, diagnostic settings, and network security group (NSG) defaults) get baked in *once*.
- **Tier 3** is just glue -- picking pattern modules and parameterising them.

If your landing-zone code is calling AVM resource modules directly, you have no place to enforce your enterprise conventions, and every team will reinvent them. That is why tier 2 exists: it gives you a controlled

opinionation layer while still letting tier 1 remain a maintained resource catalogue. The good news for that first tier is that much of the resource-module work has already been done for you.

Azure Verified Modules (AVM)

Treat AVM as the default tier-1 source, but wrap it with your own tier-2 patterns before you expose it broadly to enterprise consumers. [AVM](#) is Microsoft's official, supported library of Bicep and Terraform modules, and as of 2026 it covers the vast majority of common Azure resources. That coverage is what lets tier 2 focus on enterprise opinionation instead of reimplementing every storage account, virtual network, or key vault from scratch.

That default matters because AVM resource modules give you a maintained baseline instead of another custom library to carry forever. The practical benefits are the same ones the tier model needs from tier 1:

- Maintained by Microsoft + community; security & WAF aligned by default.
- Predictable inputs/outputs across resources.
- Diagnostic settings, customer-managed keys, role-based access control (RBAC), and private endpoints -- all parameterised consistently.
- Versioned in Microsoft Container Registry (MCR) for Bicep / Terraform Registry.

Key terms

SemVer (Semantic Versioning) -- a versioning scheme (`MAJOR.MINOR.PATCH`) where each component signals the type of change: major = breaking, minor = new features, patch = bug fixes.

Conventional Commits -- a commit-message convention (e.g. `feat:` , `fix:` , `chore:`) that enables tools like `release-please` to automate changelog generation and version bumping.

Microsoft Container Registry (MCR) -- Microsoft's public OCI registry (`mcr.microsoft.com`) used to distribute container images and Bicep modules.

OCI (Open Container Initiative) -- an industry standard for container image formats and distribution; registries that speak OCI can also host IaC modules.

DRY (Don't Repeat Yourself) -- a software principle: every piece of knowledge should have a single, authoritative representation in the codebase.

WAF (Well-Architected Framework) -- Microsoft's set of design principles for building reliable, secure, cost-efficient and performant Azure workloads.

However, AVM *pattern* modules should not become your production enterprise standard without a wrapper. They are reference implementations, useful as scaffolding and comparison material, whereas your tier-2 modules are where your naming, logging, policy, and security defaults become enforceable.

The debate -- must every estate adopt AVM?

AVM is positioned as the default tier-1 source above, but not everyone agrees.

The case for custom tier-1 modules: Organisations with mature, battle-tested module libraries (some pre-dating AVM by years) argue that migrating to AVM for standardisation's sake is cost without benefit. AVM modules are deliberately generic -- they expose every parameter to stay broadly useful, which means your team still writes opinionated wrappers on top. If you already have those opinionated modules, AVM adds a layer without removing one. Additionally, AVM coverage, while broad, is not complete: niche resources or preview features may lag, forcing you to maintain custom modules alongside AVM anyway.

The case for AVM: Starting from AVM means you inherit ongoing security patches, WAF alignment updates, and community contributions without maintaining them yourself. For greenfield estates or teams without a mature module library, building custom tier-1 modules from scratch is reinventing work Microsoft already maintains. AVM's predictable interface (`diagnostic_settings` , `private_endpoints` , `role_assignments`) also makes onboarding faster across teams.

Pragmatic middle ground: Treat AVM as default, not mandatory. Use it where it covers your needs and wrap it where it doesn't. Don't force-migrate stable custom modules that are actively maintained and well-tested -- that's disruption for compliance's sake, not engineering's.

The two examples below show what that default looks like in code: you consume the published module and pin the exact version rather than tracking an implicit latest. The syntax differs between Bicep and Terraform, but the contract is the same.

Bicep AVM example

```
module storage 'br/public:avm/res/storage/storage-account:0.14.3' = {
  name: 'sa-${uniqueString(resourceGroup().id)}'
  params: {
    name: 'stplatlogs${uniqueString(resourceGroup().id)}'
    location: location
    skuName: 'Standard_GRS'
    publicNetworkAccess: 'Disabled'
    tags: tags
  }
}
```

The important detail is `0.14.3` , because you should always pin the module version even in examples. Terraform makes the same rule visible through the `version` argument.

Terraform AVM example

```
module "storage" {
  source = "Azure/avm-res-storage-storageaccount/azurerm"
  version = "0.6.4"

  name = "stplatlogs${random_string.s.result}"
  resource_group_name = var.rg_name
  location = var.location
  account_replication_type = "GRS"
  public_network_access_enabled = false
  tags = var.tags
}
```

Once tier 1 is pinned and predictable, the next decision is where your opinionated tier-2 modules live and how consumers find them.

Where do *your* (tier-2) modules live?

Publish enterprise pattern modules from a dedicated module repo and registry once more than a few teams consume them. AVM handles tier 1, but the tier-2 pattern modules that bake in your enterprise conventions are yours to build, release, and support, so their home should make ownership and consumption obvious. In practice, three patterns show up repeatedly.

A) Dedicated `alz-modules` repo + Git tag versioning

A dedicated `alz-modules` repo, such as `github.com/<org>/alz-modules`, is the simplest place to start because the repository boundary matches the ownership boundary. You version it with SemVer Git tags such as `v1.4.2`, and Terraform consumers can reference a tagged module path such as `git::https://github.com/<org>/alz-modules.git//network/hub?ref=v1.4.2`. The tradeoff is that Git-source modules require network access from runners and do not give Bicep the same consumption model, because Bicep cannot use `module x 'git::https://...'`; if Bicep consumers matter, you need a registry. That limitation is what usually pushes growing estates toward the next pattern.

B) Private OCI registry (Azure Container Registry)

A private OCI registry, usually Azure Container Registry, becomes the more durable option once you have several teams or any serious Bicep adoption. You publish Bicep modules with `bicep publish`, publish Terraform modules to a private registry such as Terraform Cloud, Azure DevOps Artifacts, JFrog, or an OCI registry with the `tfe` provider, and consumers reference a versioned artifact:

```
module hub 'br:contoso.azurecr.io/bicep/modules/network-hub:1.4.2' = { ... }
```

This pattern gives consumers the same `br:` experience they already see with AVM, plus stronger access control and an audit trail per pull. It does add ACR and authentication setup, along with a modest operational

cost, but that cost is usually justified as soon as modules become a shared platform product rather than a single-team convenience.

C) Public registry fork

For Terraform, you can also publish to the public Terraform Registry if your modules are open source, although that is rare for enterprise ALZ modules because the patterns usually encode internal conventions. In contrast to a private registry, the public option optimises for community reuse rather than internal control. Whichever distribution mechanism you choose, it is only as trustworthy as the versioning discipline behind it, which is why the next section treats versioning as part of the module architecture rather than as release hygiene.

Versioning policy

Use SemVer, automate releases, and require pinned consumer versions, because a deployment should never consume a floating branch. Since IaC modules change live infrastructure rather than just application binaries, you need to define explicitly what each `MAJOR.MINOR.PATCH` bump means for a module. The table below gives reviewers and release automation the same vocabulary:

Bump	Triggered by
MAJOR	Breaking change: input renamed/removed, output renamed, resource type changed in a way that requires manual migration, default behaviour changes that would alter an existing deployment.
MINOR	Backwards-compatible feature: new optional input, new optional resource, new output.
PATCH	Bug fix, doc change, no behavioural change for existing callers.

You enforce that contract with release automation and compatibility tests, rather than with reviewer memory.

- **Conventional Commits** + `release-please` / `semantic-release` to automate tag creation.
- A "compat" test in continuous integration (CI) that re-applies the module against a saved fixture and fails on unexpected diffs.

Pinning strategy for consumers

The same versioning contract has to be visible in consumer repositories, because a good release process in the module repo cannot protect an environment that points at a moving target.

- **Foundation/platform repos:** pin to **exact version** (= `1.4.2`).
- **Workload repos:** pin to **minor** (`~> 1.4`) so they pick up patches automatically. Major bumps require an explicit pull request (PR).
- **Never** float to `main` / `latest` in any environment, including dev. Reproducibility is non-negotiable.

Rollout choreography

Once consumers are pinned, a tier-2 module bump becomes an intentional rollout instead of an accidental drift event. Bumping a module that 40 landing zones consume needs a process:

1. Cut module release `v1.5.0`.
2. Open automated pull requests (PRs) to all consumers via Renovate / Dependabot.
3. CI in each consumer runs `plan / what-if`. Reviewers see exactly what would change in their workload.
4. Merge happens at the consumer's pace, within a service-level agreement (SLA), such as 30 days for minor releases and 90 days for major releases.
5. Deprecation: the module repo's CHANGELOG flags removal in `v2.0.0`; an automated check in the platform pipeline reports stragglers.

That choreography only works reliably if the modules themselves are small enough, documented enough, and tested enough for consumers to trust the release notes. The design checklist turns that expectation into something you can review.

Module design checklist

Make pattern modules small, opinionated, documented, tested, and release-ready before other teams depend on them. The checklist below is intentionally practical: each item reduces the amount of tribal knowledge a consumer needs before they can trust a module release.

A pattern module should:

- ☐ Take **opinionated defaults** (tags, diagnostic settings, log analytics workspace ID, private endpoints) so callers can't forget them.
- ☐ Expose a **small input surface** (5-15 parameters, not 50). If you need 50, you have multiple modules masquerading as one.
- ☐ Output **everything a downstream module might need** (IDs, names, principal IDs of identities) -- outputs are cheap, refactoring is not.
- ☐ Include a **README.md** auto-generated from the schema (`terraform-docs` , `bicep-docs` , or AVM templates).
- ☐ Include an **examples/** folder with at least a `minimal` and a `complete` example. CI deploys both per release.
- ☐ Include **tests** -- `terraform test` or AVM Bicep test patterns.
- ☐ Include a **CHANGELOG.md** auto-generated from commits.

Once the contract is this explicit, names and namespaces become part of the same consumer experience: they help people find the right module before they ever read its README. Poor names turn even well-tested modules into hidden infrastructure, so the next section treats naming as part of design.

Naming and namespacing

Group modules by domain and name each module for the pattern it delivers, not for every underlying resource it happens to contain. In an `alz-modules` repo, that usually means a domain-oriented layout that mirrors how platform engineers and workload teams search for reusable patterns:

```

alz-modules/
├── network/
│   ├── hub/
│   ├── spoke/
│   └── private-dns/
├── identity/
│   ├── managed-identity/
│   └── role-assignment/
├── observability/
│   ├── log-analytics-workspace/
│   └── diagnostic-settings/
├── security/
│   ├── key-vault/
│   └── policy-assignment/
└── compute/
    └── vm-baseline/

```

Module names should describe **the pattern**, not the resource list, because a name such as `hub` stays readable even when the implementation includes virtual networks, firewalls, bastion, route tables, and private DNS. The README should still be explicit about what is inside, but the module path should remain stable enough that consumers can remember it and automation can depend on it. Naming discipline is one safeguard against entropy; the anti-patterns below are the failures that entropy most reliably produces.

Anti-patterns

Most module failures come from postponing reuse, floating versions, or hiding too many behaviours behind one module API. If you see any of these patterns during review, treat them as design feedback rather than as cosmetic cleanup.

- **✗ Inlined modules copy-pasted between landing zones.** The "you'll DRY it later" trap. Promote to a shared module on the second use.
 - **✗ Pinning to a branch name.** "It worked yesterday" is not a strategy.
 - **✗ A "kitchen sink" module** with 80 boolean toggles. Split it.
 - **✗ Modules that wrap a single AVM resource module with no added opinionation.** Just call AVM directly.
 - **✗ Releasing a major version without a migration guide.** Always include a `MIGRATION.md` for breaking changes.
-

A mature module ecosystem -- tiered correctly, versioned strictly, and published from a registry -- is what makes the branching and promotion strategies in the next chapter operationally safe. Without pinned modules, "it worked in non-prod" is a coincidence rather than a guarantee; with them, the diff between environments is visible, auditable, and reversible. Chapter 04 picks up the story at the branch level, addressing how code flows from a developer laptop all the way to a production subscription.

References

Use these sources for the module catalogues, registry mechanics, and release conventions referenced above.

- AVM -- Azure Verified Modules: <https://aka.ms/avm>
- AVM Bicep specs: <https://github.com/Azure/bicep-registry-modules>
- AVM Terraform specs: <https://github.com/Azure/terraform-azurerm-avm-template>
- `bicep publish` to ACR: <https://learn.microsoft.com/azure/azure-resource-manager/bicep/private-module-registry>
- Terraform module sources: <https://developer.hashicorp.com/terraform/language/modules/sources>
- Renovate: <https://docs.renovatebot.com/>
- Conventional Commits: <https://www.conventionalcommits.org/>
- SemVer: <https://semver.org/>

[← 02 IaC tooling](#) · [Index](#) · [04 Branching & environments](#) →

04 · Branching, environments & promotion

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [Recommended: trunk-based + folder-per-environment](#)
- [Alternative: branch-per-environment \("GitFlow for ops"\)](#)
- [Environment topology](#)
- [Promotion mechanics](#)
- [PR requirements](#)
- [Drift between environments](#)
- [Ephemeral environments](#)
- [Keeping environment configurations DRY -- Terragrunt and alternatives](#)
- [Anti-patterns](#)
- [References](#)

***Decision:** how does a change flow from a developer laptop to production, and how does the Git branching model map to environments?*

[← 03 Modules & registries](#) · [Index](#) · [05 Authentication](#) →

For most Azure Landing Zone (ALZ) platform repositories, you should use **trunk-based development with folder-per-environment configuration**: `main` remains the source of truth, environment differences live in reviewed parameter files, and promotion is a gated deployment sequence from non-prod to prod. Branch-per-environment is a fallback for teams whose compliance model explicitly requires branch-level releases; everyone else should avoid the drift, cherry-pick debt, and unclear rollback story it creates. This choice matters more for infrastructure than it does for ordinary application code, because a broken deployment can leave the network in a half-configured state that affects every workload above it, so your branching model has to minimise blast radius, make rollback obvious, and leave an auditable trail of who approved what. The history of how teams arrived at these models explains why the safer default is still not always the first instinct.

How we got here

When Infrastructure as Code (IaC) adoption took off around 2016, most ops teams reached for the branching model their app-dev colleagues were using: **GitFlow**. `develop` → `release/*` → `main`, each mapped to an environment, each deploy triggered by a merge. It worked -- until the first hotfix had to be cherry-picked back to `develop`, and then to `release/1.4`, and someone missed one. The next wave of teams tried **GitHub Flow** (one long branch, short feature branches) but kept the "environment = branch" idea, which ran

into the same drift problems. The **GitOps movement** (Weaveworks coined the term in 2017; Flux and ArgoCD popularised it) reframed the question: environments are *folders containing manifests*, not branches *containing code*. Combined with **trunk-based development** practices proven at high-scale shops, "one branch, folder per environment" became the dominant pattern by the early 2020s.

Key terms

GitFlow -- a branching model with parallel long-lived branches (`main` , `develop` , `release/*` , `hotfix/*`). Powerful for versioned software releases, but error-prone for infrastructure where every branch must converge.

Trunk-based development -- a model where all developers commit to a single long-lived branch (`main` / `trunk`) via short-lived feature branches, keeping integration continuous and merge conflicts small.

GitOps -- an operational pattern where the desired state of infrastructure is declared in Git and a reconciler (Flux, ArgoCD) or Continuous Integration/Continuous Delivery (CI/CD) pipeline continuously applies it, making Git the single source of truth.

Cherry-picking -- a Git operation that copies a single commit from one branch to another. Useful for targeted hotfixes, but dangerous at scale because forgetting to cherry-pick even once creates silent environment drift.

Drift -- the divergence between what your IaC code declares and what actually exists in the cloud. Drift can be caused by manual portal changes, failed applies, or cherry-pick misses between branches.

Feature flags -- conditional toggles (often a boolean variable) that let code exist in `main` without being active in all environments, allowing incomplete features to be merged safely.

Ephemeral environments -- short-lived, disposable environments (e.g. *per-PR*) spun up for testing and torn down automatically when no longer needed.

That history explains why the branch-per-environment model still survives in regulated industries that map approval to branches, usually because the audit evidence was first designed around Git branch events rather than deployment approvals. Whichever history your team carries, use the framework below to separate a real compliance constraint from inherited habit before choosing mechanics.

Decision framework

Choose the branching and promotion model by answering these questions in order. The answers are coupled - choose them together. For the vast majority of ALZ deployments, they converge on trunk-based development with folder-per-environment configuration.

1. **Monorepo or repo-per-environment?** Keep environments in the same ALZ/platform repo by default so one PR can show the non-prod and prod intent together; split repos only when environment ownership, permissions, or audit boundaries are genuinely different.
2. **Branch-per-environment or folder-per-environment?** Prefer folders such as `envs/nonprod/` and `envs/prod/` on `main` ; use long-lived environment branches only when compliance explicitly requires

branch-level releases.

3. **What environment topology do you need?** Define the minimum set of durable environments -- usually sandbox, non-prod/staging, prod, and optionally DR -- and back prod and non-prod with separate subscriptions or management groups.
4. **What exactly gets promoted?** Promote the same Git SHA, pinned module versions, and pipeline template through the sequence; environment-specific plan artifacts may differ, but they must be generated from the same reviewed source.
5. **What PR requirements make `main` safe?** Require review, status checks, CODEOWNERS approval for `envs/prod/` and modules, current branches, signed commits where required, and no force pushes.
6. **How will drift and exceptions be handled?** Detect drift with scheduled plans, treat unexplained differences as incidents, and use feature flags or parameter gates instead of leaving unfinished work off to the side in long-lived branches.

The sections below unpack those decisions, starting with the recommended model and then the branch-per-environment exception. From there, the chapter follows the change through environment design, promotion, review, drift response, and configuration reuse.

Recommended: trunk-based + folder-per-environment

The framework's default landing point is trunk-based development with folder-per-environment configuration, because it gives you one source of truth while still making every environment difference explicit and reviewable. In that model, **`main` is the source of truth for every environment**; the differences between environments live in parameter files, not in divergent branches, and a change moves through environments by being *applied* to each in sequence behind approvals rather than being *merged* from one long-lived branch to another.



In repository terms, that promotion flow works best when the same shape exists under each environment folder. The shape matters because reviewers can compare intent across environments without mentally reconciling different directory structures:

```

alz-platform/
├─ envs/
│  └─ nonprod/
│     │ └─ connectivity/  # uses modules @ pinned versions
│     │ └─ identity/
│     └─ tfvars / params files
│  └─ prod/
│     └─ ... (same shape, different parameters)
└─ modules/
  
```

That layout gives you a few operating rules that are easy to audit. `main` is the only long-lived branch, all work happens on short-lived feature branches that open pull requests into `main`, and environments are

folders rather than branches. On each pull request, `plan` or `what-if` runs for all affected environments and is posted as review evidence; after merge, the pipeline applies to **non-prod automatically** and then **gates on a manual approval before prod**.

Why this works

This model works because it attacks the failure modes that make infrastructure promotion hard. Branch-per-environment models inevitably accumulate cherry-pick debt and "what's actually in prod?" anxiety, while folder-per-environment keeps the diff between environments in the `envs/<name>/` parameter files where it is auditable and reviewable. Because history stays linear, rollback is also easier to explain under pressure: revert the merge commit and apply the resulting source.

Why people resist it

"But how do we hotfix prod without releasing the in-flight feature?"

You don't ship in-flight features to `main` until they're production-ready. Use **feature flags** (parameter toggles, conditional resources) and short branches. If a feature really must land in code but not in prod yet, gate it on an environment parameter:

```
resource "azurerm_firewall_policy_rule_collection_group" "new_rules" {
  count = var.enable_new_rules ? 1 : 0
  ...
}
```

You would keep `enable_new_rules = false` in `envs/prod/terraform.tfvars` until the change is ready, while still allowing the reviewed code to live on `main`. That distinction between hiding code in a branch and disabling behavior through an explicit parameter is what makes the branch-per-environment alternative a narrow exception rather than the default.

Alternative: branch-per-environment ("GitFlow for ops")

Branch-per-environment is still defensible when an external compliance or release process genuinely requires branch-level promotion, but you should treat it as an exception because it creates avoidable drift and cherry-pick risk. In this model, each long-lived branch maps to an environment, pipeline triggers fire on pushes to those branches, and promotion becomes a sequence of merges from `develop` to `release/*` to `main`.

```
main      → prod
release/* → staging
develop   → nonprod
feature/* → ephemeral
```

When this works

This model works best when release cycles are long, usually quarterly, and when strict change advisory boards need a human to "release" each environment through a branch event. It also fits compliance regimes that map approval to *branch* rather than *deployment*, because the audit trail follows Git merges rather than deployment gates.

When it doesn't

The same branch mapping becomes fragile as soon as platform changes are frequent, each stage contains multiple tenants, or the team has ever forgotten to cherry-pick a hotfix back to `develop`. Use it only if your compliance team mandates it, and even then, push back hard enough to confirm that branch-level evidence is really required rather than merely familiar.

The debate -- trunk-based vs branch-per-environment

This guide recommends trunk-based development, but the question is far from settled -- especially in regulated industries.

The case for branch-per-environment: *Some compliance regimes (financial services, government, healthcare) map approval gates to branches, not deployments. Their auditors aren't confused -- they genuinely want a branch-level audit trail where a prod release corresponds to a merge event, not a CI gate. Teams with strict change advisory boards, quarterly release windows, or multi-tenant environments that must not drift between stages find branch-per-environment easier to govern.*

The case for trunk-based: *Fewer long-lived branches means fewer merge conflicts, fewer forgotten cherry-picks, and a single source of truth. Environment differences live in parameter files, not in code divergence. Most modern CI systems can provide the same audit trail via deployment approvals and environment protection rules.*

Where the industry stands (2026): *Trunk-based is the dominant recommendation in the DevOps literature (DORA research, Google's engineering practices), but enterprise adoption is uneven. Many teams that switched to trunk-based report fewer incidents; others reverted after losing the "branch = frozen artifact" guarantee their auditors expected. There is no single right answer -- the best model depends on your release cadence, audit requirements, and team discipline.*

Once the branching model is explicit, define the environment estate it will protect -- how many environments exist, what each one is for, and who controls access to it. That definition turns the Git model from an abstract workflow into a set of real blast-radius boundaries.

Environment topology

Because the branching model only has meaning when it protects clear deployment targets, keep the durable environment set small, named, and subscription-separated so each boundary has a clear purpose and blast radius. Define your environments **explicitly** and document why each exists, resisting the temptation to create a new durable environment for every temporary validation need; a common pattern is:

Environment	Purpose	Subscription model
sandbox	Engineer experimentation, throwaway	Single shared sub, auto-cleanup
dev (optional)	Integration of in-flight platform changes	Dedicated sub per platform
nonprod / staging	Pre-prod validation, mirrors prod topology	Dedicated subs
prod	Production	Dedicated subs
dr (optional)	Disaster recovery	Mirrors prod, in second region

Those names only become useful if you back them with hard boundaries and automation, so apply three rules consistently. Without these controls, environment names become labels on the same shared blast radius:

1. **Prod and non-prod are in different subscriptions** (often different management groups). Anything else dilutes the value of the environment boundary.
2. **Engineers can apply in sandbox.** Pipelines apply everywhere else.
3. **Sandbox has aggressive lifecycle management.** Auto-shutdown of VMs, nightly resource group cleanup, hard cost caps.

With the environment set defined, the question is how a change moves between those boundaries without falling into the canonical trap: "it worked in staging". Promotion mechanics are where that promise either becomes enforceable or collapses into hope.

Promotion mechanics

To avoid the familiar "it worked in staging" trap, promotion must carry the same reviewed source, module versions, and pipeline logic forward so **what you tested is what you ship**. The details differ between Terraform and Bicep, but the governing idea is the same: you promote reviewed intent, not whatever happens to be current when the next environment deploys.

Promote the artifact, not the source

The safest promotion pattern is to run `terraform plan` (or `bicep build`), store the resulting artifact, and apply the reviewed output through the sequence wherever the target environment allows it. That discipline protects you from module version drift between environments when someone bumps a tag mid-flow, and from time-of-check / time-of-apply differences in upstream data sources.

In practice:

```
# pseudo-pipeline
- terraform plan -out=tfplan-nonprod -var-file=envs/nonprod.tfvars
- terraform apply tfplan-nonprod
- # Manual approval
- terraform plan -out=tfplan-prod -var-file=envs/prod.tfvars
- terraform apply tfplan-prod
```

However, you **cannot** apply a non-prod plan to prod because the state files and resources are different. In IaC, the "promote the artifact" pattern therefore means *promote the same Git SHA + the same module versions + the same pipeline template*, with environment-specific parameter files.

Lock module versions per environment

Pin module versions in `envs/<env>/versions.tf` (or a `module-versions.json` read by Bicep) so you can promote `nonprod` first, observe, then update `prod` to the same version explicitly. That makes version promotion a visible code review rather than an implicit side effect of rerunning a pipeline.

```
envs/
├─ nonprod/
│   └─ versions.tf      # module "x" { version = "1.5.0" }
└─ prod/
    └─ versions.tf      # module "x" { version = "1.4.2" } ← lags
```

When non-prod is happy after a soak period, a PR bumps prod to `1.5.0`, and the PR diff is the promotion. Of course, that diff only means something if the review process attached to it has teeth, which is why branch protection and pull request rules are part of the promotion model rather than administrative decoration.

PR requirements

Because `main` now controls every environment, you make it safe by requiring review, automated evidence, production ownership approval, and branch protection before any environment can change. The exact rule set varies by repository sensitivity, but a practical baseline for branch protection on `main` is:

- ☒ Require pull request before merging.
- ☒ Require **at least 1** reviewer (2 for foundation/policy repos).
- ☒ Require status checks to pass: `lint`, `validate`, `plan`, `policy-test`, `security-scan`.
- ☒ Require **CODEOWNERS** review for paths under `envs/prod/` and `modules/`.
- ☒ Require branches up to date before merge.
- ☒ Require **signed commits** (see [06 security](#)).
- ☒ Dismiss stale reviews on new commits.
- ☒ Disallow force pushes and branch deletion.

These controls reduce the chance that unreviewed change reaches an environment, but they do not prove the deployed environment still matches the source. Therefore, the operating model also needs an explicit drift loop that keeps Git and Azure from silently diverging after the merge.

Drift between environments

Even with strong pull request controls, assume drift will happen and design the operating model to detect it quickly, explain it, and treat unexplained differences as incidents. The simplest visible signal is a scheduled

`plan` or `what-if` run in each environment, usually weekly, with any non-empty result posted to a Teams or Slack channel where it cannot be ignored.

For Bicep with Deployment Stacks, set `denySettings: denyDelete` or `denyWriteAndDelete` so out-of-band changes are blocked at the Azure Resource Manager (ARM) layer. For Terraform, drift detection runs are your only signal, so invest in them and treat unexplained drift as incident work rather than background noise, as discussed further in [11 manageability](#).

There is, however, a complementary pattern that sidesteps long-lived environment drift entirely by making environments disposable. It does not replace the durable environment chain, but it gives risky module changes a cheaper place to fail.

Ephemeral environments

That recurring drift loop protects durable environments, while ephemeral environments give you a different kind of confidence: pull request (PR)-scoped validation for modules and platform slices without pretending that every pull request needs a full disposable copy of the entire ALZ. For pattern modules and platform components, you can spin up a **PR-scoped environment** automatically, run the tests that need real Azure resources, and then remove the slice before it becomes another environment to govern:

- Workflow on PR creation: `terraform apply` to a uniquely named resource group (`pr-<number>-<sha>`).
- Run integration tests against it.
- Workflow on PR close: `terraform destroy` .

This is a per-PR slice of the modules being changed, not a copy of the whole landing zone, because the latter is usually too expensive and too slow to be useful. Once those short-lived slices exist beside durable environments, the remaining maintenance problem is how to keep the repeated environment configuration readable without adding unnecessary orchestration tooling.

Keeping environment configurations DRY -- Terragrunt and alternatives

Start with native `envs/` folders, parameter files, and Continuous Integration (CI) matrices, and add Terragrunt, Atmos, or Terramate only when repeated environment boilerplate becomes measurable pain. Modules solve the don't repeat yourself (DRY) problem for *resource definitions*, but a separate DRY problem lurks in *environment configurations*: the backend blocks, provider blocks, and `.tfvars` that differ per environment. When you have five environments × eight workloads, even a well-structured `envs/` folder accumulates significant boilerplate.

Key terms

Terragrunt -- a thin wrapper around Terraform (by Gruntwork) that generates backend configs, provider blocks, and input variables from a shared template, reducing per-environment boilerplate. It also manages cross-stack dependencies (e.g. "the spoke needs the hub's VNet ID").

Atmos -- a framework by Cloud Posse that provides environment composition, stack configuration inheritance, and component orchestration for Terraform projects.

Terramate -- an orchestration tool that adds code generation, change detection, and execution ordering to Terraform/OpenTofu projects without wrapping the CLI.

What Terragrunt solves

Terragrunt earns its keep when the repetition is not just cosmetic but operational: backends, providers, inputs, dependencies, and execution order all have to stay aligned across many stacks. The table below shows the specific places where it replaces hand-maintained repetition with generated or inherited configuration.

Problem	Terragrunt approach
Duplicated backend blocks	<code>generate "backend"</code> in a root <code>terragrunt.hcl</code>
Duplicated provider blocks	<code>generate "provider"</code> with inherited variables
Per-env variable files	<code>inputs = { ... }</code> inheritance from parent directories
Cross-stack dependencies	<code>dependency</code> blocks that read outputs from other stacks
Execution order	<code>run-all plan</code> applies stacks in dependency order

Terragrunt remains **widely used** and solves these problems well. It was the de facto standard for multi-environment Terraform orchestration from roughly 2019 to 2023, and many mature estates run it successfully today.

Why we don't recommend it as the default for ALZ

For a *typical ALZ estate* -- three to five environments, a handful of workloads, one cloud -- the extra layer often costs more than it saves. The most common costs show up in day-to-day debugging and pipeline maintenance:

- **Additional domain-specific language (DSL) to learn.** Terragrunt's HCL-dialect (`dependency` , `generate` , `include`) is conceptually simple but adds a debugging layer between the engineer and Terraform. Error messages refer to generated files, not the source.
- **Versioning surface.** You now pin Terraform *and* Terragrunt versions, and their compatibility matrix is not always smooth.
- **CI complexity.** Most pipeline examples assume `terraform plan` ; Terragrunt `run-all` requires different caching, parallelism, and artifact strategies.
- **Native alternatives have closed part of the gap:**

Terragrunt capability	Native equivalent (2026)
DRY backend config	Partial backend config (<code>-backend-config=</code>) + CI variables
DRY provider config	<code>envs/<env>/provider.tf</code> shared via symlink or CI template
Per-env variables	<code>envs/<env>/*.tfvars</code> + matrix pipeline
Cross-stack data	<code>terraform_remote_state</code> data source or explicit outputs piped via CI
Execution order	Pipeline DAG stages / explicit <code>needs:</code> / <code>dependsOn:</code>
For Bicep	Parameter files + Deployment Stacks handle all of the above natively

Where native tooling still falls short is orchestration across dozens of Terraform stacks with complex inter-dependencies, especially when you want a single `run-all plan` command. In that specific shape of estate, Terragrunt or Terramate genuinely saves time, but that threshold is higher than many ALZ teams reach at the start.

Recommendation

For most ALZ teams, a well-structured `envs/` folder, `.tfvars` per environment, and a CI matrix build achieve the same DRY outcome with **less tooling surface**. Adopt Terragrunt, Atmos, or Terramate when the environment × workload matrix grows large enough that the native approach produces measurable duplication pain, not as a default starting point.

The debate -- Terragrunt and DRY orchestrators

The recommendation above ("start native, adopt Terragrunt when the pain is real") is itself debated.

The counter-argument: Experienced practitioners -- including Gruntwork, Cloud Posse, and many platform engineers managing 50+ landing zones -- argue that Terragrunt's overhead is modest compared to the duplication and orchestration bugs you accumulate without it. They point out that "native equivalents" (symlinks, `-backend-config=`, CI variable injection) are fragile workarounds that break in non-obvious ways, whereas Terragrunt's `dependency` blocks and `run-all` give you a tested, deterministic execution graph. From their perspective, the advice to "wait until it hurts" means you only adopt good tooling after accruing tech debt -- the equivalent of saying "don't write tests until you have bugs."

This guide's position: For a small-to-mid ALZ (fewer than ~15 stacks), the native approach is simpler and has a lower bus-factor risk. Beyond that threshold the tradeoff tilts, and a DRY orchestrator becomes the pragmatic choice. Reasonable engineers disagree on where the tipping point is.

With the promotion model, environment boundaries, and configuration tooling now tied together, the remaining risk is less subtle: shortcuts that bypass non-prod validation, shared state isolation, artifact promotion, or independent review. These are the choices that usually feel convenient in the moment and expensive during an incident.

Anti-patterns

Avoid these shortcuts because each one breaks a safety mechanism established earlier in the chapter. If you find yourself making one of them, treat it as a signal that the normal path is too slow or too unclear and fix that path instead.

- **✗ `main` deploys straight to prod with no non-prod stop.** The classic "we'll add staging later".
 - **✗ One state file shared across environments.** A failed `apply` in non-prod can corrupt prod state.
 - **✗ Cherry-picking commits between long-lived branches.** You will lose one eventually; that's how outages start.
 - **✗ "It worked in non-prod" without artefact promotion.** Non-prod and prod ran against different module versions.
 - **✗ Merging your own PR.** Even for "trivial" changes. Especially in foundation repos.
-

With branching strategy, environment topology, and promotion mechanics in place, the structural decisions for your ALZ estate are largely complete. What remains are the operational details that determine whether the structure stays trustworthy over time: authentication (Chapter 05), security controls, state management, and drift response. Get the three foundational decisions right -- topology, tooling, and promotion model -- and the later chapters are refinement. Get any one of them wrong and no amount of clever pipeline YAML will compensate.

References

- Trunk-based development: <https://trunkbaseddevelopment.com/>
 - GitHub branch protection: <https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches>
 - Azure DevOps branch policies: <https://learn.microsoft.com/azure/devops/repos/git/branch-policies>
 - Hashicorp, *Recommended Terraform workflow*: <https://developer.hashicorp.com/terraform/cloud-docs/recommended-practices/part1>
 - Microsoft, *CAF -- environments*: <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/considerations/environments>
-

[← 03 Modules & registries](#) · [Index](#) · [05 Authentication](#) →

05 · Authentication & identity for pipelines

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [The hard rule](#)
- [What to use, by runner](#)
- [OIDC federation -- how it actually works](#)
- [SPN vs Managed Identity vs User-Assigned MI with federation](#)
- [RBAC scope -- least privilege per environment](#)
- [Authenticating to other systems](#)
- [Developer access](#)
- [Audit & rotation](#)
- [Multi-tenant scenarios](#)
- [Anti-patterns](#)
- [References](#)

***Decision:** how do CI/CD runners and engineers authenticate to Azure without ever holding a long-lived secret?*

[← 04 Branching & environments](#) · [Index](#) · [06 Security →](#)

Use OpenID Connect (OIDC) federated credentials for every Azure deployment pipeline, and ban long-lived Service Principal Name (SPN) secrets or certificates from continuous integration and continuous delivery (CI/CD) systems. GitHub Actions, Azure DevOps, GitLab, and Azure-hosted self-hosted runners all now have a secretless path: the runner exchanges an OIDC token, or a Managed Identity (MI) token when it is already running inside Azure, for a short-lived Microsoft Entra access token, and that Azure identity receives only the role assignments it needs for the environment it deploys. The tradeoff is not whether secretless authentication is mature enough, because it is; the real design choice is which identity object carries the trust, how tightly its federated subject is constrained, and where its Azure role-based access control (RBAC) assignment ends. This chapter walks you through those choices for pipelines, engineers, other platforms, and multi-tenant estates, while keeping the authentication story separate from the secrets and supply-chain controls that follow in the next chapter.

How we got here

Pipeline authentication should now be treated as a solved, secretless problem, because the industry spent a decade moving from fragile credentials to OIDC federation and Azure CI/CD should move with it. In the early days you minted a **service principal with an X.509 certificate**, mounted it onto your build agent, and

prayed nobody copied the PFX. When that proved operationally painful, the community moved to **client secrets** -- easier to handle, easier to leak, and leak they did, repeatedly, in committed YAML files and CI logs. Microsoft shipped **Managed Identities** in 2017, which solved the problem elegantly *for workloads running in Azure*, but CI/CD runners such as GitHub-hosted agents, Jenkins on-premises workers, and GitLab software as a service (SaaS) jobs still needed long-lived credentials. GitHub announced **OIDC for Actions** in 2021, Entra added support for **workload federated credentials** the same year, and the entire problem class evaporated almost overnight: the runner asks GitHub for a short-lived signed JSON Web Token (JWT), exchanges it with Entra, and receives a one-hour access token. No secret ever exists.

Key terms

SPN (Service Principal Name) -- the shorthand this guide uses for a Microsoft Entra service principal, an identity object that represents an application or service and authenticates non-human workloads to Azure.

MI (Managed Identity) -- an Azure-managed identity attached to a resource (virtual machine (VM), App Service, Azure Kubernetes Service (AKS) pod) that obtains tokens automatically without storing any secret. Comes in system-assigned (tied to one resource) and user-assigned (reusable across resources) variants.

OIDC (OpenID Connect) -- an identity layer built on OAuth 2.0 that lets one system prove its identity to another via signed tokens, without exchanging long-lived secrets.

JWT (JSON Web Token) -- a compact, signed token format used to carry identity claims (e.g. "this request comes from repo X, branch main, environment prod").

Entra ID -- Microsoft's cloud identity platform (formerly Azure Active Directory / AAD). Manages users, groups, SPNs, and federated credentials.

PIM (Privileged Identity Management) -- an Entra ID feature that provides just-in-time, time-limited elevation to privileged roles, requiring approval and multi-factor authentication (MFA).

SIEM (Security Information and Event Management) -- a platform (e.g. Microsoft Sentinel) that aggregates logs from across your estate for threat detection and incident response.

Conditional Access -- Entra ID policies that enforce requirements (MFA, compliant device, location) before granting access to resources.

By 2024 federated credentials had spread to **user-assigned managed identities** as well, and Azure DevOps shipped its own equivalent, which means the old excuse that only some runner types can avoid secrets no longer holds. There is now no defensible reason to store an Azure secret in a CI system -- and yet, surveys keep showing that most do. That gap between what the platform supports and what estates still configure is why the next section starts with a decision sequence rather than another history lesson.

Decision framework

The decision is straightforward: eliminate stored Azure credentials first, then choose the federated identity and RBAC scope that minimize blast radius for each runner and environment. Because the history above

leaves you with several legitimate implementation shapes rather than a single product button, answer these questions in order before wiring a pipeline:

1. Which runner is authenticating -- GitHub Actions, Azure DevOps, GitLab, or self-hosted?

- GitHub Actions and GitLab use issuer/subject OIDC claims.
- Azure DevOps should use a workload identity federation service connection.
- Self-hosted runners on Azure should use managed identity where possible.

2. Which identity object should receive the trust -- SPN, managed identity, or user-assigned managed identity with federation?

- Prefer user-assigned managed identity with federation when it fits; use an SPN with a federated credential when platform support or tooling requires it.

3. What is the RBAC scope per environment -- management group, subscription, or resource group?

- Scope by environment and layer, not by convenience; prod and non-prod need separate identities.

4. Does the pipeline authenticate to systems beyond Azure?

- Apply the same rule to GitHub, Datadog, Vault, registries, and Terraform platforms: federate or use issued tokens, not stored secrets.

5. For multi-tenant estates, which tenant owns each federated identity?

- Create one identity per target tenant and let the pipeline select the right credential rather than stretching one identity across tenants.

The detailed guidance below turns those answers into concrete Azure and CI/CD configuration choices. It starts by making explicit the one rule that should not vary by runner, tool, or tenant.

The hard rule

No CI/CD system should contain a long-lived Azure credential, whether that credential is a service principal client secret, a certificate, or a manually configured token that survives beyond the workflow run that needed it. In 2026 there is no good reason to store such a credential in a GitHub Actions secret, an Azure DevOps service connection of type "manual", or a Jenkins credential, because OIDC federation is broadly supported and removes the entire class of leaked-secret incidents. Once you accept that rule, the runner no longer changes the destination; it only changes the route you take to obtain the short-lived token.

What to use, by runner

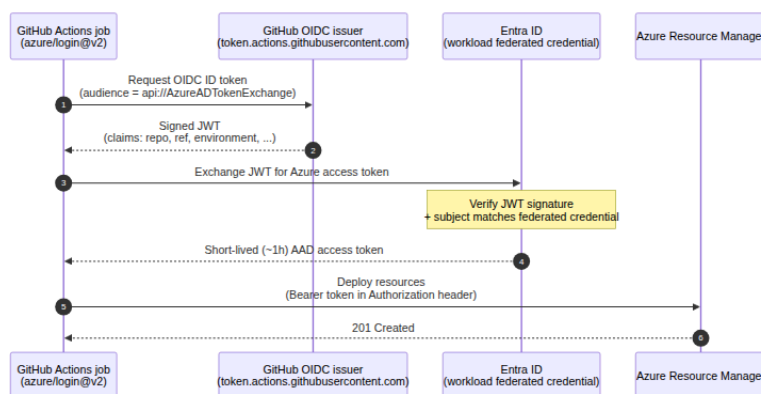
Different runners expose different token sources, but the pattern stays constant: federate from SaaS CI, use managed identity for Azure-hosted self-hosted runners, and reserve interactive `az login` for humans rather than falling back to stored Azure credentials because a platform feels different. The table below maps each runner to its secretless path without changing that rule.

Runner	Recommended	How
GitHub Actions	OIDC → Entra workload federated credential → SPN	<code>azure/login@v2</code> with <code>client-id</code> , <code>tenant-id</code> , <code>subscription-id</code> , no secret
Azure DevOps Pipelines	Workload identity federation service connection	"Workload identity federation" connection type
GitLab CI	OIDC ID tokens → federated credential → SPN	<code>id_tokens:</code> block + <code>azure/login</code>
Self-hosted runner on Azure VM/AKS	Managed Identity	System- or user-assigned MI on the runner host
Local developer workstation	<code>az login</code> (interactive) + Conditional Access	Never share an SPN secret with developers

The table gives you the operational answer, but it is worth understanding the GitHub Actions version of the exchange because the same mental model applies to the other SaaS runners as well. Once you can trace that exchange, the identity-object choice that follows becomes much easier to evaluate.

OIDC federation -- how it actually works

OIDC federation is the default pipeline authentication pattern because the whole exchange happens inside a single workflow run without creating, transmitting, or storing a reusable secret; only short-lived signed tokens cross the wire, and Entra accepts them only when their issuer, audience, and subject match a trust that you configured ahead of time. The diagram below shows that exchange in its most common GitHub Actions form.



The trust is established once on the **Entra app registration** or user-assigned managed identity by adding a *federated credential* that restricts which workflow, environment, branch, or repository can request a token. After that, every run repeats the same exchange against Azure Resource Manager (ARM) without ever relying on a secret you could accidentally print, copy, or forget to rotate.

Setup (Bicep example)

```
resource app 'Microsoft.Graph/applications@v1.0' = {
  displayName: 'sp-alz-platform-prod'
}

resource fic 'Microsoft.Graph/applications/federatedIdentityCredentials@v1.0' = {
  parent: app
  name: 'github-prod'
  audiences: [ 'api://AzureADTokenExchange' ]
  issuer: 'https://token.actions.githubusercontent.com'
  // Restrict to the prod environment in this specific repo:
  subject: 'repo:contoso/alz-platform:environment:prod'
}
```

Setup (CLI)

```
APP_ID=$(az ad app create --display-name sp-alz-platform-prod --query appId -o tsv)
az ad sp create --id $APP_ID

az ad app federated-credential create --id $APP_ID --parameters '{
  "name": "github-prod",
  "issuer": "https://token.actions.githubusercontent.com",
  "subject": "repo:contoso/alz-platform:environment:prod",
  "audiences": ["api://AzureADTokenExchange"]
}'
```

Choosing a `subject` template

The subject is the most security-critical field in that trust configuration, because it decides which repository context can impersonate the Azure identity. Be **as restrictive as possible**:

Pattern	When to use
<code>repo:org/repo:environment:prod</code>	Recommended. Ties to a GitHub Environment with reviewers.
<code>repo:org/repo:ref:refs/heads/main</code>	Only main branch can deploy.
<code>repo:org/repo:pull_request</code>	Read-only plan jobs from PRs (forks excluded).
<code>repo:org/repo:*</code>	✗ Too permissive. Anyone with write to <i>any</i> branch can assume the SPN.

You can attach **multiple** federated credentials to the same SPN, such as one for the `prod` environment and one for the `main` branch deploy job, but each entry should still describe a narrow, intentional path into Azure rather than a convenient wildcard. The workflow then consumes that trust without receiving anything that needs to be stored as a secret.

GitHub Actions consumer

```
permissions:
  id-token: write  # ← required for OIDC
  contents: read

jobs:
  deploy:
    environment: prod  # ← matches the subject filter
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${vars.AZURE_CLIENT_ID}
          tenant-id: ${vars.AZURE_TENANT_ID}
          subscription-id: ${vars.AZURE_SUBSCRIPTION_ID}
      - run: az account show
```

Those values can live in `vars` rather than `secrets`, because the client ID, tenant ID, and subscription ID identify the trust target but cannot authenticate by themselves. The next design question is therefore not how to hide those values, but which Azure identity object should sit behind them.

SPN vs Managed Identity vs User-Assigned MI with federation

Prefer a user-assigned managed identity with federation when the platform and tooling support it, use a federated SPN when you need the app registration model, and avoid any pattern that lets humans share the pipeline identity. This choice matters because the identity object determines both how you configure trust and what future operators can accidentally do to weaken it.

Identity type	Used for	Notes
Service Principal (app registration)	GitHub / ADO / GitLab pipelines	Add a federated credential -- no secret.
User-assigned Managed Identity	Self-hosted runners on Azure	MI's now also support federated credentials → can be used from non-Azure runners too.
System-assigned Managed Identity	A runner VM/AKS workload that won't be replaced	Tied to the lifecycle of the host.

Microsoft now recommends **user-assigned MI with federation** even for GitHub Actions, because MIs cannot have client secrets created against them at all, which gives you defence in depth against a future shortcut. SPNs remain valid and are still common where tooling expects an app registration, but whichever object you choose, its role assignments should be scoped as if the federated subject were your last line of defence rather than your only one.

RBAC scope -- least privilege per environment

The safest practical model is one federated identity per **environment** × **layer**, with that identity granted only the Azure scope that the layer genuinely deploys. This keeps the authentication boundary aligned with the repository and pipeline boundary from earlier chapters: a platform production workflow should not inherit the permissions of a foundation workflow, and a non-production token should never be useful in production.

For example:

```
sp-alz-foundation-prod      → Owner @ tenant root MG (rare use)
sp-alz-platform-nonprod    → Contributor @ platform-nonprod MG
sp-alz-platform-prod       → Contributor @ platform-prod MG
sp-alz-landingzone-corp-app01-prod → Contributor @ corp-app01-prod sub
```

- **Do not reuse identities across environments.** A leaked non-prod token must not give access to prod.
- Prefer **Contributor** + a few **explicit role assignments** (User Access Administrator, Network Contributor) over a blanket Owner.
- Foundation identities need **Owner** at tenant scope to manage RBAC and policy -- restrict their use with strict federated credential subjects and GitHub Environment approval gates.
- Use **Privileged Identity Management (PIM)** for any standing high-privilege human access; pipeline identities should not need PIM elevation since their blast radius is limited by the federated credential subject.

Use Azure Resource Manager scopes thoughtfully

Granting at the management group scope cascades to all subscriptions inside, so landing-zone identities are usually easier to reason about when they receive subscription-scope grants instead. That makes reviews and incident response more auditable, because you can look at a subscription and see the small set of pipeline identities that are allowed to change it. That covers the Azure plane; landing zone pipelines, however, rarely speak to ARM alone, and the same authentication rule has to follow them into GitHub APIs, container registries, secret stores, and third-party tooling.

Authenticating to other systems

Every non-Azure integration should use federation, platform-issued tokens, or fine-grained app credentials before you consider a stored secret. The Azure identity model gives you the main pattern, but pipelines often need more than just Azure, so the same decision has to be repeated for each target system:

Target	Recommendation
GitHub (gh CLI, REST)	<code>{{ secrets.GITHUB_TOKEN }}</code> (built-in) or a GitHub App with fine-grained perms
Azure DevOps	Workload identity federation (the same SPN) + ADO PAT only as last resort
Azure Container Registry	<code>az acr login</code> after <code>azure/login</code> (uses AAD) -- never docker login with credentials
Terraform Cloud / Enterprise	Dynamic provider credentials (TFC's own OIDC to Azure)
Vault	OIDC auth method + AppRole as fallback

The principle from the Azure sections is deliberately unchanged: federate or use an issued token, and do not store a long-lived secret unless the target platform leaves you no credible alternative. Pipelines are only one half of the authentication picture, though, because the engineers who trigger, debug, and approve them need their own distinct identity path -- one that never crosses with the deploy SPN.

Developer access

Humans and pipelines must use different identities, with engineers relying on interactive sign-in, Conditional Access, and PIM rather than deploy SPNs. That separation preserves audit quality: when a pipeline changes Azure, you can attribute the deployment to the workflow and its approvals, and when an engineer investigates Azure directly, you can attribute the action to the person. In practice, these patterns keep that boundary intact:

- Engineers `az login` interactively. Conditional Access enforces MFA + device compliance.
- For "deploy from my laptop" cases (sandbox only), they get a *role assignment on the sandbox subscription* via PIM.
- Production access for break-glass: a dedicated emergency account, MFA, PIM-elevated, with full audit alerting. Not the deploy SPN.

Because humans and pipelines now have separate identities, audit has something meaningful to observe: unexpected sign-ins, unexpected role assignments, and drift in the federated credentials themselves. The next control loop turns that separation into evidence you can review and alert on.

Audit & rotation

OIDC removes secret rotation from the normal pipeline path, but it does not remove the need to audit token use, role assignments, and federated credential drift. In fact, because the credential no longer expires in the old secret-rotation sense, the control you operate shifts from rotating strings to proving that only the intended workflow subjects can still obtain tokens.

Operational controls:

- Stream Entra **sign-in logs** to your SIEM. Filter on the deploy SPN app IDs.
- Alert on:
 - Sign-ins with subject claims that don't match your federation rules (the federated credential should reject these -- alert if they appear).
 - Unexpected source IPs (GitHub-hosted runners come from known ranges).
 - SPN being granted any new role assignment outside the change window.
- Review federated credentials quarterly. Remove anything that no longer maps to an active workflow.
- If you still must use SPN client secrets, rotate them ≤ 90 days, ideally via Key Vault rotation policies -- but really, just move to OIDC.

Those controls assume the identity, the subscription, and the tenant all line up neatly, which is not always true in enterprise estates created through acquisitions, sovereign requirements, or separate business units. When tenant boundaries enter the design, you keep the same rule but duplicate the trust per tenant.

Multi-tenant scenarios

For multi-tenant estates, create one federated identity per target tenant and let the pipeline choose the tenant-specific credential; do not stretch one SPN across tenants with Guest access. If your platform spans multiple Entra tenants because of acquisitions, sovereign clouds, or hard organisational boundaries, the cross-tenant glue belongs in the pipeline, where it can select the right credential for the target subscription, rather than in Entra as a broad guest relationship that is difficult to reason about later. This keeps the same blast-radius model you used for environments and layers, even when the estate crosses tenant boundaries, and it also makes the anti-patterns easier to spot.

Anti-patterns

These choices reintroduce long-lived secrets, blur audit trails, or expand blast radius after federation has solved the core problem. Each one is a regression to the failure modes the chapter has been removing section by section.

- **✗ Service principal client secret stored in GitHub secrets** -- whoever can edit the workflow file can exfiltrate it via `echo`.
- **✗ One SPN with Owner @ root for "convenience"** -- the full-estate blast radius is unacceptable.
- **✗ Federated credential subject `repo:org/repo:*`** -- defeats the purpose of federation.
- **✗ Engineers sharing an `azureuser@contoso.onmicrosoft.com` account for "lab access"**. Always individual identities.
- **✗ No alerting on the deploy SPN**. You'd never know if it was abused.
- **✗ Same SPN used by humans and pipelines**. Audit becomes impossible.

Authentication is the gate that governs everything else in this book. Get it wrong and every other control becomes optional -- a determined attacker holding a long-lived Owner secret can undo your policy

assignments, drain your state storage, and cover their tracks before morning. Get it right with OIDC federation, scoped identities, and PIM-gated human access, and the controls in the next chapter become substantially cheaper to enforce. Chapter 06 builds on this foundation to address secrets that genuinely do need to be stored, the integrity of the artefacts your pipeline produces, and the supply-chain risks that no authentication scheme alone can neutralise.

References

- GitHub, *Configuring OIDC in cloud providers -- Azure*: <https://docs.github.com/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-azure>
 - Microsoft, *Workload identity federation*: <https://learn.microsoft.com/entra/workload-id/workload-identity-federation>
 - Microsoft, *Federated identity credentials on managed identities*: <https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/how-manage-federated-identity-credentials>
 - Azure DevOps, *Workload identity federation for service connections*: <https://learn.microsoft.com/azure/devops/pipelines/library/connect-to-azure>
 - Microsoft, *Securing Azure pipelines (defender for DevOps)*: <https://learn.microsoft.com/azure/defender-for-cloud/defender-for-devops-introduction>
-

[← 04 Branching & environments](#) · [Index](#) · [06 Security](#) →

06 · Secrets & supply-chain security

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [The threat model](#)
- [Secrets in code -- eliminate, don't manage](#)
- [Branch protection that matters](#)
- [Supply-chain hygiene](#)
- [Signed commits](#)
- [Securing CI runners](#)
- [Workflow hardening \(GitHub Actions\)](#)
- [State file security \(Terraform\)](#)
- [Policy / compliance gates](#)
- [Incident playbook \(one paragraph\)](#)
- [Sovereign Landing Zone \(SLZ\) -- when compliance demands more](#)
- [Protecting vended resources -- defence in depth](#)
- [Anti-patterns](#)
- [References](#)

Decision: *how do you keep credentials out of code, prove the integrity of what's deployed, and stay ahead of supply-chain attacks?*

[← 05 Authentication](#) · [Index](#) · [07 State management →](#)

Secure an Azure Landing Zone (ALZ) Infrastructure as Code (IaC) repo by eliminating long-lived secrets first, then layering branch protection, signed provenance, pinned dependencies, isolated runners, hardened workflows, protected state, and policy gates. The central tradeoff is that every extra control adds friction, but every missing control leaves a path from a pull request, dependency, or runner into your tenant. Treat every pipeline dependency and Continuous Integration (CI) runner as a potential compromise path: use OpenID Connect (OIDC) plus Azure Key Vault instead of stored credentials, require CODEOWNERS and status checks before merge, pin every Action, module, and tool by immutable digest, and map regulated or sovereign controls directly to repo and pipeline enforcement. That defence-in-depth stance is easier to apply once you see how the threat shifted from accidentally committed passwords to compromised build systems, which is where the chapter starts.

How we got here

For a long time, "secrets management" in IaC meant *"don't commit them"* -- enforced by code review, vibes, and luck. Luck ran out repeatedly: Uber's AWS keys in a public Git repo (2016), countless `.tfvars` leaks, shared access signature (SAS) tokens in `terraform.tfstate` files uploaded to misconfigured backends. Push-side defences emerged around 2018: `truffleHog`, `gitLeaks`, eventually GitHub's **native secret scanning** and **push protection**. Then the threat shifted *upstream*: the SolarWinds attack (2020) and the Codecov bash-uploader compromise (2021) showed that a trusted dependency could be the attack vector. The IaC world felt this directly with the `tj-actions/changed-files` Action compromise of March 2025, which exfiltrated secrets from thousands of pipelines that had pinned by *tag* rather than by *commit SHA*. The response is the defence-in-depth model this chapter describes: keep secrets out of code, sign what you ship (Sigstore, Supply-chain Levels for Software Artifacts (SLSA) attestations), pin every dependency by digest, and assume your CI runner will eventually be compromised.

Those four imperatives become the decision framework below: decide which controls are mandatory, where they run, and who owns exceptions.

Decision framework

Pick the security controls by answering these questions in order. The hard rules are non-negotiable; the rest decide how much additional isolation, evidence, and compliance mapping your ALZ repo needs.

1. Have you eliminated long-lived secrets from the repo and pipelines?

- Hard rule: yes. Use OIDC for Azure authentication and Key Vault for application secrets; anything else should be treated as a temporary exception with an owner and expiry. See [05 authentication](#).

2. Which branch protection rules are mandatory?

- Require CODEOWNERS review, at least one approval (two for foundation and policy), required status checks, stale-review dismissal, conversation resolution, no force-push or deletion, and re-approval after changes from forks.

3. How will you secure the supply chain?

- Pin GitHub Actions, modules, providers, and tools by immutable SHA or digest; enable dependency scanning and controlled update automation; publish SBOMs and signed release artefacts where your deployment process consumes build outputs.

4. Which runners execute sensitive plans and applies?

- Use GitHub-hosted larger runners with static IPs for ordinary workflows; use ephemeral, isolated self-hosted runners only when production or regulated deployments need network controls that hosted runners cannot provide.

5. Are commits, tags, and artefacts signed?

- Prefer Sigstore / `gitsign` for organisation-wide keyless commit signing and artifact attestations; allow GPG or SSH signing where individuals need traditional key ownership.

6. If you are regulated or sovereign, which controls map to repo and pipeline enforcement?

- Map SLZ, audit, residency, encryption, and separation-of-duties requirements to branch rules, policy gates, runner isolation, protected environments, and evidence artefacts before implementation starts.

The full analysis below explains the threat paths behind those choices and the concrete controls that close them.

The threat model

The decision framework only becomes useful when you map each control to a concrete attack path, because attackers will target Git history, contributor accounts, runners, dependencies, state, and drift rather than the tidy categories in your governance model. For an IaC repo that controls an enterprise Azure estate, the realistic attack vectors are:

1. **Leaked secrets** in Git history.
2. **Compromised contributor account** pushing a malicious change.
3. **Compromised CI runner** exfiltrating tokens or planting backdoors.
4. **Malicious / typo-squatted module dependency** (Terraform Registry, npm, PyPI, GitHub Actions in `uses:`).
5. **State file leakage** exposing secrets baked into resources.
6. **Drift / out-of-band change** introducing an unreviewed configuration.

Because each path uses a different part of the delivery system, the controls below deliberately overlap instead of pretending that one gate can carry the whole security model. The first and most common path is still credential leakage, so the practical work starts with removing secrets rather than merely hiding them better.

Secrets in code -- eliminate, don't manage

The only durable pattern for the credential path is to eliminate long-lived secrets from repos and pipelines, using OIDC for Azure authentication and Azure Key Vault for application secrets. These controls reinforce each other rather than substituting for each other:

- **OIDC for cloud auth** (see [05 authentication](#)).
- **Reference Key Vault for application secrets**, never inline them:

```
module app 'app.bicep' = {
  params: {
    sqlPassword: keyVault.getSecret('sql-admin-password')
  }
}
```

- **Generate secrets in-pipeline** (e.g. random storage shared access signatures (SAS)) and write them straight to Key Vault -- they never appear in logs or state.

- **Mark all sensitive Terraform variables** `sensitive = true` so they don't print in plan output. (They will still appear in state -- see below.)

Pre-commit secret scanning

Secret scanning needs to run on every developer machine **and** in CI, because local hooks catch mistakes before they enter history while the pipeline catches missing or bypassed hooks. Configure both layers with the same allowlist discipline:

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.0
    hooks:
      - id: gitleaks
  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.5.0
    hooks:
      - id: detect-secrets
```

In CI, run `gitleaks detect` on every pull request (PR) with `--redact` and fail the build on any finding. Maintain an **allowlist file** for false positives, reviewed quarterly.

GitHub's native controls add the server-side layer that developer hooks cannot provide:

- **Push protection** for secret scanning -- blocks pushes that contain known secret patterns. Enable for every repo.
- **Secret scanning alerts** -- also scans dependency files.

If a real secret is committed, **rotate the credential first**, then purge history (`git filter-repo` / GitHub support), because the secret was public the moment it was pushed. Eliminating and scanning for secrets closes the credential path, but it still leaves the path where a bad change is reviewed too casually; that is where branch protection takes over.

Branch protection that matters

After secrets are removed from the normal delivery path, branch protection has to prevent a compromised contributor account from turning a malicious change into approved code. Require CODEOWNERS review, status checks, signed commits, stale-review dismissal, linear history, and no force-push or deletion on every protected branch.

Setting	Recommended
Require PR before merge	✓
Require review approvals	≥ 1 (≥ 2 for foundation/policy)
Require review from CODEOWNERS	✓
Dismiss stale reviews on new commits	✓
Require re-approval after fork updates	✓
Require signed commits	✓ (Sigstore/gitsign or GPG)
Require linear history	✓
Require status checks	lint, plan, security-scan, policy
Require conversation resolution	✓
Restrict who can push	only the deploy bot for <code>main</code>
Disallow force push / deletion	✓

GitHub's **rulesets** let you apply these controls across all repos in an organisation, so prefer rulesets over per-repo branch protection when you need consistent enforcement. Once your own contributors are constrained, the remaining attack surface is the code you *don't* own: the Actions, modules, and tools your pipeline fetches from the internet.

Supply-chain hygiene

The supply-chain side of the same argument is that every external dependency should be pinned by immutable digest or SHA, updated through controlled automation, and accompanied by evidence of what the pipeline consumed.

Pin everything by digest, not by tag

Tags are mutable because a malicious maintainer can re-point `v1.0.0` to a compromised commit, so pin GitHub Actions by SHA:

```
# ✗ Bad -- tag is mutable
- uses: hashicorp/setup-terraform@v3

# ✓ Good -- pinned to a SHA
- uses: hashicorp/setup-terraform@b9cd54a3c349d3f38e8881555d616ced269862dd # v3.1.2
```

Use **Dependabot** with `dependency-type: version-update:semver-major` disabled and `version-update:semver-patch` auto-merged after CI passes, so you get security updates without unsupervised major bumps. For Terraform modules, lock with `.terraform.lock.hcl` (committed) and run `terraform init -upgrade` only via PRs.

SBOM for IaC?

For IaC, Software Bill of Materials (SBOM) generation is less mature than it is for application code, but it is still worth doing because it gives your security tools something concrete to inspect:

- For Terraform, generate a module dependency report (`terraform providers schema -json` + custom tooling).
- For Bicep, `bicep build --stdout` then parse `metadata` blocks plus `br:` module references.
- Store SBOMs as build artifacts; ingest into your Software Composition Analysis (SCA) tool (GitHub Advanced Security (GHAS), Snyk, etc.).

Key terms

SBOM (Software Bill of Materials) -- a machine-readable inventory of every component (modules, providers, Actions) your pipeline depends on, used to track known vulnerabilities.

SCA (Software Composition Analysis) -- tooling that scans your dependency tree for known CVEs and licence risks.

GHAS (GitHub Advanced Security) -- GitHub's security suite including code scanning (CodeQL), secret scanning, Dependabot, and dependency review.

SLSA (Supply-chain Levels for Software Artifacts) -- a framework (pronounced "salsa") that defines increasingly strict levels of build integrity, from source to artefact.

Sigstore -- an open-source project providing keyless code-signing and verification, removing the burden of managing GPG/PGP keys.

GPG (GNU Privacy Guard) -- a traditional public-key cryptography tool used to sign Git commits and verify author identity.

CMK (Customer-Managed Key) -- an encryption key you own and manage in Azure Key Vault, used instead of Microsoft-managed keys for data-at-rest encryption.

`pull_request_target` -- a GitHub Actions trigger that runs with the base branch's secrets and permissions even when a fork opens a PR. If the workflow checks out and executes fork code, it creates a privilege escalation path.

Provenance / SLSA

After you know what the pipeline consumed, provenance proves that the artifact being deployed is the artifact the trusted workflow produced. Use [GitHub's artifact attestations](#) to sign your deploy artifacts (compiled Bicep, planned Terraform), and have the deploy job verify the attestation before applying:

```
- uses: actions/attest-build-provenance@v1
  with:
    subject-path: dist/main.json

# in the deploy job:
- run: gh attestation verify dist/main.json --owner ${{ github.repository_owner }}
```

This makes a "build a malicious artifact and apply it directly" attack detectable, and the same provenance story should extend backward from artifacts to the commits and tags that produced them.

Signed commits

Because artifact attestations only prove the integrity of build outputs, you should also require commits and release tags to be signed and verified before they become trusted inputs. Sigstore / `gitsign` is the default for organisation-wide enforcement, although there are two practical options:

- **Sigstore / `gitsign`** -- keyless, OIDC-backed signing. No GPG keys to manage; signatures verifiable on GitHub.
- **GPG / SSH signing** -- traditional, requires key distribution.

In CI, verify the signatures on the commits included in each pull request:

```
- name: Verify signatures on commits in this PR
  run: |
    base=${{ github.event.pull_request.base.sha }}
    head=${{ github.event.pull_request.head.sha }}
    git log --pretty='%H %G?' "$base..$head" | awk '$2 != "G" { print "unsigned: "$1;
  exit 1 }'
```

Signatures make unauthorized source changes visible; runner isolation limits what a compromised job can do after signed code starts executing.

Securing CI runners

Once source and artifacts are verifiable, the next question is where that trusted code executes. Use GitHub-hosted larger runners with static IPs for most jobs, and reserve ephemeral isolated self-hosted runners for production or regulated deployments, because GitHub-hosted runners are convenient but have caveats for sensitive estates:

Concern	GitHub-hosted	Self-hosted on Azure
IP allowlist	Possible via larger runners with static IPs	Trivially yours
Network egress control	Limited	Full (NSG/Firewall)
Tooling control	Microsoft-maintained images	You patch (more work, more control)
Compromise blast radius	High (shared infra)	Limited

For most workflows, GitHub-hosted **larger runners with static IPs** give you the best balance of operational simplicity and access control; for foundation and production deployments, use self-hosted runners on a **dedicated subscription** where the network boundary is part of the control. Self-hosted runners must be:

- Ephemeral (one job per VM, then destroyed). Use the [actions-runner-controller](#) on AKS or Azure VM Scale Sets with auto-scale.
- In a network-isolated subscription with egress only to required APIs (Azure Resource Manager (ARM), Microsoft Graph, GitHub).
- Authenticated via a Managed Identity (no personal access token (PAT) for runner registration -- use the GitHub App-based runner registration).

Runner isolation narrows blast radius; workflow hardening reduces the chance that a job becomes malicious in the first place.

Workflow hardening (GitHub Actions)

Runner isolation reduces blast radius, but the workflow YAML itself is still production security code and should start with least privilege, bounded execution, environment gates, and no checkout credentials left behind. In practice, a handful of settings make an outsized difference:

```
permissions:                                # ← always start with least privilege
  contents: read

concurrency:                                # ← prevent duplicate parallel applies
  group: deploy-prod
  cancel-in-progress: false

jobs:
  deploy:
    environment: prod                        # ← gates with reviewers
    timeout-minutes: 60                     # ← bound runaway jobs

    steps:
      - uses: actions/checkout@<sha>
        with:
          persist-credentials: false         # ← no GITHUB_TOKEN left on disk
      - uses: actions/setup-node@<sha>
      # ...
```

Run `actionlint` and `zizmor` (workflow security scanner) in CI so the workflow definition is checked with the same seriousness as the IaC it runs. Even a hardened, well-scoped workflow produces an artefact that deserves its own security treatment: the Terraform state file.

State file security (Terraform)

The first downstream artifact to secure is Terraform state, because state files **contain secrets in cleartext** by default and should be protected like Azure Key Vault. State can include database passwords, storage keys, and certificates, so treat the backend as a privileged system rather than a convenient blob container:

- Backend storage account: **firewall-restricted, private endpoint, CMK encryption, soft-delete + versioning enabled, diagnostic logging to Log Analytics Workspace (LAW).**

- Access only via the deploy service principal (SPN); humans access via Privileged Identity Management (PIM) with full audit.
- Use `azurerm` backend with `use_oidc = true` so the runner authenticates the same way it authenticates ARM.
- Never download state to a developer laptop.

The backend design details are covered in [07 state management](#), but the security point here is simpler: locking down the artefacts your pipeline produces addresses only one dimension of compliance. The other is preventing those artefacts from representing non-compliant configurations in the first place.

Policy / compliance gates

Once state is protected, policy gates make sure the secured pipeline is not faithfully deploying something non-compliant. Treat policy as a **merge-blocking CI check**, not a deployment afterthought:

- Custom Azure Policy definitions live in version control alongside their assignments.
- On PR, run `Conftest` / `Checkov` / `tfsec` / **PSRule for Azure** against the planned changes -- block the merge on critical findings.
- Detail in [09 testing & policy](#).

These checks reduce the likelihood of a bad change reaching production, but they do not remove the need for a rehearsed response when a credential leak or compromised pull request gets through anyway.

Incident playbook (one paragraph)

Even with policy gates in place, you should assume that a credential leak or compromised pull request will happen and document the response order before the incident. When something happens -- leaked credential, compromised PR -- the response order is:

1. **Revoke** the credential / app role / token.
2. **Rotate** anything touched by the compromised identity.
3. **Audit** Entra sign-in logs and ARM activity logs for the blast window.
4. **Re-deploy** affected resources from a known-good Git SHA.
5. **Postmortem**, then update playbook and protections.

Document this in your repo (`docs/runbooks/credential-leak.md`), because when you need it, you won't have time to invent it. For regulated or sovereign estates, the same evidence discipline has to be designed into the landing zone itself rather than added after the first audit request.

Sovereign Landing Zone (SLZ) -- when compliance demands more

Where compliance demands sovereignty, treat the Sovereign Landing Zone (SLZ) as a composable overlay on ALZ and map each sovereignty requirement to policy, evidence, and pipeline controls. SLZ is not a separate product; it is a **variant layer** that sits on top of the standard ALZ and adds sovereignty controls at three levels:

Level	Controls
L1 -- Baseline	Data residency policies, encryption requirements, audit logging to sovereign region.
L2 -- Enhanced	Confidential computing management groups, restricted service endpoints, HSM-backed key management.
L3 -- Full sovereign	Customer-managed keys everywhere, no data leaving the sovereign region, confidential VMs for management workloads.

Because SLZ uses the same ALZ library and modules, the difference is in the **archetype overrides** and additional policy assignments. If you later need to add sovereign controls to a standard ALZ, you can layer SLZ archetypes onto your existing configuration without re-deploying from scratch.

 *From the ALZ Weekly Questions -- When to use SLZ over ALZ? Think of SLZ as a composable overlay: ALZ base + SLZ layer + your local customisations. The `alzlibtool` handles the composition.*

Those overlay controls govern what can be deployed; the next concern is protecting the baseline resources after subscription vending creates them.

Protecting vended resources -- defence in depth

Whether you deploy standard ALZ or SLZ overlays, platform-vended resources need overlapping controls because no single guardrail is enough. When subscription vending creates baseline resources such as resource groups, networking, and diagnostic settings, the platform team needs to prevent workload teams from accidentally modifying or deleting them through deny assignments, scoped Role-Based Access Control (RBAC), and deny policies:

1. **Deny assignments via Deployment Stacks** -- when the platform team's Deployment Stack owns baseline resources with `denyWriteAndDelete`, workload teams physically cannot modify those resources even with Contributor role on the subscription.
2. **RBAC at resource group scope, not subscription scope** -- instead of granting `Contributor` on the entire subscription, grant it only on the resource groups the app team owns. The platform-managed resource groups (e.g. `rg-networking`, `rg-diagnostics`) have separate, restrictive RBAC.
3. **Deny-action policies with tag-based exclusions** -- a custom deny policy prevents deletion of resources tagged `managed-by: platform`. App teams can manage resources without that tag freely.

These mechanisms layer on top of each other, so if any one fails, the others still protect the baseline.

 **From the ALZ Weekly Questions -- Subscription Vending: Repo Structure, Security & Multi-Tenant**
The combination of deny assignments + scoped RBAC + tag-based deny policies gives you defence in depth without blocking legitimate workload operations.

The anti-patterns below are what happens when teams knowingly remove one of those overlapping controls for speed and then forget that the exception exists.

Anti-patterns

These shortcuts are red flags because they trade short-term delivery speed for persistent compromise paths.

-  **Adding `--allow-secret` patterns to bypass scanning.** That's how real secrets slip through.
-  **`pull_request_target` with checkout of PR code.** Classic privilege escalation vector -- a PR from a fork can read your secrets. Use `pull_request` for untrusted PRs.
-  **Self-hosted runners on a long-lived VM with cached credentials.** A single compromised job persists into the next.
-  **GitHub Personal Access Tokens for cross-repo automation.** Use a GitHub App with fine-grained, time-limited installation tokens.
-  **Disabling `terraform plan` policy checks "just for this PR".** It becomes permanent.

Security is not a single gate but a set of overlapping controls, each assuming the others will occasionally fail. The practices in this chapter -- eliminating secrets at source, hardening runners and workflows, signing artefacts, and enforcing policy checks before merge -- are most valuable precisely because no single one of them is foolproof. Chapter 07 shifts focus from the pipeline itself to what the pipeline produces: the state files and deployment stacks that record the current shape of your Azure estate, and which carry their own considerable operational risk if you get the topology wrong.

References

- GitHub, *Security hardening for GitHub Actions*: <https://docs.github.com/actions/security-guides/security-hardening-for-github-actions>
- GitHub, *Secret scanning*: <https://docs.github.com/code-security/secret-scanning>
- OpenSSF, *SLSA*: <https://slsa.dev/>
- Sigstore, *gitsign*: <https://github.com/sigstore/gitsign>
- Microsoft, *PSRule for Azure*: <https://azure.github.io/PSRule.Rules.Azure/>
- Bridgecrew, *Checkov*: <https://www.checkov.io/>
- Aqua, *tfsec*: <https://aquasecurity.github.io/tfsec/>
- gitleaks : <https://github.com/gitleaks/gitleaks>
- actionlint : <https://github.com/rhysd/actionlint>
- zizmor (Actions audit): <https://github.com/woodruffw/zizmor>

-  ALZ Weekly -- *When to use SLZ over ALZ?*: <https://www.youtube.com/watch?v=r8h7F6IJIqw>
-  ALZ Weekly -- *Subscription Vending: Repo Structure, Security & Multi-Tenant*:
<https://www.youtube.com/watch?v=11PmT0t6TUI>

[← 05 Authentication](#) · [Index](#) · [07 State management →](#)

07 · State management

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [Terraform backend choice](#)
- [Locking](#)
- [Blast radius -- how to size state files](#)
- [Cross-state references](#)
- [State surgery -- when you must](#)
- [Bicep -- Deployment Stacks](#)
- [Drift detection](#)
- [Anti-patterns](#)
- [References](#)

Decision: for Terraform, where does state live, how is it locked, and what is the blast radius of a single apply ? For Bicep, the equivalent question is how you carve up Deployment Stacks.

← [06 Security](#) · [Index](#) · [08 CI/CD pipeline patterns](#) →

Treat state topology as an operational safety control, not as a footnote. For Terraform, size state files to limit blast radius: use one Terraform state file per environment per workload, default to a remote backend with locking, and never share state between environments, never treat local state as harmless, and never let consumers read a producer's state just to get an output. For Azure-native Terraform, your default backend should be Azure Storage with blob lease locking, private endpoints, customer-managed keys (CMKs), role-based access control (RBAC), and a dedicated state subscription; for production Bicep, the equivalent lifecycle boundary is a Deployment Stack with explicit scope, `denySettings`, and reviewed `actionOnUnmanage`. State is where Terraform keeps its map of the world -- and historically where teams have kept their worst secrets, their most spectacular outages, and their longest Friday evenings -- so the choices in this chapter are about operational safety as much as tool configuration.

How we got here

To see why that operational framing matters, start with the earliest Terraform shops, which kept `terraform.tfstate` on the engineer's laptop, occasionally emailing it around or -- worse -- committing it to Git. Concurrent applies were "managed" by Slack message ("*@everyone don't apply, I'm applying*"), and the inevitable state-file conflicts were resolved by hand-merging JSON. Hashicorp shipped **remote backends with locking** (S3 + DynamoDB; later `azurerm` with blob leases) around 2017, which solved the concurrency problem but introduced a new one: many teams pointed *every* environment at the same backend

and the same state file, producing a single explosive `apply` whose blast radius was the whole estate. The 2018-2022 period was a slow lesson in **state granularity** -- one state per (workload × environment) -- and in **state-as-a-secret-store** hardening. Bicep deliberately avoided the question for years because Azure Resource Manager (ARM) was the source of truth, but customers kept asking for the *features* state provided: managed-resource tracking, drift protection, and "what would removing this line do?" Microsoft's answer was **Deployment Stacks**, which reached general availability (GA) in 2024 and give you Terraform-style guarantees with no state file to operate. Those histories now leave you with two related decisions: where Terraform state should live, and how Bicep stacks should draw their lifecycle boundary.

Key terms

Remote backend -- a shared, network-accessible location (e.g. Azure Blob Storage) where Terraform stores state, enabling locking and team collaboration.

Blob lease -- an Azure Storage locking mechanism that Terraform's `azurerm` backend uses to prevent two concurrent `apply` operations against the same state file.

State surgery -- manually editing Terraform state (via `terraform state mv`, `rm`, or `import`) to fix drift between state and reality without recreating resources.

force-unlock -- a Terraform command that manually releases a stuck state lock, typically needed after a runner crash during `apply`.

denySettings -- a Deployment Stacks property that prevents out-of-band changes to managed resources (e.g. `denyDelete` blocks manual deletions).

actionOnUnmanage -- a Deployment Stacks property that controls what happens to resources removed from the template: `deleteAll`, `detachAll`, or per-type settings.

Azure Blueprints -- a now-deprecated Azure governance service (sunset July 2026) that bundled ARM templates, policy assignments, role assignments, and resource groups into a versioned, assignable package. Superseded by Deployment Stacks.

Decision framework

With that history and vocabulary in place, pick the state model by answering these questions in order. The first two answers set the tool and storage model; the later answers reduce blast radius, secret exposure, and lifecycle risk before any pipeline runs `apply`.

1. Terraform or Bicep -- which dictates the state model?

- Terraform → choose and harden a remote backend; state is an operator-owned artifact.
- Bicep → use Deployment Stacks for production lifecycle management; Azure owns the stack metadata.

2. Which Terraform backend fits the operating model?

- Choose a backend that supports locking -- no exceptions.
- Azure-native ALZ → `azurerm` backend in Azure Storage with blob lease locking is the recommended default.

- Terraform Cloud / Enterprise → viable when you also want run UI, Sentinel, and dynamic provider credentials.
- `s3` + DynamoDB → only when AWS is already part of the platform.
- Local state → never for anything persistent or shared.

3. How small should one state file be?

- Use one state per (*environment* × *workload*) as the default blast-radius boundary; never one mega-state for the estate.
- Size by unit of deployment, not by repo or team.
- A single `apply` should touch resources owned by one team and deployable in under 15 minutes.

4. How do downstream stacks reference upstream state?

- Prefer explicit outputs published to App Configuration, tagged resource groups, or another controlled configuration store.
- Use `terraform_remote_state` only when the consumer is trusted to read the producer's full state contents.
- For Bicep, use `existing` lookups or the same explicit output store.

5. For Bicep, are Deployment Stacks managing production lifecycle?

- Use stacks for lifecycle tracking, `denySettings`, `actionOnUnmanage`, and managed identity rotation.
- Raw `az deployment` is acceptable for ephemeral PR environments, not for production landing zones.

6. How is state secured?

- Treat state like a secret store: encrypted at rest with CMK, network isolated, access audited, humans elevated through PIM, and soft delete and versioning enabled.
- Put Terraform state in a separate subscription, behind a private endpoint, encrypted with customer-managed keys, and accessed through RBAC rather than shared keys.

Quick reference of the core choices:

Decision	Recommended default	Why
Terraform backend	Azure Storage <code>azurerm</code> backend	OIDC, blob lease locking, private endpoint, CMK, Azure RBAC.
Terraform granularity	One state per (<i>environment</i> × <i>workload</i>)	Limits corruption and bad-apply blast radius.
Cross-state references	Explicit outputs in App Configuration or tags	Avoids leaking producer state secrets to consumers.
Bicep lifecycle	Deployment Stacks for prod	Gives managed resource tracking, deny settings, and unmanage behaviour.
State security	Dedicated subscription + private endpoint + CMK + RBAC	Keeps the ledger outside the blast radius of the resources it manages.

State is the **most operationally dangerous** part of Terraform. A corrupt state file at 2 a.m. on a Friday is a career-defining moment, and the same principle applies to Bicep stacks: scope is blast radius. The full analysis starts with the Terraform backend because that is the first place you either contain or amplify every later risk.

Terraform backend choice

Because the framework's first Terraform question is storage, start with the backend that gives you Azure-native controls by default: for Azure Landing Zone (ALZ) implementations, use the Terraform `azurerm` backend in Azure Storage, and choose another backend only when its operating model is already part of your platform. The choice is not cosmetic; it determines locking behaviour, authentication options such as OpenID Connect (OIDC), network isolation, and whether your identity model carries through end to end.

Backend	Recommendation
<code>azurerm</code> (Azure Storage)	Default for Azure-native shops. Supports OIDC, blob lease locking, CMK, private endpoint.
Terraform Cloud / Enterprise (HCP)	Strong if you also want run UI, Sentinel policy, and dynamic provider credentials. Costs money.
<code>s3</code> + DynamoDB	Only if you genuinely have AWS in the picture.
Local state	Never. Not even for dev. ¹

¹ This is a strong stance -- see the debate box below.

The debate -- local state in sandbox environments

"Never local state, not even for dev" is the safest blanket rule, but practitioners with personal sandbox subscriptions and aggressive auto-cleanup policies (e.g. Azure subscription lifecycle management, nightly resource-group purge) argue that remote state adds unnecessary ceremony for truly disposable, exploratory work. A more nuanced position: **remote state for anything that persists or is shared; local state is acceptable for isolated sandboxes with automatic teardown** -- provided the team understands this is a conscious exception, not the default.

Recommended `azurerm` backend setup

With that caveat out in the open, the Azure Storage pattern is deliberately boring: one **dedicated subscription** (`sub-tfstate-prod`) hosts state for everything. Inside it, use **one storage account per environment, one container per workload, and one blob (state file) per environment-workload**.

```

sub-tfstate-prod/
├── rg-tfstate-prod
│   └── sttfstateprod001
│       ├── platform-connectivity-prod
│       │   └── terraform.tfstate
│       ├── platform-identity-prod
│       │   └── terraform.tfstate
│       ├── lz-corp-app01-prod
│       │   └── terraform.tfstate
│       └── ...

```

```

terraform {
  backend "azurerm" {
    resource_group_name = "rg-tfstate-prod"
    storage_account_name = "sttfstateprod001"
    container_name       = "platform-connectivity-prod"
    key                  = "terraform.tfstate"
    use_oidc              = true
    use_azuread_auth     = true
  }
}

```

Once the layout exists, harden the storage account as though it were a secret store, because in practice that is what Terraform state becomes:

- ☐ Public network access **disabled**, private endpoint only.
- ☐ **Microsoft Entra ID authentication** for blob (`use_azuread_auth = true`); shared-key access disabled at the storage account level.
- ☐ **Customer-managed key** (CMK) in a Key Vault you control.
- ☐ **Soft delete** for blobs and containers (≥ 30 days).
- ☐ **Versioning** enabled -- `terraform.tfstate` versions are your "undo" button.
- ☐ **Diagnostic logs** to a Log Analytics workspace; alert on any access from outside the runner subnets.
- ☐ **RBAC**: only the deploy service principal (SPN) has `Storage Blob Data Contributor` ; humans only via PIM with elevation alerts.

After the backend is isolated and recoverable, the next failure mode is concurrency: two runners trying to mutate the same ledger at once.

Locking

Backend hardening protects the ledger from exposure and deletion, but locking protects it from concurrent writers. Blob lease locking is mandatory for Terraform, and you still need pipeline-level concurrency so two jobs never contend for the same state in the first place. The Terraform `azurerm` backend uses Azure Storage **blob leases** for this purpose, and the behaviour is simple enough that your pipeline contract can be explicit:

- Each `plan` / `apply` acquires a lease on the state blob.
- If a lease is held, other operations fail fast with the lock ID.

- If a runner crashes mid-apply, the lease eventually expires (default 60s acquire retry) -- but a manual `terraform force-unlock <ID>` is sometimes needed.

Pipeline contract

- **Concurrency control** at the pipeline level (`concurrency: deploy-<env>` in GitHub Actions) so you never enqueue two `apply` s for the same state.
- If a job fails mid-apply, the next job blocks on lock, alerts the team, and someone investigates *before* force-unlocking.

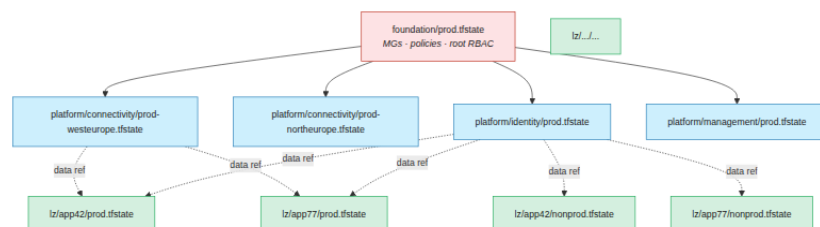
Locking prevents the concurrent-apply catastrophe. The complementary concern is the sheer amount of infrastructure a single apply can reach -- which is entirely a function of how you carve your state files.

Blast radius -- how to size state files

Once locking ensures that only one writer touches the file, the next question is how much infrastructure that writer can affect. Keep state wide and shallow: a single `apply` should affect one team's resources, in one environment, and finish in under 15 minutes. Smaller state files reduce the blast radius of corruption or a bad apply, although they also increase the number of cross-state references you need to manage; the practical sweet spot is this:

Layer	Recommended state granularity
Foundation	One state per <i>environment</i> (prod, nonprod).
Platform -- connectivity	One state per <i>environment</i> $\times$ <i>region</i> (one for <code>prod-west europe</code> , one for <code>prod-northeurope</code>).
Platform -- identity / mgmt	One state per <i>environment</i> .
Landing zone	One state per (<i>workload</i> $\times$ <i>environment</i>). Never share.

Visualised as a tree, a healthy state layout is wide and shallow -- many small files at the leaves, a handful at the root:



The practical test is the same regardless of the tree shape: if a single `apply` no longer touches resources owned by one team, or if `plan` takes 20 minutes, your state is too big. At that point, split the file before the pipeline normalizes long plans and broad approvals as business as usual.

Subscription vending -- the three-layer state model

When your estate vends (provisions) application landing zone subscriptions at scale, a **three-layer state architecture** prevents the common trap of one giant state file managing hundreds of subscriptions:

Layer	Scope	Storage	Owned by
Platform	Management groups, policies, hub networking	Dedicated storage account in the management subscription	Platform team
Vending	One state file per vended subscription -- resource groups, baseline RBAC, VNet peering, tags	Separate storage account (or container per subscription)	Platform team (automated)
Workload	Application resources inside the vended subscription	App team's own storage account	Application team

This separation means:

- A **bad vending apply** only corrupts one subscription's scaffold, not the entire estate.
- **Parallel deployments** -- each subscription's state file is independent, so your CI pipeline can deploy 50 subscriptions concurrently.
- **Clear ownership** -- the platform team owns layers 1-2; the app team owns layer 3. State permissions follow the same boundary.

For Bicep estates, the same principle applies via Deployment Stacks: one stack per vended subscription, scoped at subscription level.

 *From the ALZ Weekly Questions -- Subscription Vending: Repo Structure, Security & Multi-Tenant*
The three-layer model also lets you use `git diff` in CI to build a matrix of changed subscriptions -- only deploying what changed, not re-planning the entire estate.

Cross-state references

The wide-and-shallow model only works if you resist the temptation to stitch the leaves back together through unrestricted state reads. Prefer explicit published outputs over direct state reads:

`terraform_remote_state` is convenient, but it grants access to everything in the producer's state file. A common example is a workload stack that needs the hub virtual network ID, and you usually have two ways to provide it.

A) terraform_remote_state data source

```
data "terraform_remote_state" "connectivity" {
  backend = "azurerm"
  config = {
    resource_group_name = "rg-tfstate-prod"
    storage_account_name = "sttfstateprod001"
    container_name       = "platform-connectivity-prod"
    key                  = "terraform.tfstate"
    use_oidc              = true
  }
}

resource "azurerm_virtual_network_peering" "to_hub" {
  remote_virtual_network_id =
data.terraform_remote_state.connectivity.outputs.hub_vnet_id
  ...
}
```

Pros: simple, no extra infrastructure, real-time. **Cons:** the consumer needs **read access to the producer's state file** -- which contains all of the producer's secrets in cleartext.

B) Outputs in Azure (recommended)

The producer writes its outputs to **Azure App Configuration** or **tagged on a known resource group**:

```
resource "azurerm_app_configuration_key" "hub_vnet_id" {
  configuration_store_id = data.azurerm_app_configuration.platform.id
  key                   = "platform/connectivity/hub-vnet-id/${var.env}"
  value                 = azurerm_virtual_network.hub.id
  label                 = var.env
}
```

The consumer reads via the same App Configuration:

```
data "azurerm_app_configuration_key" "hub_vnet_id" {
  configuration_store_id = data.azurerm_app_configuration.platform.id
  key                   = "platform/connectivity/hub-vnet-id/${var.env}"
  label                 = var.env
}
```

Pros: consumers never see producer state. Outputs are first-class, versioned in App Configuration, with point-in-time history. **Cons:** an extra component; subtle race if consumer reads before producer publishes.

For Bicep, the equivalent is `existing` lookups or App Configuration / resource group (RG) tags -- Bicep has no remote-state concept by design.

Even the best-designed topology will eventually require manual intervention -- a resource renamed mid-flight, an import needed for something created out-of-band, a module refactor that moved resources between states. That is what state surgery is for, and it deserves a ceremony proportionate to the risk.

State surgery -- when you must

Even with clean output contracts, you will eventually meet a resource that no longer lines up with the state file that owns it. Avoid state surgery unless there is no safer path; when you must perform it, use reviewed Terraform commands from a logged runner, back up the state first, and prove the post-surgery plan is clean. The operating rule is deliberately conservative:

1. **Always back up** the current state first: `terraform state pull > backup-$(date +%s).tfstate`
2. Use `terraform state mv / rm / import` rather than hand-editing JSON.
3. Do it from a controlled runner, not your laptop, so the action is logged.
4. Open a PR with a `STATE-SURGERY.md` describing the why and the commands.
5. Run `plan` afterwards -- it must show **no changes**, otherwise stop.

For the `import` workflow, prefer **HashiCorp Configuration Language (HCL) `import` blocks** (Terraform 1.5+) over the legacy CLI command because they are reviewable in a pull request.

```
import {
  to = azurerm_resource_group.legacy
  id = "/subscriptions/.../resourceGroups/rg-legacy"
}
```

All of the above applies to Terraform. If you are working in Bicep, you have a different but equally powerful option -- one that sidesteps the state problem altogether.

Migrating state from CAF-Enterprise-Scale to AVM

That same controlled approach matters most if your estate was built on the classic Cloud Adoption Framework Enterprise-Scale (CAF-ES) module (`Azure/terraform-azurerm-caf-enterprise-scale`), because moving to Azure Verified Modules (AVM) changes state addresses as well as code. The ALZ team provides a **Golang state migration tool** that automates much of this:

1. **Phase 1 -- Connectivity and management resources.** The tool reads your existing state, maps resource addresses from CAF-ES module paths to AVM module paths, and generates Terraform `import` blocks. Resources like VNets, firewalls, Log Analytics workspaces, and ExpressRoute circuits move cleanly.
2. **Phase 2 -- Management groups and policies.** The policy structure differs significantly between CAF-ES and AVM. The tool produces an **issues CSV** listing resources that require manual review or re-creation.

For that migration, the practical guidance is narrower than the tool's capabilities:

- The tool works from **any CAF-ES version** -- you don't have to be on the latest before migrating (though older versions produce more issues).
- **Only import resources you cannot easily delete and recreate** -- ExpressRoute circuits, firewalls with live BGP sessions, DNS zones with active records. For management groups and policies, a clean re-deploy with Deployment Stacks handling the cutover is often simpler.
- The accelerator generates a starting Terraform configuration with migration comments showing where to add `import` blocks.

- The migration tool is **reusable** beyond ALZ -- it can restructure any Terraform state file by mapping old module addresses to new ones.

 *From the ALZ Weekly Questions -- Migrating from CAF-Enterprise-Scale to AVM Think of migration as resource mapping → attribute mapping → `terraform apply` with import blocks. The tool handles the first two steps; you review and apply.*

Bicep -- Deployment Stacks

Terraform needs a hardened state backend because the state file is yours to operate; Bicep takes the opposite route by letting Azure keep the lifecycle metadata. For production Bicep ALZ deployments, use Deployment Stacks, and reserve raw deployments for disposable environments where lifecycle protection is not required. Deployment Stacks give you the state-like benefits that raw `az deployment` lacks:

- A stack tracks the resources it deployed.
- `denySettings` blocks out-of-band modifications (`denyDelete` or `denyWriteAndDelete`).
- `actionOnUnmanage` controls what happens to resources removed from the template (`detach` , `delete` , or `deleteAll`).

Because stacks are the lifecycle boundary, carve them the same way you'd carve Terraform state -- one stack per (workload × environment), scoped at subscription or resource group.

```
az stack sub create \
  --name lz-corp-app01-prod \
  --location swedencentral \
  --template-file main.bicep \
  --parameters @prod.bicepparam \
  --deny-settings-mode denyWriteAndDelete \
  --deny-settings-excluded-actions "Microsoft.Compute/virtualMachines/start/action" \
  --action-on-unmanage deleteAll
```

The tradeoff against raw `az deployment` is straightforward: stacks are slower to create, but they are vastly safer for production, while raw deployments remain fine for ephemeral pull request (PR) environments. That distinction matters most after deployment, where lifecycle tracking starts paying you back.

A key Day-2 benefit: when the ALZ library removes a deprecated policy assignment, the Deployment Stack's `actionOnUnmanage: deleteAll` setting **automatically cleans up the orphaned assignment** -- no manual intervention required. This closes a gap that previously made Bicep ALZ harder to maintain than Terraform ALZ for policy lifecycle management.

 *From the ALZ Weekly Questions -- How to Stay Current with ALZ Azure Policies Deployment Stacks gave Bicep ALZ automatic cleanup of deprecated policies -- matching what Terraform achieves through state tracking.*

The debate -- are Deployment Stacks production-ready at scale?

Deployment Stacks GA'd in late 2024 and are maturing rapidly, but the community has less collective operational experience with them than with Terraform state, which has a decade of battle scars.

Concerns from practitioners: `what-if` for stacks is still in preview (as of early 2026) and less reliable than `terraform plan`. Tooling around stacks -- refactoring, automated drift detection, import of existing resources -- is thinner than Terraform's mature ecosystem (Spacelift, Env0, state-mv, etc.). Teams running early production stacks have reported edge cases with complex resource dependencies and `denySettings` interactions that are not yet well-documented. Migration from Blueprints is documented but not extensively battle-tested by the community.

The optimistic view: Stacks remove an entire category of operational pain (state corruption, lock contention, backend availability). Microsoft is investing heavily, and the feature set is advancing quickly. For new Bicep estates, stacks are the obvious default; the rough edges are real but narrowing.

For greenfield Bicep estates, Deployment Stacks are the right bet. If you're evaluating a migration from Terraform specifically to avoid state, make sure you're gaining more than you're giving up in ecosystem maturity, and test thoroughly in non-prod before trusting `denySettings` with your production estate.

Deployment Stacks vs Azure Blueprints (deprecated)

Teams migrating from **Azure Blueprints** (deprecated July 2026) will recognise the ambition: both aim to deploy a set of resources as a governed unit with lifecycle tracking and policy enforcement. The execution is very different.

Capability	Azure Blueprints	Deployment Stacks
Status	Deprecated (July 2026)	GA (2024)
Scope	Management group or subscription	Management group, subscription, or resource group
Template language	ARM JSON only	Bicep or ARM JSON
Versioning	Built-in (draft → published → assigned)	External -- you version the template in Git; the stack always applies the latest
Drift protection	Resource locks applied per-artefact	<code>denySettings</code> at the ARM layer -- more granular, supports excluded principals and actions
Unmanage behaviour	Resources orphaned on blueprint deletion	Configurable via <code>actionOnUnmanage</code> : delete, detach, or per-type
Policy / role bundling	Blueprints could assign policies and roles as artefacts	Stacks deploy whatever the template contains -- policies and roles are just resources
Sequencing	Built-in artefact sequencing	Template-level <code>dependsOn</code> (standard ARM/Bicep)
What-if / plan	No	<code>az stack ... --what-if</code> (preview)
State storage	Azure-managed (opaque)	Azure-managed (opaque) -- same model, better transparency

Blueprints were right about one important idea: a landing zone is a *governed package*, not just resources but also policies, roles, and locks deployed as a single unit with lifecycle tracking. That concept survives in Stacks.

Where Blueprints failed was execution: they were coupled to ARM JSON, used an opaque versioning system disconnected from Git, offered no real plan/what-if experience, and never reached enough adoption to justify continued investment.

The migration path is therefore conceptual rather than mechanical: replace each Blueprint assignment with a Deployment Stack whose Bicep template reproduces the same resources, policy assignments, and role assignments. The `denySettings` property replaces Blueprint locks, Git versioning replaces the built-in draft/published workflow, and Microsoft's [migration guide](#) gives you the detailed steps.

Once stacks replace Blueprints or raw deployments, the remaining operational question is the same one Terraform asks every week: has reality drifted from the declared model?

Drift detection

After you have defined state files or stacks, you need scheduled proof that reality still matches them; absence of drift must be verified, not assumed. Run a weekly scheduled `plan` for every Terraform state file, or `what-if` for every Bicep stack:

- Empty diff → all green.
- Non-empty diff → post to a "platform drift" Teams channel; create an issue; require triage within an SLA.

For Bicep stacks with `denySettings`, drift should be impossible by construction, but human break-glass paths and excluded actions exist, so you still verify and route the full operational process through [11 manageability](#).

The anti-patterns below are the architectural choices most likely to turn a controlled `apply` into an all-hands incident.

Anti-patterns

The anti-patterns are familiar now because each one reverses a safety property the chapter has been building toward: they expand blast radius, weaken locking, or expose secrets before the first production apply.

- **✗ One state file for the whole estate.** "Apply" becomes a ritual, changes pile up, and one mistake breaks everything.
- **✗ Local state on engineer laptops.** Even in dev. Especially in dev.
- **✗ State backend in the same subscription as the resources it manages.** A bad `apply` could nuke the storage account holding the state. Put state in its own subscription.
- **✗ Sharing state files across teams via `terraform_remote_state` without realising it leaks all producer secrets.** Use App Configuration outputs instead.

- **✗ Editing state files by hand.** Even with `jq`. There is no validation; one typo and the state is unrecoverable.
- **✗ Bicep deployments without stacks for prod.** You lose drift protection.

State -- or its Bicep equivalent -- is the operational ledger for your entire Azure estate. The decisions in this chapter determine whether that ledger is a reliable audit trail or a liability waiting to ruin someone's weekend. A granular, hardened, properly-referenced state topology is also the prerequisite for the CI/CD pipeline patterns in the next chapter: once you know exactly what each apply touches and why it is safe to run concurrently, designing the pipeline around it becomes considerably more straightforward.

References

- Hashicorp, *Backend configuration*: <https://developer.hashicorp.com/terraform/language/settings/backends/azurerem>
 - Hashicorp, *State*: <https://developer.hashicorp.com/terraform/language/state>
 - Microsoft, *Deployment stacks*: <https://learn.microsoft.com/azure/azure-resource-manager/bicep/deployment-stacks>
 - Microsoft, *Migrate from Blueprints to Deployment Stacks*: <https://learn.microsoft.com/azure/governance/blueprints/concepts/deployment-stacks-migration>
 - Microsoft, *Azure Blueprints deprecation*: <https://learn.microsoft.com/azure/governance/blueprints/overview>
 - Microsoft, *Storage account security baseline*: <https://learn.microsoft.com/security/benchmark/azure/baselines/storage-security-baseline>
 - Hashicorp, *import blocks*: <https://developer.hashicorp.com/terraform/language/import>
 - 🧑‍🔧 ALZ Weekly -- *Subscription Vending: Repo Structure, Security & Multi-Tenant*: <https://www.youtube.com/watch?v=11PmT0t6TUI>
 - 🧑‍🔧 ALZ Weekly -- *Migrating from CAF-Enterprise-Scale to AVM*: <https://www.youtube.com/watch?v=DSBWjQlVpSs>
 - 🧑‍🔧 ALZ Weekly -- *How to Stay Current with ALZ Azure Policies*: https://www.youtube.com/watch?v=ddcVKS_MKkk
-

← 06 Security · [Index](#) · 08 CI/CD pipeline patterns →

08 · CI/CD pipeline patterns

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [GitHub Actions vs Azure DevOps Pipelines](#)
- [The standard two-workflow shape](#)
- [Reusable workflow \(the actual work\)](#)
- [PR comments that are actually useful](#)
- [Detecting changed environments](#)
- [Pipeline-as-code, but DRY](#)
- [Long-running operations & retry](#)
- [Manual operations & the "break glass" pipeline](#)
- [Caching and runtime](#)
- [Anti-patterns](#)
- [References](#)

***Decision:** what does the workflow that turns a commit into a deployed Azure resource look like -- and how do you keep it consistent across many repos?*

[← 07 State management](#) · [Index](#) · [09 Testing & policy](#) →

For most Azure Landing Zone (ALZ) infrastructure as code (IaC) estates, the safest default is a two-workflow continuous integration and continuous delivery (CI/CD) shape: every pull request validates and plans, every merge to `main` deploys through environment gates, and both workflows delegate the real work to reusable pipeline templates. GitHub Actions and Azure DevOps Pipelines are both first-class Azure deployment platforms in 2026, so you should choose between them by looking at where your code, identity federation, network egress, and audit controls already work. Once that platform decision is made, path-based change detection, readable pull request (PR) plan comments, idempotent retry, audited break-glass workflows, and targeted caching are what turn the pipeline from a fragile script held together with `sleep 30` into the control point that catches bad infrastructure changes before they reach production. The history of pipeline automation explains why those guardrails matter, because most of them exist to correct mistakes the previous generation of tooling made routine.

How we got here

The ALZ pipeline consensus landed on reusable workflows, ephemeral runners, and OpenID Connect (OIDC) because every earlier generation of hand-built deploy jobs drifted, leaked state, or hid change history. Early infrastructure pipelines were **Jenkins freestyle jobs** with the Terraform commands pasted into a

textbox; the pipeline definition lived in the Jenkins UI, not in Git, so reviewing a deploy meant taking a screenshot. The "pipeline-as-code" movement (Jenkinsfile in 2016, then Travis/CircleCI YAML, then Azure Pipelines YAML in 2019, then GitHub Actions in late 2019) finally put deploy logic into version control, but every repo copy-pasted the same workflow, and within a year the copies had drifted. **Reusable workflows** (GitHub Actions, 2021) and **YAML templates with extends:** (Azure DevOps) gave central platform teams a way to own the pipeline once and have every consumer stay in sync. Around the same time, the GitOps community pushed the idea of **ephemeral, short-lived runners**, and after a string of self-hosted runner compromises in 2022-2024, that became the security baseline. The patterns in this chapter assume reusable workflows, ephemeral runners, and OIDC auth as the consensus shape of an IaC pipeline in 2026; before you get into those patterns, however, you still need to settle which platform will host them.

Key terms

Pipeline-as-code -- defining your CI/CD pipeline in a version-controlled file (e.g.

`.github/workflows/*.yaml`, `azure-pipelines.yaml`) rather than a GUI, so it is reviewable, auditable, and versionable alongside the code it deploys.

Reusable workflows -- GitHub Actions feature (`workflow_call`) that lets you define a workflow once in a central repository and call it from many consumer repos, keeping pipeline logic DRY.

Matrix builds -- a CI/CD pattern that spawns parallel jobs for each entry in a list (e.g. one job per environment or workload), so changes to many targets are validated concurrently.

Ephemeral runners -- CI/CD build agents that are created for a single job and destroyed afterward, eliminating the risk of credential leakage or cross-job contamination.

`workflow_dispatch` -- a GitHub Actions trigger that allows manually starting a workflow from the UI or API, typically used for break-glass operations.

Idempotent -- a property meaning that running the same operation multiple times produces the same result as running it once. Critical for `apply` retries: re-applying the same plan should not create duplicate resources.

DX (Developer Experience) -- the overall quality of the tooling, documentation, and workflows from a developer's perspective.

Decision framework

Use this sequence to turn pipeline design into explicit choices instead of inherited YAML. The recommended default is GitHub Actions or Azure DevOps Pipelines where your estate already operates, a PR-plan plus main-merge deploy shape, reusable definitions, path-filtered matrices, readable PR plan comments, and a gated break-glass workflow.

1. GitHub Actions or Azure DevOps Pipelines?

- Pick the platform your IdP federation, private network egress, runner model, and audit story already support.

- If the code already lives on github.com, default to GitHub Actions; if it lives on dev.azure.com, default to Azure DevOps Pipelines.
- Do not run both unless a hard compliance, hosting, or migration constraint justifies the extra syntax and operations burden.

2. What is your standard workflow shape?

- Recommended: one PR-triggered workflow that validates and plans, plus one `main`-merge workflow that deploys through environment gates.
- Treat production approval as an environment control, not an ad-hoc prompt inside a shell script.

3. How do you keep CI definitions DRY?

- Use reusable workflows, Azure Pipelines templates, and composite actions so the platform team fixes the pipeline once.
- Version shared pipeline contracts and roll updates through Renovate or Dependabot instead of editing dozens of repos manually.

4. How do you detect which environments to plan or apply on each change?

- Use path filters plus a generated matrix so each changed environment, workload, or subscription is planned independently.
- Maintain a module-to-consumer map so shared module changes trigger every affected environment.

5. How do you surface plan output?

- Post a PR comment with a summary table, highlighted destructive changes, and a collapsed full diff.
- Make the reviewer answer "what will change?" without downloading artifacts or reading raw terminal logs.

6. Do you need a break-glass or manual pipeline for incidents?

- Yes -- build it deliberately with `workflow_dispatch`, constrained inputs, protected environments, required reviewers, and audit notifications.
- A gated manual workflow is safer than engineers running Terraform locally against production state.

Those answers deliberately start with the hosting platform, because the rest of the design is easier once you know where the pipeline will run. The comparison starts there, then the rest of the chapter assumes the same two-workflow shape regardless of which platform you choose.

GitHub Actions vs Azure DevOps Pipelines

Because the decision framework has already narrowed the first choice to platform fit, you should treat GitHub Actions and Azure DevOps Pipelines (ADO) as equally capable ways to deploy Azure Landing Zones and choose the one your source control, identity federation, network egress, and audit controls already make reliable. The comparison is less about feature checkboxes than about operational gravity:

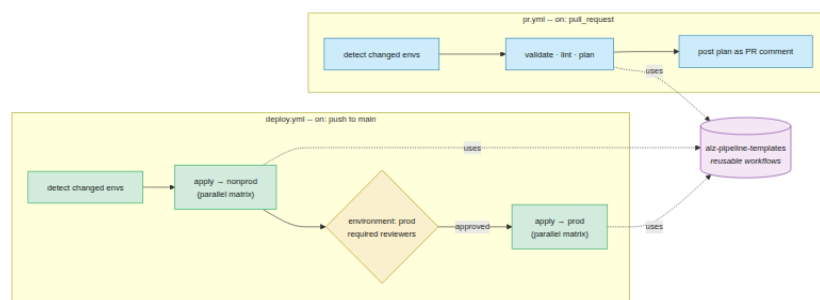
Factor	GitHub Actions	Azure DevOps Pipelines
Already there	Yes if code is on github.com	Yes if code is on dev.azure.com
OIDC to Azure	Mature, simple	Mature ("Workload identity federation")
Reusable workflow / template DX	<code>workflow_call</code> , composite actions	YAML templates, extends
Marketplace ecosystem	Larger (Actions Marketplace)	Smaller
Approval gates / environments	GitHub Environments	Environments + approvals
Cost	Generous free + per-minute	Per-user + per-pipeline minute
Secret store integration	OIDC + Key Vault works well	Variable groups + Key Vault native

In practice, the source control location should win: if the code lives on github.com, use GitHub Actions; if it lives on dev.azure.com, use ADO. Mixing both platforms is defensible only when a hard compliance, hosting, or migration constraint outweighs the cognitive overhead of two pipeline syntaxes.

The patterns below are illustrated with **GitHub Actions**, although the same shape works in ADO. Whichever platform you choose, the anatomy is the same -- and it fits in two files, which is why the workflow shape matters more than the logo on the runner.

The standard two-workflow shape

Once the platform is chosen, every IaC repo should expose the same two entry points: a PR validation workflow that proves the change is safe, and a main-branch deploy workflow that applies it through environment gates from non-prod into prod. That separation keeps review and deployment coupled without making them the same event. The PR workflow gives reviewers a plan before merge, the deploy workflow applies only after the protected branch accepts the change, and both delegate the actual work to a shared reusable workflow so the per-repo files stay short and consistent.



The diagram's central dependency is the shared **reusable workflow** repository, such as `alz-pipeline-templates`, because the two local files should describe *when* work runs rather than reimplementing *how* Terraform or Bicep runs.

pr.yml (validation)

```

name: PR validation

on:
  pull_request:
    branches: [main]

permissions:
  contents: read
  id-token: write
  pull-requests: write # to post the plan as a PR comment

concurrency:
  group: pr-${{ github.event.number }}
  cancel-in-progress: true

jobs:
  detect:
    # Detect which envs/workloads changed → matrix output
    runs-on: ubuntu-latest
    outputs:
      changed: ${ steps.detect.outputs.changed }
    steps:
      - uses: actions/checkout@<sha>
        with: { fetch-depth: 0 }
      - id: detect
        run: ./scripts/detect-changed-envs.sh >> $GITHUB_OUTPUT

  validate:
    needs: detect
    if: needs.detect.outputs.changed != '[]'
    strategy:
      fail-fast: false
      matrix:
        target: ${ fromJson(needs.detect.outputs.changed) }
    uses: contoso/alz-pipeline-templates/.github/workflows/tf-validate.yml@v2
    with:
      working-directory: envs/${{ matrix.target }}
    secrets: inherit

```

deploy.yml (apply)

```

name: Deploy

on:
  push:
    branches: [main]
    workflow_dispatch:

permissions:
  contents: read
  id-token: write

concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: false # never cancel an in-flight apply

jobs:
  detect:
    runs-on: ubuntu-latest
    outputs:
      changed: ${ steps.detect.outputs.changed }
    steps:
      - uses: actions/checkout@<sha>
        with: { fetch-depth: 0 }
      - id: detect
        run: ./scripts/detect-changed-envs.sh >> $GITHUB_OUTPUT

  nonprod:
    needs: detect
    strategy:
      max-parallel: 4
      matrix:
        target: ${ fromJson(needs.detect.outputs.changed) }
    uses: contoso/alz-pipeline-templates/.github/workflows/tf-apply.yml@v2
    with:
      working-directory: envs/nonprod/${{ matrix.target }}
      environment: nonprod
      secrets: inherit

  prod:
    needs: nonprod
    strategy:
      max-parallel: 4
      matrix:
        target: ${ fromJson(needs.detect.outputs.changed) }
    uses: contoso/alz-pipeline-templates/.github/workflows/tf-apply.yml@v2
    with:
      working-directory: envs/prod/${{ matrix.target }}
      environment: prod # ← reviewers configured here
      secrets: inherit

```

With those two files in place, the **GitHub Environment prod** becomes the approval boundary rather than an ad-hoc prompt buried inside a shell script. It has:

- Required reviewers (e.g. 2 from the platform team).
- A **wait timer** (e.g. 10 minutes) so a "ship it" can be aborted.

- **Deployment branches** restricted to `main`.
- The **OIDC client-id vars** for the prod service principal (SPN).

That boundary is only useful if the jobs behind it are consistent, which is why the next layer down is the reusable workflow that every repo calls.

Reusable workflow (the actual work)

Those short repo workflows work because the Terraform or Bicep mechanics live in a reusable workflow, where every repo calls the same tested deploy contract instead of carrying its own slightly different shell script. The reusable workflow is therefore the single source of truth for Azure login, tool setup, plan/apply behavior, and failure reporting, and every consumer repo calls it through the same versioned interface.

`alz-pipeline-templates/.github/workflows/tf-apply.yml`:

```
on:
  workflow_call:
    inputs:
      working-directory: { required: true, type: string }
      environment:       { required: true, type: string }

jobs:
  apply:
    runs-on: ubuntu-latest
    environment: ${ inputs.environment }
    defaults: { run: { working-directory: ${ inputs.working-directory } } }
    steps:
      - uses: actions/checkout@<sha>
        with: { persist-credentials: false }

      - uses: hashicorp/setup-terraform@<sha>
        with: { terraform_version: 1.9.5 }

      - uses: azure/login@<sha>
        with:
          client-id:      ${ vars.AZURE_CLIENT_ID }
          tenant-id:      ${ vars.AZURE_TENANT_ID }
          subscription-id: ${ vars.AZURE_SUBSCRIPTION_ID }

      - run: terraform init -input=false
      - run: terraform plan -input=false -out=tfplan -var-file=terraform.tfvars
      - run: terraform apply -input=false -auto-approve tfplan
      - if: failure()
        run: |
          gh issue create --title "Apply failed: ${ inputs.working-directory }" \
            --body "${ github.server_url }/${ github.repository }/actions/runs/${ github.run_id }"
        env:
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
```

A matching `tf-validate.yml` does `init`, `validate`, `plan`, `tflint`, and `tfsec` or `checkov`, then posts the plan to the PR. That last step is where the reusable workflow becomes visible to reviewers, so the

comment needs to be written for human decision-making rather than for log archival. The next section turns that output into something a reviewer can act on quickly.

PR comments that are actually useful

Once the reusable workflow produces a plan, the PR comment should summarize risk first and hide raw detail behind an expandable section, because reviewers need to answer "what will change?" before they decide whether they care about every provider diff line. Raw plan output dumped into a comment is unreadable, so format it as a review artifact:

- **Collapsed** `<details>` **block** with the full plan inside.
- A **summary table** at the top: `+ N` to add `· ~ M` to change `· - K` to destroy.
- **Highlight destructive changes** in bold.
- For Bicep, post the `what-if` output similarly -- use `--result-format` to get JSON, then render to markdown.

You can use the `hashicorp/setup-terraform` Action's built-in PR comment mechanism, or you can roll your own with `actions/github-script` when you need more control over formatting. Either way, a readable plan comment is only half the story; the other half is making sure the pipeline only plans the things that actually changed, which is where the detect script earns its keep.

Detecting changed environments

The reason the plan comment can stay useful at scale is that the pipeline uses git diff, path filters, and a matrix to plan or apply only the environments affected by a change. That detect script is the small piece of plumbing that makes a layered repo scale:

```
#!/usr/bin/env bash
# scripts/detect-changed-envs.sh
set -euo pipefail
base="${GITHUB_BASE_REF:-main}"
git fetch origin "$base" --depth=1
changed_dirs=$(git diff --name-only "origin/$base"...HEAD \
  | grep '^envs/' \
  | awk -F/ '{print $2/"$3"}' \
  | sort -u)
json=$(printf '%s\n' "$changed_dirs" | jq -R . | jq -s -c .)
echo "changed=${json:-[]}"
```

This produces `["nonprod/connectivity","prod/identity"]`, and the matrix explodes that list into parallel jobs. When a shared module changes, the same mechanism should trigger plan runs in **every** consumer environment of that module, which means you need a module-to-consumer map maintained in the repo rather than a hand-edited exception list in the workflow.

Scaling to subscription vending -- the git-diff matrix pattern

The same detect-and-matrix pattern becomes even more important when your repo vends application landing zone subscriptions, as described in [01 Repository topology -- subscription vending](#), because a sequential mega-apply does not survive hundreds of subscriptions:

1. Each vended subscription has a config file (`.tfvars` or `.bicepparam`) in a flat or shallow directory structure (e.g. `subscriptions/app01-prod/`).
2. The `detect-changed-envs.sh` script (or equivalent) uses `git diff` to identify which subscription directories changed.
3. The pipeline builds a **matrix** from the changed set and deploys each subscription independently and in parallel -- not a sequential mega-apply.

```
# GitHub Actions example -- subscription vending matrix
jobs:
  detect:
    runs-on: ubuntu-latest
    outputs:
      subscriptions: ${ steps.detect.outputs.changed }
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - id: detect
        run: |
          changed=$(git diff --name-only origin/main...HEAD \
            | grep '^subscriptions/' \
            | awk -F/ '{print $2}' | sort -u \
            | jq -R . | jq -s -c .)
          echo "changed=${changed}" >> "$GITHUB_OUTPUT"

  deploy:
    needs: detect
    if: needs.detect.outputs.subscriptions != '[]'
    strategy:
      matrix:
        subscription: ${ fromJson(needs.detect.outputs.subscriptions) }
      max-parallel: 10
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying ${ matrix.subscription }"
        # ... terraform plan/apply scoped to this subscription
```

The result is that adding a new subscription becomes a **single config file**, and the pipeline discovers it automatically instead of waiting for someone to remember a second YAML edit.



From the ALZ Weekly Questions -- [Subscription Vending: Repo Structure, Security & Multi-Tenant](#)

The git-diff matrix pattern is the recommended CI/CD approach for subscription vending. It avoids pipeline timeouts and allows 10+ parallel deployments.

Detection gives you scale; the next challenge is keeping a growing fleet of repos from drifting apart.

Pipeline-as-code, but DRY

Detection gives you scale inside a repo, but consistency across repos comes from centralizing pipeline logic and versioning it, because per-repo YAML copies diverge faster than platform teams can review them.

Several patterns keep many repos consistent without hiding the pipeline from the teams that depend on it:

- **Reusable workflows** (`workflow_call`) -- call the same job from many repos, versioned by tag (`@v2`).
- **Composite actions** for short, repo-agnostic snippets (e.g. "post plan to PR").
- **Org-wide required workflows** -- GitHub lets you enforce that a workflow *must* run on every PR in selected repos. Use this for policy/security checks you can't allow to be removed.
- **Renovate** / Dependabot to bump the pinned `@vX.Y.Z` references in every consumer when you ship a new template version.

Keeping workflows aligned through don't repeat yourself (DRY) pipeline contracts solves the consistency problem. The next challenge is reliability, because Azure is eventually consistent and some operations simply fail on the first attempt even when the configuration is correct.

Long-running operations & retry

Because the pipeline now owns repeated execution across many repos, retries should be limited to idempotent, known-flaky operations, and service-specific waits should live where the service knowledge belongs. Some Azure operations are legitimately flaky, including Key Vault soft-delete naming, Microsoft Entra ID propagation, and role assignment lag. You should not paper over those behaviors with `sleep 30`:

- In Terraform, use `time_sleep` only as a last resort; prefer `null_resource` with `local-exec` that polls the desired state.
- Implement **idempotent retry** at the pipeline step level -- `nick-fields/retry` action with `max_attempts: 3` for known-flaky steps.
- Bake bug-specific waits into modules (with comments linking to the upstream issue) rather than at the pipeline level -- the module is where the knowledge belongs.

Even disciplined retry does not cover every incident, so you still need a manual path; the difference is that this path should be more controlled than the normal deploy flow, not less.

Manual operations & the "break glass" pipeline

A manual break-glass path is necessary for incidents, but it should be more constrained, observable, and auditable than the normal deploy path rather than a back door around it. You will need this path eventually, so build it consciously:

- `workflow_dispatch` workflow with **inputs**: target environment, target workload, action (`plan` / `apply` / `import` / `destroy`).
- Restricted to a small group via the `environment: break-glass` (with two reviewers).

- Logs everything; posts a notification to a security channel automatically.

That controlled workflow is far better than engineers running Terraform locally against production state, because it preserves review, environment protection, and audit history even during an incident. With the operational edge cases handled, you can then take a few cheap wins on raw speed that cost almost nothing to add.

Caching and runtime

After the workflow is safe and observable, runtime tuning should stay conservative: cache only deterministic tool artifacts, and spend on larger runners when `init` and `plan` time dominates review feedback. The cheap wins are deliberately narrow:

- Cache `~/.terraform.d/plugin-cache` and the `.terraform` provider directory across runs.
- Use `setup-terraform` with `terraform_wrapper: false` if you parse the plan output yourself -- the wrapper truncates large outputs.
- For Bicep, cache the compiled JSON between `pr` and `deploy` workflows via `actions/cache` keyed on the file SHA.
- Larger runners pay for themselves on `init` heavy plans (more network bandwidth, more CPU for `plan`).

These optimizations should make the same safe pipeline faster, not create a second, less governed path for impatient deployments. That distinction is the common thread in the anti-patterns below.

Anti-patterns

The anti-patterns all share the same failure mode: they hide deploy risk, fork the shared pipeline contract, or turn automation back into a manual handoff. They are tempting precisely because they feel faster in the moment, but each one removes a guardrail the rest of the chapter has been building toward.

- **✗ Per-repo, hand-rolled workflows.** They diverge within a quarter. Use reusable workflows.
- **✗ `apply` without an explicit `plan` artifact** -- what got applied is not necessarily what was reviewed.
- **✗ `continue-on-error: true`** to "make the build green" while investigating. Fix or skip; never silently swallow.
- **✗ Pipelines that need a human to copy a value between steps.** That's not automation, that's a Slack thread.
- **✗ Using `cancel-in-progress: true` on the `deploy concurrency group`.** An in-flight `apply` should always finish; cancellation can corrupt state.
- **✗ One pipeline that deploys all environments serially in one run.** Use environment gates between non-prod and prod, not a long script.

The patterns in this chapter give the pipeline its shape. What that pipeline *validates* -- the static checks, policy assertions, and integration tests -- is the subject of the next chapter. A fast, well-structured pipeline

running weak checks still lets bad changes through; what you run inside the workflow matters as much as how the workflow is arranged.

References

These references support the reusable workflow, environment gate, template, and Terraform automation patterns recommended above.

- GitHub, *Reusable workflows*: <https://docs.github.com/actions/using-workflows/reusing-workflows>
 - GitHub, *Required workflows*: <https://docs.github.com/actions/using-workflows/required-workflows>
 - GitHub, *Environments*: <https://docs.github.com/actions/deployment/targeting-different-environments/using-environments-for-deployment>
 - Azure DevOps, *YAML templates*: <https://learn.microsoft.com/azure/devops/pipelines/process/templates>
 - Hashicorp, *Run Terraform in CI*: <https://developer.hashicorp.com/terraform/tutorials/automation/automate-terraform>
 -  ALZ Weekly -- Subscription Vending: Repo Structure, Security & Multi-Tenant: <https://www.youtube.com/watch?v=11PmT0t6TUI>
-

[← 07 State management](#) · [Index](#) · [09 Testing & policy](#) →

09 · Testing, validation & policy-as-code

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [The testing pyramid for IaC](#)
- [Layer 1 -- Static checks \(every commit\)](#)
- [Layer 2 -- Policy-as-code on the plan](#)
- [Layer 3 -- Live integration tests](#)
- [Policy-as-code vs Azure Policy](#)
- [Test coverage targets](#)
- [Anti-patterns](#)
- [References](#)

Decision: *what automated checks block a PR, and what's the contract between platform and application teams expressed as policy?*

← [08 CI/CD pipeline patterns](#) · [Index](#) · [10 Code quality](#) →

For infrastructure as code (IaC), your testing strategy should behave like a three-layer quality gate: run static checks on every commit, enforce policy-as-code against the resolved plan before merge, and reserve live integration tests for foundation and reusable modules where fidelity justifies the cost. That gate still needs Azure Policy behind it, because pull request (PR)-time checks give developers fast feedback while runtime policy catches portal, command-line, and emergency changes that never passed through the repository. The practical target is 100% policy and integration coverage for foundation, roughly 80% unit and plan-policy coverage for shared modules, and plan policy plus selective integration for landing-zone compositions. The reason this layered model exists is easier to see if you start with how infrastructure testing evolved.

How we got here

For most of Terraform's first decade, "testing" meant *running `terraform plan` and squinting at the diff*. The brave wrote shell scripts that ran `apply`, hit a few endpoints with `curl`, then `destroy` d everything -- a category formalised in 2018 as **Terratest** (Gruntwork's Go library). It worked, but writing infrastructure tests in Go was a steep ask for ops teams, and Kitchen-Terraform never quite caught on.

That friction explains why the **policy-as-code** movement changed the centre of gravity. HashiCorp's Sentinel and the Cloud Native Computing Foundation's **Open Policy Agent (OPA)** made it practical to parse the plan and reject bad changes before they touched Azure, rather than running the deployment and checking the result afterwards. Microsoft's **PSRule for Azure** (2020) then brought hundreds of Azure Well-Architected Framework-aligned rules out of the box, while Bridgecrew's **Checkov** (2019) and Aqua's **tfsec** (2019) made

multi-cloud scanning trivial. Native **terraform test** finally landed in Terraform 1.6 (October 2023), which made real integration testing accessible to anyone who could write HashiCorp Configuration Language (HCL).

The result is the modern IaC pipeline as a **defence-in-depth pyramid**: static linting, plan-time policy, integration tests on modules, and runtime Azure Policy as the safety net. The decision framework below turns that history into specific tool choices before the layer-by-layer analysis.

Key terms

Policy-as-code -- expressing governance rules (naming, allowed SKUs, required tags) in version-controlled code so they are testable, reviewable, and enforceable automatically.

Open Policy Agent (OPA) -- a CNCF-graduated general-purpose policy engine. Policies are written in the **Rego** language and evaluated against structured data (e.g. a Terraform plan JSON).

SARIF (Static Analysis Results Interchange Format) -- a JSON-based standard for reporting findings from static-analysis tools. GitHub's code scanning UI can ingest SARIF files directly.

Terratest -- a Go library by Gruntwork for writing automated integration tests that deploy real infrastructure, validate it, and tear it down.

CIS (Center for Internet Security) -- an organisation that publishes security benchmarks (e.g. CIS Azure Foundations Benchmark) used by tools like Checkov and PSRule.

Dynamic blocks -- a Terraform construct (`dynamic "block_name" { ... }`) that generates repeated nested blocks from a collection, used to keep module code DRY.

Fixture resources -- pre-created, known-good resources (or resource definitions) used as stable inputs for integration tests, avoiding hard-coded names that cause collisions.

Smoke test -- a lightweight, fast test that checks whether the most critical path works at all (e.g. "did the deployment succeed and is the resource reachable?"), without exhaustive validation.

Decision framework

With that history and vocabulary in mind, pick the test strategy by answering these questions in order. The goal is not to maximise tools; it is to put the cheapest reliable gate in front of each class of failure and reserve expensive live tests for the layers where they buy real confidence.

1. Which static-check tools do you mandate?

- Terraform estates should require **terraform fmt**, **terraform validate**, **tflint**, **terraform-docs**, and secret scanning.
- Bicep estates should require **bicep format**, **bicep build**, the Bicep linter, PSRule where applicable, and secret scanning.
- Cross-cutting checks should include YAML/JSON/Markdown formatting, **actionlint** for GitHub Actions, and security scanning for workflows.

2. Which policy-as-code engine evaluates the plan?

- Pick one primary engine for the PR gate so developers get a consistent failure model.
- Use **PSRule for Azure** as the Azure-first default, **Checkov** for broad multi-IaC scanning, and **Conftest/OPA** when you need highly custom organisation-specific rules.
- Combining engines is acceptable, but only when each has a clear job and duplicate findings are suppressed.

3. How much live integration testing is justified?

- Foundation and shared modules deserve live tests because their blast radius is high and they are reused everywhere.
- Workload compositions usually start with plan/policy checks and add Terratest, `terraform test`, or `az deployment what-if` smoke tests only for complex module interactions or high-risk changes.
- Run live tests in a sandbox subscription with per-PR isolation and automatic teardown.

4. How do you reconcile policy-as-code with Azure Policy?

- Treat them as defence in depth: policy-as-code gives pre-deploy PR feedback, while Azure Policy enforces continuously after deployment and catches portal/CLI drift.
- Neither replaces the other; the sustainable pattern is to generate both from the same control intent where possible.

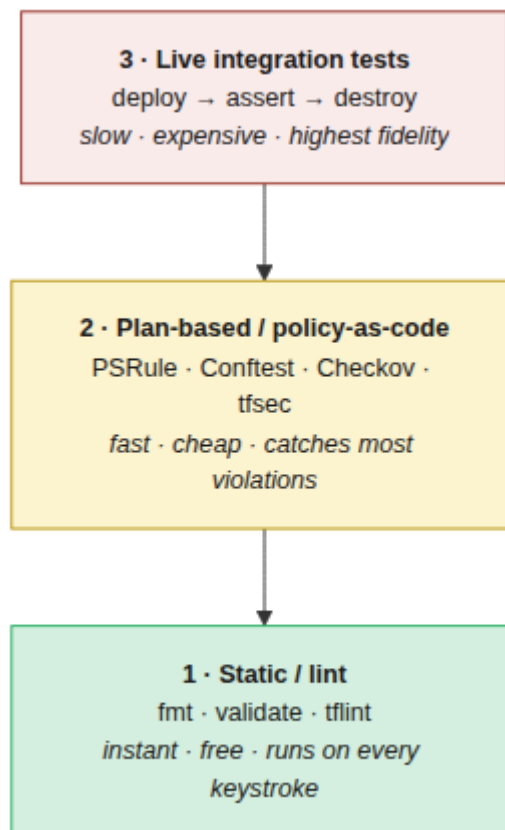
5. What is your coverage target by layer?

- Foundation: 100% policy coverage plus integration tests for critical deployment paths.
- Modules: about 80% unit/plan-policy coverage plus integration tests for major code paths.
- Landing zones: static and plan-policy coverage for every PR, with selective integration tests for complex compositions.

The full analysis -- why each layer exists, which tools fit, and where the tradeoffs appear -- follows below.

The testing pyramid for IaC

The IaC gate works best as a pyramid: static checks run everywhere, plan-time policy evaluates every PR, and live tests are reserved for the places where the extra cost buys confidence that cheaper gates cannot provide.



The bottom is cheap and fast, while the top is expensive and slow, so you should push every check downward until it loses fidelity and keep only the remaining high-value scenarios at the live-test layer. That starting point matters because the first layer should feel almost invisible to the developer, which is why static checks belong on every commit rather than in a weekly quality ritual.

Layer 1 -- Static checks (every commit)

Because static checks sit at the base of the pyramid, they should be mandatory on every developer machine and every continuous integration (CI) run: they are the fastest way to catch syntax, formatting, documentation, and workflow mistakes before review.

Terraform

Tool	What it does
<code>terraform fmt -check -recursive</code>	Formatting. Fail PR on diff.
<code>terraform validate</code>	Catches typos, missing variables, type errors.
<code>tflint</code>	Provider-aware lints (deprecated args, AzureRM-specific issues). Use the <code>terraform-linters/tflint-ruleset-azurerm</code> plugin.
<code>terraform-docs</code>	Generates module READMEs. CI verifies docs are up to date.

Bicep

Tool	What it does
<code>bicep build</code>	Catches syntax + type errors at compile time.
<code>bicep format --check</code>	Formatting.
<code>bicep lint</code> (built-in)	Common authoring issues; configured via <code>bicepconfig.json</code> .
<u>PSRule for Azure</u>	Static analysis for ARM/Bicep against Azure WAF + CIS rules.

Cross-cutting

- **EditorConfig** + Prettier for YAML / Markdown / JSON.
- `actionlint` for GitHub Actions; `zizmor` for Actions security audit.
- `markdownlint` for docs.
- Run them all from a single `pre-commit` config -- one file, one mental model.

These checks catch what the *author* got wrong: a typo, an invalid type, a stale generated README, or a workflow file that no longer parses. The next layer enforces what the *organisation* requires, which is an entirely different problem because it needs the resolved deployment intent rather than the raw source file.

Layer 2 -- Policy-as-code on the plan

Once the source is clean, the next gate should enforce enterprise rules against the resolved plan at PR time so non-compliant resources are rejected before anything is created in Azure.

What to enforce

The controls in this layer are the ones where the organisation cares about the deployed shape, not just whether the code compiles.

- **Naming convention** (resource names match a regex).
- **Required tags** (`CostCenter` , `Owner` , `DataClassification`).
- **Region restrictions** (no resources in `westeurope` if the workload is data-resident in `swedencentral`).
- **Forbidden SKUs** (no Basic SKU public IPs in production; no Standard_LRS storage for prod databases).
- **Network controls** (no public endpoints on storage / SQL; private endpoints required).
- **Encryption** (CMK on key vaults, soft-delete + purge protection enabled).
- **RBAC** (no `Owner` assignments outside the platform team's identity).

Tools

The tool choice is less important than the consistency of the failure model, so pick one primary engine for the PR gate and add specialist engines only where they have a distinct job.

Tool	Best for
<u>PSRule for Azure</u>	Bicep + Terraform; ships hundreds of WAF rules out of the box. Recommended default.
<u>Checkov</u>	Multi-IaC (Terraform, Bicep, ARM, K8s); fast onboarding.
<u>tfsec</u>	Terraform-only; merged into Trivy now.
<u>Conftest</u> (Open Policy Agent/Rego)	When you need to write <i>organisation-specific</i> rules and you're comfortable in Rego.
<u>Sentinel</u> (HCP)	Only if you're on Terraform Cloud/Enterprise.

Pattern: PSRule on Bicep in CI

```
- name: Build Bicep to ARM
  run: bicep build envs/prod/connectivity/main.bicep --outdir build/

- name: Run PSRule
  uses: microsoft/ps-rule@<sha>
  with:
    modules: PSRule.Rules.Azure
    inputPath: build/
    baseline: Azure.Pillar.Security
    outputFormat: Sarif
    outputPath: psrule.sarif

- uses: github/codeql-action/upload-sarif@<sha>
  with:
    sarif_file: psrule.sarif
```

With this pattern, PSRule findings show up in the GitHub **Security** tab and on the PR, which keeps governance feedback in the same review surface that developers already use.

Pattern: Checkov on Terraform plan

```
- run: terraform plan -out tfplan && terraform show -json tfplan > tfplan.json
- uses: bridgecrewio/checkov-action@<sha>
  with:
    file: tfplan.json
    output_format: sarif
    soft_fail: false
```

Here again, the resolved plan is the important boundary: checking against the **plan JSON** is more accurate than checking `.tf` files because it captures resolved variables, modules, and dynamic blocks.

Custom Rego example (Conftest)

When the built-in rule sets do not express your exact control, a small Open Policy Agent rule can carry the organisation-specific intent. This example forbids public storage accounts in any non-sandbox environment:

```
package main

deny[msg] {
  resource := input.resource_changes[_]
  resource.type == "azurerm_storage_account"
  resource.change.after.public_network_access_enabled == true
  not is_sandbox(input)
  msg := sprintf("storage account %q must not allow public network access",
[resource.address])
}

is_sandbox(plan) {
  plan.variables.environment.value == "sandbox"
}
```

```
conftest test --policy ./policies tfplan.json
```

Plan-time policy stops non-compliant configuration from ever touching Azure, which is already a large improvement over discovering the problem after `apply`. For well-isolated modules, though, you sometimes need to go one step further and prove the thing actually deploys and behaves correctly under real conditions.

Layer 3 -- Live integration tests

Because live tests sit at the expensive top of the pyramid, you should deploy, assert, and destroy real infrastructure only for foundation and reusable modules by default; add workload integration tests when composition risk is higher than the CI cost.

Terraform

- **terraform test** (built-in since 1.6) -- assertion blocks, can run ephemeral applies. Default choice in 2026.
- **Terratest** (Go) -- older, more flexible, but heavier maintenance.

```
# tests/main.tfctest.hcl
run "creates_hub_vnet" {
  variables {
    address_space = "10.0.0.0/16"
    location      = "swedencentral"
  }

  assert {
    condition      = output.vnet_id != ""
    error_message = "Hub VNet was not created"
  }
}
```

Bicep

- **PSRule unit tests** for static rules.
- `az deployment what-if` against a sacrificial resource group as a smoke test.
- For full integration: deploy via Deployment Stack to a per-PR resource group, run `az` queries to assert state, then `az stack sub delete --action-on-unmanage deleteAll`.

Where they run

- **Module repos:** every PR runs the full integration test suite in a dedicated test subscription.
- **Workload repos:** integration tests are usually unnecessary if your modules are well-tested. Plan/policy checks suffice.

The debate -- can you skip integration tests for workloads?

*The advice above ("plan/policy checks suffice for workloads") saves time and CI cost, but it's a **risk acceptance decision**, not a universal best practice.*

The counter-argument: Modules are unit-tested in isolation; they validate that the module works, not that your specific composition of modules works. Unique parameter combinations, conditional feature flags, cross-module data flows, and custom policies can all produce failures that only integration tests catch. Teams that have deployed workload changes relying solely on plan-time checks have hit issues -- resource dependency ordering, eventual-consistency race conditions, private-endpoint DNS propagation -- that only manifest at apply time.

When skipping is reasonable: If your tier-2 modules are heavily tested, your workload compositions are simple (thin wrappers calling modules with `.tfvars`), and you have strong runtime policy as a safety net, the residual risk may be low enough to accept.

When it isn't: If workloads compose modules with complex conditionals, `for_each` over dynamic data, or cross-stack references, integration tests on the composition are the only way to catch interaction bugs before production.

At this point you have checks at every stage of development, from the developer workstation to the live module test. There is still a gap, however: nothing in the repository prevents someone from creating a non-compliant resource directly through the portal or the command line. That is where Azure Policy comes in -- and it needs to stay in sync with everything above.

Policy-as-code vs Azure Policy

The repository gate and Azure Policy should operate as defence in depth from a shared control intent, because using only one leaves either developer feedback or runtime enforcement uncovered.

The distinction is easier to reason about if you treat policy as two layers with different timing and blast-radius characteristics:

1. **Repo-side (PR-time) policy** -- Checkov/PSRule/Conftest. Catches issues *before* deployment. Fast feedback, free, scoped to what you can see in code.

2. Platform-side (runtime) Azure Policy -- applied at the management group / subscription, enforces continuously, including for resources created outside IaC.

You need **both** because PR-time policy gives instant developer feedback and prevents the bad PR from merging, while Azure Policy is the *insurance* that catches anything the PR check missed, including resources created through the portal, CLI, or scripts.

The two should therefore be **expressed from the same intent**. A common pattern looks like this:

- Source of truth: a YAML file describing each control.
- Generator script produces:
 - A PSRule / Checkov rule for PR time.
 - An Azure Policy `policyDefinition` and assignment for runtime.

That way, drift between what CI checks and what Azure enforces is impossible by construction, which is the only sustainable way to keep the two-layer model trustworthy.

The debate -- does PR-time policy violate "Azure as the single control plane"?

One of the CAF design principles for Azure Landing Zones is to use **Azure as the single platform for operations management and policy**. Adding PSRule, Checkov, or Conftest as a second enforcement layer at PR time creates a tension with that principle: you now have policy logic in two places -- Azure Policy in the cloud and linting rules in your CI pipeline -- with no guarantee they stay in sync.

The case for Azure Policy only:

- One control plane means one place to audit, one place to update, and one compliance dashboard. No rule drift, no "it passed CI but Azure Policy denied it" confusion.
- Azure Policy's `what-if` and `DoNotEnforce` assignment modes can surface violations before deployment, partially closing the feedback gap.
- Fewer tools to maintain, licence, and train on.

The case for dual-layer:

- Azure Policy acts after deployment (or at deployment time at the earliest). PR-time checks catch issues minutes into a developer's workflow, not after a 15-minute `terraform plan` + `apply` cycle.
- Not all controls map cleanly to Azure Policy. Code-level concerns -- "this module is missing a `description`", "this variable has no validation block" -- are invisible to Azure Policy because they don't exist as deployed resources.
- Azure Policy cannot block a PR from merging. If your governance model requires that non-compliant code never reaches the main branch, you need a CI-side gate.

Where most teams land: the dual-layer approach wins in practice, but the "single source of truth" pattern described above is non-negotiable -- without it, rule drift between the two layers erodes trust in both. Teams that skip the generation step and hand-maintain parallel rule sets in Azure Policy and PSRule/Checkov inevitably diverge, and the resulting "it passed CI but was denied at deploy" incidents undermine the entire governance model.

Azure Policy lifecycle in this repo

That shared-intent model becomes concrete in the repository when policy objects have a predictable lifecycle and a clear promotion path.

- Policy *definitions* in code (Bicep/Terraform).
- Policy *assignments* in the foundation/policy repo.
- Initiative/Set definitions for grouped controls.
- On PR: `az policy assignment create --enforcement-mode DoNotEnforce` against a test management group, then run a compliance scan, then destroy.
- On merge: deploy with `Default` enforcement.

For the effect itself, treat audit and deny as stages in the control's lifecycle rather than as arbitrary preferences.

- **Audit:** rolling out a new control. Run for ≥ 2 weeks, generate the exemption list from existing non-compliant resources, then flip to deny.
- **Deny:** steady state for any control where remediation is cheap and the violation has a real impact.

Built-in policy versioning

That lifecycle becomes more subtle when you assign Microsoft built-ins, because Azure built-in policies now carry a **version field** in their metadata (SemVer: `MAJOR.MINOR.PATCH`). Microsoft updates built-in definitions in place, so your existing assignments automatically pick up the latest version unless you pin to a specific one.

 **From the ALZ Weekly Questions -- How to Stay Current with ALZ Azure Policies Patch versions auto-apply** -- you cannot pin to a patch. Minor and major version updates require you to update the reference in your library metadata. This means your compliance posture can shift at the patch level without a PR in your repo.

This creates a **silent drift risk**: a built-in policy you assigned two years ago may have changed its logic, added parameters, or expanded its scope. Your compliance posture shifts without a PR, a review, or a test in your pipeline, so the repository needs compensating controls of its own.

You mitigate that risk by making built-in changes visible before they become operational surprises:

- **Snapshot built-in definitions in your repo.** Export the JSON of every built-in you assign (via `az policy definition show` or the [Azure/azure-policy](#) repo) and store it alongside your assignments. A scheduled CI job diffs the live definition against your snapshot and opens a PR when the version changes -- giving your team a chance to review before accepting.
- **Subscribe to the Azure Policy changelog.** Microsoft publishes [built-in policy change logs](#); feed them into your governance channel.
- **Test assignments, not just definitions.** When a built-in updates, re-run your compliance scan against your fixture resources to verify the new version doesn't flag resources that were previously compliant (false positives) or miss resources that should be flagged (false negatives).

Updating ALZ policies -- the practical workflow

Those compensating controls only help if policy updates are routine rather than exceptional. Whether you use Bicep or Terraform, the ALZ library is the **source of truth** for which policies are assigned and at what version, so updating policies should be a normal Day-2 workflow rather than a special project.

Terraform workflow:

1. Update `metadata.json` in your repo to reference a newer ALZ library version.
2. Run `terraform plan` -- the ALZ provider dynamically pulls the library and shows what policy definitions, assignments, or role definitions changed.
3. Review the plan output carefully -- don't blindly approve.
4. Merge the PR → pipeline runs `terraform apply`.

Bicep workflow:

1. Delete the local `lib/` directory that contains the generated ALZ library files.
2. Run the `alzlib` tool to regenerate from the latest library version: `alzlib generate`.
3. Run the `Update-ALZReferences.ps1` script to align your parameter files.
4. Open a PR -- the pipeline runs `what-if` to show changes.
5. Merge → pipeline deploys via Deployment Stacks (which automatically clean up deprecated policy assignments).

 *From the ALZ Weekly Questions -- [How to Stay Current with ALZ Azure Policies Deployment Stacks](#) give Bicep a major advantage here: when a policy is removed from the ALZ library, the stack automatically deletes the orphaned assignment. Terraform achieves the same via state tracking, but Bicep teams historically had to clean up manually.*

EPAC as an alternative policy management tool

If the ALZ-native workflow is too narrow for your operating model, **Enterprise Policy as Code (EPAC)** is the main alternative to evaluate. EPAC is a community-driven tool that provides a structured way to manage Azure Policy assignments at scale, especially across **multi-tenant** environments, and it uses a declarative JSON/CSV format for complex policy ecosystems with hundreds of assignments.

EPAC makes sense when its additional structure solves a real scale problem:

- Large multi-tenant estates where the ALZ-native policy management feels limiting.
- Organisations that want a single policy repository covering multiple ALZ instances.
- Teams that need advanced features like policy exemption management, effect overrides, and compliance reporting.

The same flexibility also introduces risks that you need to own before adopting it:

- EPAC is **community-driven** -- it has no Microsoft product lifecycle, no SLA, and no guaranteed long-term support.
- If the maintainers step away, you own the codebase. Only adopt EPAC if your team has the skills and willingness to **fork and maintain** it independently.

- EPAC has its own sequencing requirements that can conflict with how ALZ deploys policies. Integration requires careful planning.

If you do not use EPAC but still want external policy management, you can disable ALZ-native policy assignments and manage policies entirely through your own definitions:

- **Bicep:** set the policy module references to `null` or `no` in the configuration.
- **Terraform:** use an **empty archetype** that deploys management groups without any policy assignments.

 *From the ALZ Weekly Questions -- Using EPAC for Azure Policy in ALZ EPAC is a power tool -- powerful in expert hands, dangerous if adopted without deep understanding. The ALZ team's position: if you don't understand EPAC well enough to fork it, you probably shouldn't depend on it.*

The custom → built-in lifecycle

As the policy estate matures, custom controls should not become permanent by accident. A common lifecycle starts with a custom policy, then forces you to revisit it when Microsoft ships an equivalent built-in:

1. **You write a custom policy** because no built-in exists for a specific control (e.g. "deny storage accounts without infrastructure encryption").
2. **Months later, Microsoft ships a built-in** that covers the same control -- often with better alias coverage, edge-case handling, and ongoing maintenance.
3. **You now maintain a custom policy that duplicates a built-in** -- accruing maintenance cost and risking divergence.

The responsible lifecycle is to review those custom policies deliberately instead of letting duplicate controls accumulate:

- **Inventory custom policies quarterly.** For each, check whether a built-in equivalent now exists. The `Azure/azure-policy` repo and `az policy definition list --filter "policyType eq 'BuiltIn'"` are your search tools.
- **Don't auto-swap.** A built-in may have different parameter names, a broader or narrower scope, or different default values. Test the built-in against your estate in audit mode before replacing.
- **Migration steps:**
 1. Assign the built-in in **audit mode** alongside the custom policy.
 2. Compare compliance results -- they should match. Investigate discrepancies.
 3. Once confident, remove the custom assignment and flip the built-in to **deny** (or whichever effect applies).
 4. Retire (but keep archived) the custom definition.
- **Tag custom policies with `lifecycle: custom-pending-review`** so your quarterly inventory script can flag them automatically.

The debate -- should you always prefer built-ins?

*Built-ins are maintained by Microsoft and get free updates, but they are also **opaque**: you can't modify their logic, and a version bump can change behaviour without your consent. Custom policies give you full control at the cost of full maintenance. Some teams prefer to wrap built-ins in initiatives with explicit version pins and treat the built-in as an upstream dependency (similar to how you'd pin a module version). Others mandate custom-only for critical controls so that every change flows through their PR process. Neither approach is universally superior -- the right balance depends on how much governance automation your team can sustain.*

After you have made that lifecycle decision, the remaining question is how much coverage each layer deserves.

Test coverage targets

Once the pyramid and policy lifecycle are clear, coverage targets should be set by layer rather than by vanity percentage: foundation needs exhaustive policy and integration coverage, modules need broad unit and plan-policy coverage, and landing zones need plan-policy coverage plus selective live tests.

Artifact	Minimum coverage
Foundation / policy layer	100% policy coverage + integration tests for critical deployment paths
Tier-2 pattern modules	Static + ≥80% unit / plan-policy coverage + ≥ 1 integration test per major code path
Tier-3 workload composition	Static + plan-policy on every PR + selective integration tests for complex or high-risk compositions
Custom Azure Policy	Compliance test against a fixture resource that should pass + a fixture that should fail
Pipeline templates	Unit test the templates with <code>act</code> or by running them against a sample repo

These targets are intentionally conservative because a 45-minute test suite that engineers skip is worse than no tests at all. Cut scope ruthlessly, parallelise what remains, and treat build time as a metric worth watching; otherwise the pyramid becomes a theoretical control that your teams route around in practice.

Anti-patterns

The failures to avoid are the same ones the pyramid is designed to prevent: late feedback, unenforced findings, and tests so slow or brittle that engineers route around them.

- ✗ **All policy lives in Azure Policy.** Developers find out at deploy time, after they've waited for a 15-min `terraform plan` to come back. Move what you can to PR time.
- ✗ **All policy lives at PR time.** Anyone clicking in the portal bypasses your controls.
- ✗ **`soft_fail: true` on Checkov to "fix later".** Later never comes.

-  **Integration tests against a shared test subscription with hard-coded names.** Two PRs run simultaneously → name collision → both fail.
-  **Tests that take 45 minutes.** Engineers will avoid them. Parallelise or trim coverage.

The discipline of layered validation ultimately comes down to shortening the feedback loop: moving pain from the 4 AM incident to the 30-second pre-commit hook. Once that loop is tight, the question shifts to the *experience* of working inside the repo day-to-day -- which is the subject of the next chapter. The references below provide the tool and policy background for the testing model described here.

References

- PSRule for Azure: <https://azure.github.io/PSRule.Rules.Azure/>
 - Checkov: <https://www.checkov.io/>
 - Conftest: <https://www.conftest.dev/>
 - Hashicorp, *terraform test* : <https://developer.hashicorp.com/terraform/language/tests>
 - Terratest: <https://terratest.gruntwork.io/>
 - Microsoft, *Azure Policy as code*: <https://learn.microsoft.com/azure/governance/policy/concepts/policy-as-code>
 - OPA / Rego: <https://www.openpolicyagent.org/docs/latest/policy-language/>
 -  ALZ Weekly -- *How to Stay Current with ALZ Azure Policies*: https://www.youtube.com/watch?v=ddcVKS_MKkk
 -  ALZ Weekly -- *Using EPAC for Azure Policy in ALZ*: https://www.youtube.com/watch?v=x1I_XhC6GtA
-

[← 08 CI/CD pipeline patterns](#) · [Index](#) · [10 Code quality](#) →

10 · Code quality, linting & developer experience

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [The "five-minute onboarding" target](#)
- [Devcontainer / mise / asdf](#)
- [Makefile as a UX layer](#)
- [Pre-commit hooks](#)
- [Auto-generated module documentation](#)
- [Conventional commits + automated releases](#)
- [Editor configuration](#)
- [Repository templates](#)
- [Issue & PR templates](#)
- [Local development against Azure](#)
- [Anti-patterns](#)
- [References](#)

Decision: *what does it look like to clone the repo and submit a high-quality PR on day one? DX is a force multiplier -- invest in it.*

[← 09 Testing & policy · Index · 11 Manageability →](#)

Design the repo so a new engineer can clone it, run `make bootstrap`, produce a safe plan, and open a high-quality pull request (PR) on day one -- ideally in five minutes. To make that promise real, standardise on one supported setup path: devcontainer for Visual Studio Code and Codespaces estates, or `mise` for lighter local workflows; wrap commands in `make`; enforce formatting, linting, secret scanning, and commit-message rules with pre-commit and continuous integration (CI); and generate module docs instead of hand-writing them. Developer experience is not polish: it is how Azure Landing Zone (ALZ) teams keep quality high without depending on memory or tribal knowledge, and that is why the chapter starts with the path a new contributor actually walks.

How we got here

The original "developer experience" of an infrastructure as code (IaC) repo was a one-line README: "install Terraform and run `terraform apply`." Three months later, half the team was on Terraform 0.11 and half on 0.12, and a fresh clone took an afternoon to bootstrap. Tooling caught up in waves: `tflint` (2018) added Azure-aware lints, `terraform-docs` (2019) generated module READMEs from the schema, and `pre-commit-terraform` (2019) bundled the common checks behind a single `pre-commit install`. The

biggest leap was the **devcontainer** specification (Microsoft, open-sourced 2022) and **GitHub Codespaces**; for the first time, you could pin every tool, every extension, and every shell helper in one JSON file and have CI use the *identical* image. Tools like **mise** and **asdf** brought the same discipline to engineers who do not want a container around their editor. A mature ALZ repo in 2026 should therefore let a new joiner go from `git clone` to a green `plan` in under five minutes, not because the number is magical, but because the repo should have removed avoidable choices before the engineer arrives.

Key terms

Devcontainer -- a specification (`.devcontainer/devcontainer.json`) that defines a reproducible development environment inside a container, including all tools, extensions, and settings. GitHub Codespaces and VS Code use it natively.

mise / asdf -- version-manager tools that pin CLI tool versions (Terraform, az, tfint) per project via a dotfile (`.mise.toml` / `.tool-versions`), ensuring every developer uses the same versions without containers.

Taskfile -- a modern alternative to `Makefile` (`Taskfile.yml`) for defining project-level commands (e.g. `task plan` , `task lint`), with YAML syntax and dependency support.

Pre-commit hooks -- Git hooks that run checks (formatting, linting, secret scanning) automatically before each `git commit` , catching issues at the earliest possible stage.

Conventional Commits -- a commit-message convention (e.g. `feat:` , `fix:` , `chore:`) that enables automated changelog generation and semantic version bumping.

EditorConfig -- a simple dotfile (`.editorconfig`) that standardises indent style, charset, and line endings across editors and IDEs.

Codespaces -- GitHub's cloud-hosted development environment that launches a devcontainer in the browser, eliminating local setup entirely.

With those terms in place, the remaining question is no longer whether developer experience matters, but which baseline you make mandatory before the repo scales beyond the first few contributors. The decision framework below turns that question into a short sequence of choices you can apply to a real repository.

Decision framework

Choose the developer-experience baseline by answering these questions in order. The answers decide which setup path is mandatory, which checks run locally, and how much automation the repo needs before teams scale it.

1. What's your "first PR in five minutes" path?

- Document one golden path: `git clone` , `make bootstrap` , `make plan` , open PR.
- If the first run cannot be five minutes, publish the real target and keep reducing it; large estates may choose a fifteen-minute ceiling.

2. Devcontainer, **mise** , or **asdf** for tool versions?

- VS Code / Codespaces standard → devcontainer.
- Mixed editors or no container requirement → `mise` is the recommended lightweight default.
- Existing `asdf` estate → keep `asdf`, but do not support both `mise` and `asdf` unless migration is underway.

3. Which pre-commit hooks are mandatory?

- At minimum: format, lint, secret scan, and conventional-commit check.
- Run the same hooks in CI so local and pipeline failures match.

4. Do you auto-generate module docs?

- Terraform modules → `terraform-docs` with CI drift checks.
- Bicep modules → generated documentation from `bicep generate-params`, `PSDocs.Azure`, or metadata parsing.

5. Do you need Conventional Commits and automated releases?

- Yes if you publish reusable module versions or want Renovate / Dependabot consumers to see meaningful changelogs.
- Optional for a private environment-only repo with no versioned module consumers.

6. Do you need repository templates or a scaffolding tool?

- Use a GitHub template for repeatable landing-zone repos.
- Use Backstage or `copier` when teams need prompts, ownership metadata, or policy-driven scaffolding.

Those answers give you the order of implementation: first make onboarding measurable, then pin the tools, then wrap and enforce the commands that keep every contribution consistent. The first of those steps matters most because it converts developer experience from a feeling into a target you can test.

The "five-minute onboarding" target

After the decision framework has forced you to name the golden path, the first concrete target is a meaningful plan within five minutes, with a published fifteen-minute ceiling only when repo size makes five minutes impossible. A new engineer should be able to run the supported path without choosing tools or reading tribal notes:

```
gh repo clone contoso/alz-platform
cd alz-platform
make bootstrap      # installs everything
make plan ENV=nonprod WORKLOAD=connectivity
```

If that sequence produces a meaningful plan within **five minutes** of the first `git clone`, the repo has turned onboarding into an engineered path rather than a scavenger hunt; if it does not, the extra minutes become a friction tax you pay on every rotation, incident, and pull request.

The debate -- is five minutes realistic?

Five minutes is aspirational and achievable for small-to-mid repos, but practitioners managing large estates (100+ modules, heavy provider sets) report that `terraform init` alone can take 2-3 minutes, and devcontainer image pulls add another 2-5 on first use. Hitting the five-minute target at scale requires pre-built devcontainer images (ongoing maintenance cost), local provider caching, and carefully curated module sets -- effort that not every team can justify.

*A more realistic goal for large estates: **under fifteen minutes** to first meaningful plan. Still fast, less likely to set an expectation that fails on day one. The five-minute target remains the aspiration; document what your actual number is and actively work to reduce it.*

Because large estates can make the first run legitimately slower, the practical target is not a slogan but a measurement you keep reducing. The next two sections describe the foundations that make the measurement credible: **one pinned toolchain** and **one command surface** (`Makefile` or `Taskfile`).

Devcontainer / mise / asdf

The five-minute path only works when the repo, not the individual engineer, decides which tool versions are valid. Pin those versions in the repo, choosing devcontainer for container-first teams and `mise` as the lightweight default when engineers work directly on their machines; keep `asdf` where it is already the estate standard, but avoid supporting both lightweight managers unless you are actively migrating. In practice, the choice usually narrows to these approaches.

Devcontainer (recommended for VS Code / Codespaces shops)

```
// .devcontainer/devcontainer.json
{
  "name": "alz-platform",
  "image": "mcr.microsoft.com/devcontainers/base:ubuntu-24.04",
  "features": {
    "ghcr.io/devcontainers/features/azure-cli:1": {},
    "ghcr.io/devcontainers/features/terraform:1": { "version": "1.9.5" },
    "ghcr.io/devcontainers/features/github-cli:1": {},
    "ghcr.io/devcontainers/features/powershell:1": {}
  },
  "postCreateCommand": "make bootstrap",
  "customizations": {
    "vscode": {
      "extensions": [
        "hashicorp.terraform",
        "ms-azuretools.vscode-bicep",
        "github.vscode-github-actions",
        "redhat.vscode-yaml"
      ]
    }
  }
}
```

The same container then runs locally in Visual Studio Code, in GitHub Codespaces, and, with minor tweaks, in CI, which removes the usual "works on my machine" gap by making the developer shell and the pipeline shell nearly identical.

mise / asdf (lighter)

```
# .mise.toml
[tools]
terraform = "1.9.5"
bicep      = "0.30.3"
azure-cli  = "2.65.0"
node       = "20.18.0"
```

Engineers run `mise install` once and have the same versions as CI, which keeps the local workflow light while still making version drift visible in review.

Once the tools are pinned, the next problem is not correctness but memory: you still need to make those tools easy to invoke without asking every contributor to remember provider-specific incantations. That is where the `Makefile` becomes a user interface rather than a build relic.

Makefile as a UX layer

A pinned toolchain becomes useful only when engineers and CI invoke it through the same short, supported command surface. Hide the long incantations behind memorable `make` targets so the repo teaches the workflow and the pipeline exercises the same path developers use locally.

```

ENV ?= nonprod
WORKLOAD ?= connectivity
TF_DIR := envs/${ENV}/${WORKLOAD}

.PHONY: bootstrap fmt lint plan apply test clean

bootstrap:
    pre-commit install
    cd $(TF_DIR) && terraform init -input=false

fmt:
    terraform fmt -recursive
    bicep format

lint:
    pre-commit run --all-files
    tflint --recursive

plan:
    cd $(TF_DIR) && terraform plan -var-file=terraform.tfvars -out=tfplan

apply:
    @if [ "$(ENV)" = "prod" ]; then \
        echo "Use the pipeline for prod. Refusing to apply locally."; exit 1; \
    fi
    cd $(TF_DIR) && terraform apply tfplan

test:
    cd modules && terraform test

clean:
    find . -type d -name '.terraform' -prune -exec rm -rf {} +

```

The important detail is not just convenience: `make apply` refuses to run against `prod`, which means the command surface can encode governance as well as ergonomics. After the repo has one way to run commands, you can move the fastest quality gates even earlier, before a commit is created.

Pre-commit hooks

The command surface gives contributors a clear way to run checks, but `pre-commit` moves the highest-return checks to the moment when they are cheapest to fix. Make fast local checks mandatory, then run the same hooks in CI so developers see the pipeline verdict before they push.

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-yaml
    - id: check-json
    - id: check-added-large-files

- repo: https://github.com/antonbabenko/pre-commit-terraform
  rev: v1.92.0
  hooks:
    - id: terraform_fmt
    - id: terraform_validate
    - id: terraform_tflint
    - id: terraform_docs
      args: [--args=--config=.terraform-docs.yml]

- repo: https://github.com/bridgecrewio/checkov
  rev: 3.2.255
  hooks:
    - id: checkov
      args: [-d, ., --quiet, --soft-fail] # informational locally; fatal in CI

- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.4
  hooks:
    - id: gitleaks

- repo: https://github.com/rhysd/actionlint
  rev: v1.7.1
  hooks:
    - id: actionlint
```

Run the **same hooks in CI** -- `pre-commit run --all-files` -- so what is caught locally and what is caught in CI is identical. Use a `commit-msg` hook such as `commitlint` / Husky when Conventional Commits are part of the release process; otherwise, the commit convention becomes another rule that only exists after review has already started.

Hooks handle the checks that can be computed quickly. Documentation is a different discipline, and it degrades fastest when you ask engineers to keep repeated reference material synchronized by hand.

Auto-generated module documentation

Because generated checks already run in the developer path, module documentation should be generated from source metadata and treated as drift when it falls behind. Every module's `README.md` is generated from its variables and outputs, engineers do not write those tables by hand, and CI verifies that the checked-in documentation is current.

For Terraform modules, that usually starts with `.terraform-docs.yml` :


```

formatter: markdown table
output:
  file: README.md
  mode: inject
  template: |-
    <!-- BEGIN_TF_DOCS -->
    {{ .Content }}
    <!-- END_TF_DOCS -->
sort:
  enabled: true
  by: required

```

CI then turns that generation rule into an enforceable contract:

```
- run: terraform-docs --output-check markdown table modules/network/hub
```

For Bicep, use `PSDocs.Azure` or parse the `metadata` blocks with a custom script. Either way, the pattern is the same: the source describes the interface, generated documentation exposes it, and CI prevents drift. Once modules are documented this way, releases can also become mechanical rather than ceremonial.

Conventional commits + automated releases

Use Conventional Commits with automated releases whenever modules are versioned and consumed outside the repo, because consumers need a readable contract for what changed without reverse-engineering every diff. The commit message becomes that contract only if you force it into a parseable shape:

```

feat(network/hub): add Azure Firewall premium SKU support
fix(identity): correct role assignment scope for spoke VNets
chore(ci): bump terraform to 1.9.5

BREAKING CHANGE: hub module renamed param `firewall_sku_name` → `firewall_sku`

```

The supporting tools are deliberately boring:

- `commitlint` + Husky to validate locally.
- `release-please` (recommended) or `semantic-release` to automate:
 - Tag + GitHub Release.
 - `CHANGELOG.md` update.
 - Migration notes in release body.

The benefit compounds because every consumer's Renovate bot can post a "v1.5.0 → v1.6.0" PR with the changelog inline, turning the release note into part of the upgrade workflow rather than a separate document someone has to find.

Structured commits make the changelog practically free. The remaining pieces of the developer-experience puzzle are smaller but still worth locking down: editor settings that prevent trivial formatting noise, and templates that shape what contributions look like before they hit the pipeline.

Editor configuration

After release automation has removed ambiguity from versions, editor configuration removes ambiguity from whitespace, line endings, and formatting choices. Commit those settings so tools enforce them consistently instead of reviewers negotiating them in comments.

```
# .editorconfig
root = true

[*]
charset = utf-8
end_of_line = lf
insert_final_newline = true
trim_trailing_whitespace = true
indent_style = space
indent_size = 2

[*.{tf,tfvars}]
indent_size = 2

[*.bicep]
indent_size = 2

[*.md]
trim_trailing_whitespace = false # MD line breaks
```

Combine EditorConfig with Prettier for YAML, Markdown, and JSON, while letting Bicep and Terraform use their own formatters. These files do not make the repo more sophisticated; they make routine diffs quieter, which matters when the same standards need to be stamped into more than one repository.

Repository templates

When teams create more than one landing-zone repo, templates or scaffolding are the only reliable way to keep the developer experience you just designed from drifting on day one. For organisations spinning up many landing-zone repos, a **GitHub template repository** plus `gh repo create --template` gives every new repo the same starting point:

- The standard `Makefile`, `.pre-commit-config.yaml`, `devcontainer`.
- A starter `envs/` skeleton.
- Standard `.github/workflows/*.yaml` calling reusable workflows.
- A pre-filled `CODEOWNERS` you customise.

For more sophisticated scaffolding with prompts, use **Backstage templates** or `copier`, especially when teams need to capture ownership metadata, environment choices, or policy-driven defaults as part of creation rather than as a follow-up task.

Templates provision the repo. The next layer shapes how contributions land in it, so reviewers get the evidence they need before the pipeline becomes the first reviewer.

Issue & PR templates

Contribution templates should capture the evidence reviewers need before the pipeline becomes the first reviewer. The goal is not bureaucracy; it is to make the right thing the easy thing by asking for scope, rationale, affected environments, and required evidence while the author still has the context.

A pull request template can stay short and still change behavior:

```
## What

## Why

## How (high-level)

## Affected environments
- [ ] nonprod
- [ ] prod

## Checklist
- [ ] `terraform plan` reviewed in CI
- [ ] PSRule findings reviewed
- [ ] Module bumps (if any) include CHANGELOG link
- [ ] If breaking change, ADR added under `docs/adr/`
```

Issue templates serve the same purpose for work that has not yet become a pull request:

- `bug.yml` (structured form: env, repro, expected vs actual)
- `feature.yml`
- `change-request.yml` (for production changes that need a paper trail)

Templates make review evidence routine, but they do not mean every experiment belongs in a shared environment. The final developer-experience decision is where local Azure testing is allowed to happen.

Local development against Azure

Local Azure experimentation is useful only when it is isolated from shared team environments and governed by expiry controls. Engineers should be able to test safely, but the repo should make the safe path explicit:

- Each engineer has a personal **sandbox subscription** or resource group (RG) with their name as the owner.
- `make plan ENV=sandbox WORKLOAD=hub` runs against their own sub via `az login`.
- **Aggressive lifecycle:** all sandbox resource groups have a `DeleteAt` tag and a scheduled function/runbook tears down anything past expiry.
- No shared "team-dev" subscription -- they always become the prod everyone forgot was prod.

That sandbox boundary closes the loop that began with onboarding: you give engineers a fast path, pin the tools, enforce the checks, and still prevent local convenience from becoming unmanaged shared state. The anti-patterns that follow are the places where that loop most often breaks.

Anti-patterns

- **✗ README that says "install Terraform 1.x"** without pinning a version. Tomorrow's release breaks today's repo.
- **✗ Linters that produce warnings nobody reads.** Either fail the build or remove the linter.
- **✗ Module READMEs written by hand.** They drift from the variables in one PR.
- **✗ `make apply` that works against prod.** Pipeline-only or you lose audit trail.
- **✗ Devcontainer that nobody uses.** Make it the *only* supported path -- including for CI -- and it stays maintained.

Taken together, those anti-patterns show why developer experience is the invisible multiplier: when your team can move fast and confidently, the quality of every other practice in this book improves. The Makefile runs the tests, the hooks run the linters, the devcontainer keeps the tools consistent, and the PR template makes the right information the default -- all of it compounding quietly. The next chapter turns from how you build and ship the platform to how you keep it observable and manageable once it is running in production. If you want to trace the tooling behind those recommendations, the references below are the primary starting points.

References

- `pre-commit` : <https://pre-commit.com/>
 - `pre-commit-terraform` : <https://github.com/antonbabenko/pre-commit-terraform>
 - `terraform-docs` : <https://terraform-docs.io/>
 - Devcontainers: <https://containers.dev/>
 - `mise` : <https://mise.jdx.dev/>
 - `release-please` : <https://github.com/googleapis/release-please>
 - `commitlint` : <https://commitlint.js.org/>
 - Backstage software templates: <https://backstage.io/docs/features/software-templates/>
-

[← 09 Testing & policy](#) · [Index](#) · [11 Manageability →](#)

11 · Manageability & day-2 operations

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [CODEOWNERS -- your first line of governance](#)
- [Drift detection](#)
- [Blast radius management](#)
- [RBAC for the IaC system itself](#)
- [Lifecycle of a landing zone](#)
- [Runbooks \(in-repo\)](#)
- [Cost & usage management](#)
- [Observability of the IaC pipeline itself](#)
- [Anti-patterns](#)
- [References](#)

Decision: *how do you keep an ALZ healthy after the first deploy -- ownership, drift, blast radius, lifecycle, and runbooks?*

← [10 Code quality](#) · [Index](#) · [12 Naming & tagging](#) →

Manageability is a first-class Azure Landing Zone (ALZ) design concern, not a day-2 afterthought you can bolt on once the first deployment has succeeded. In practice, that means you require team-based CODEOWNERS review, run scheduled drift detection from `main`, keep each apply's blast radius narrow, tightly scope the Infrastructure as Code (IaC) pipeline identity, version runbooks in the repo, and measure the deployment system like a production service. The real test of any ALZ implementation is not whether the first `apply` succeeds -- it is whether the system stays coherent under operational pressure, when teams bypass the pipeline "just this once", subscriptions that were meant to be temporary become permanent, and identities quietly accumulate permissions like barnacles.

How we got here

Modern ALZ operations should be boring by design: ownership is explicit, drift is visible or blocked, and privileged human action is exceptional. That was not where the discipline started. The first generation of "infrastructure as code" was really *infrastructure as code, plus quite a lot of clicking when nobody was looking*. Drift wasn't a concept; it was a fact of life that a `vnet` created by Terraform on Monday would have an extra subnet by Friday because someone needed to *just quickly fix something*.

The tooling and governance model caught up only after that pain became obvious. Tools like **driftctl** (2021) and **Terraformer** brought visibility, and scheduled `terraform plan -detailed-exitcode` runs became a

standard day-2 practice. Microsoft's **Deployment Stacks** with `denySettings` (GA 2024) finally made out-of-band changes *blockable at the Azure Resource Manager (ARM) layer*, not just detectable after the fact. **CODEOWNERS** (GitHub 2017) gave path-based governance that scales beyond "the platform team reviews everything", and **Privileged Identity Management (PIM)** in Microsoft Entra made standing high-privilege access the exception rather than the rule. Day-2 operations in 2026 is no longer a heroic firefighting practice; done right, it is mostly Renovate PRs, drift dashboards, and the occasional runbook, which is why the decision framework starts with the controls that keep those routines predictable.

Key terms

Drift detection -- the practice of periodically running `terraform plan` (or `az stack show`) and alerting when the actual cloud state diverges from what the code declares.

driftctl -- an open-source tool (now archived) that scanned AWS/Azure resources and compared them against Terraform state to find unmanaged or drifted resources.

KQL (Kusto Query Language) -- the query language used by Azure Resource Graph, Log Analytics, and Microsoft Sentinel for querying structured data.

DORA metrics -- four key metrics from the DevOps Research and Assessment programme: deployment frequency, lead time for changes, change failure rate, and time to restore service. Used to benchmark engineering performance.

Subscription vending -- the automated process of creating and configuring a new Azure subscription (with budget, policies, RBAC, and networking) so it's ready for a workload team to use.

EA / MCA -- Enterprise Agreement / Microsoft Customer Agreement -- the two main Azure billing models through which subscriptions are created programmatically.

Zombie subscriptions -- subscriptions that are no longer actively used or maintained but remain billable and potentially insecure because no decommission process was followed.

Bus factor -- the number of team members who would need to be unavailable before the project stalls. A bus factor of 1 means a single person's absence can halt operations.

Decision framework

With that operational history in mind, answer these day-2 questions before the first production landing zone goes live; the safe default is team-owned review, scheduled drift detection, narrow state, least-privilege automation, versioned runbooks, and pipeline health metrics. Those defaults are sequenced deliberately, because the controls later in the chapter assume that the earlier ones have made ownership and change paths explicit.

1. How is CODEOWNERS structured?

- Use path globs to assign ownership to teams, not individuals.
- Require CODEOWNERS review through branch protection on protected paths.
- Keep ownership rules short enough to review quarterly; generate them from a source of truth if the repo grows large.

2. How do you detect drift?

- Run a scheduled `plan` / `what-if` from `main` and alert on diffs.
- For legitimate drift, codify or import it according to policy; for illegitimate drift, revert it and fix the process gap.
- Document which Azure Policy effects are intentionally ignored or codified.

3. How do you keep blast radius small?

- Use many small state files, normally one per workload per environment.
- Run one apply per workload per environment rather than estate-wide applies.
- Add Deployment Stacks `denySettings`, locks, soft delete, and purge protection for critical resources.

4. Who has access to the IaC system itself?

- Scope the CI service principal only to the management group or subscription level it actually needs.
- Give humans read access for diagnosis and PIM-controlled break-glass write access only.
- Alert on owner assignments, PIM elevation, and unusual service principal sign-ins.

5. Where do runbooks live?

- Keep runbooks in an in-repo `/runbooks/` folder, versioned with the code they describe.
- Make them short, copy-pasteable, and verified during game days or incident reviews.

6. How do you measure pipeline health?

- Track lead time, deployment frequency, change failure rate, and time to restore on the IaC repo itself.
- Alert when production applies fail, drift detection fires, or expected pipelines stop running.

The detailed practices behind those answers follow below. They begin with ownership because every other operational control depends on knowing who is accountable for a path, a pipeline, or a subscription.

CODEOWNERS -- your first line of governance

After the decision framework tells you that ownership is the first day-2 control, the practical implementation starts with GitHub `CODEOWNERS`: use path globs that point to teams rather than individuals, and make those reviews mandatory on protected paths. The file enforces *who must approve* a pull request (PR) for any given path, and in a layered repo it is the difference between governance that scales and a review process that depends on whoever happens to be online.

```
# .github/CODEOWNERS
# Everything by default
*                                @org/platform-engineering

# Foundation needs a second pair of eyes from cloud governance
/envs/prod/foundation/          @org/platform-engineering @org/cloud-governance
/policies/                      @org/cloud-governance @org/security

# Per-landing-zone owners
/envs/prod/lz-corp-app01/        @org/team-app01 @org/platform-reviewers
/envs/prod/lz-corp-app02/        @org/team-app02 @org/platform-reviewers

# Pipeline templates: only platform team
/.github/workflows/              @org/platform-engineering
```

The file only becomes a control when you combine it with **branch protection** that *requires* CODEOWNERS review; without that requirement, CODEOWNERS is informational only, which means the governance model looks better in the repository than it behaves in production. That distinction matters more as the repository grows, because informal review habits rarely survive team turnover or workload onboarding.

 **CODEOWNERS complexity scales with repo size.** *The example above assumes a single repo containing foundation, platform, and landing zones -- the monorepo model. In practice, the recommended layered few-repo topology splits these into separate repos, which means each repo's CODEOWNERS file stays short and obvious (a handful of rules instead of hundreds). If you do run a monorepo or a large shared landing-zone repo, expect the CODEOWNERS file to grow with every onboarded team -- review it quarterly, and consider generating it from a source-of-truth YAML to avoid stale entries.*

Because ownership data ages as people leave, teams reorganise, and applications retire, review the file quarterly rather than waiting for a blocked production PR to reveal that the only listed approver has moved on. Ownership governance tells you *who* must approve a deliberate change, but it says nothing about changes that never went through the PR process at all; detecting those requires something more active.

Drift detection

Once ownership covers deliberate change, drift detection covers the changes that happen outside the review path: run it from `main` on a schedule, alert on every unexpected diff, and either codify, import, or revert based on policy. Drift is any change made to Azure resources outside your IaC, and it will happen -- a firefighter clicks in the portal, an automation script bypasses the pipeline, or a temporary exception quietly survives the incident that created it -- so the first duty of the system is to make that divergence visible.

A commonly overlooked drift source: Modify and DINE policies

Not all drift comes from humans. **Modify** and **DeployIfNotExists (DINE)** Azure policies deliberately change or create resources outside your IaC pipeline -- and they're *supposed* to. A DINE policy that adds

diagnostic settings to every new storage account is doing its job, but `terraform plan` will report those settings as drift because the state file never knew about them.

That legitimate policy behaviour creates a tension you need to resolve deliberately. If you do not choose a policy for these diffs, drift reports either become noisy enough to ignore or incomplete enough to mistrust:

- **Suppress the diff** (e.g. `ignore_changes` in Terraform, or omit the property from Bicep) and accept that the policy is the source of truth for that attribute. Clean drift reports, but the IaC no longer describes the full resource.
- **Codify the policy's effect** in your IaC so the plan matches reality. Complete resource description, but duplicates intent already expressed by the policy -- and changes to the policy require matching IaC updates.
- **Accept the noise** and train the team to recognise known policy-induced diffs during triage. Honest, but erodes trust in drift reports over time.

There is no universally agreed best practice here. Many teams use a hybrid: `ignore_changes` for attributes managed *exclusively* by policy (e.g. diagnostic settings added by DINE), and codify attributes where the IaC and policy must agree (e.g. TLS version enforced by Modify). Document which approach you chose and why -- undocumented `ignore_changes` blocks are a maintenance hazard.

Pattern: scheduled `plan` / `what-if`

```
# .github/workflows/drift.yml
on:
  schedule:
    - cron: '0 6 * * 1' # Mondays 06:00 UTC
  workflow_dispatch:

jobs:
  drift:
    strategy:
      matrix:
        env: [nonprod, prod]
        workload: [connectivity, identity, management]
    uses: contoso/alz-pipeline-templates/.github/workflows/tf-drift.yml@v2
    with:
      working-directory: envs/${{ matrix.env }}/${{ matrix.workload }}
```

The scheduled workflow is useful because `tf-drift.yml` runs `terraform plan -detailed-exitcode` and gives each outcome a clear operational meaning. That explicit mapping keeps the response predictable when the job fires on a Monday morning instead of during an interactive troubleshooting session:

- Exit 0 → no changes 
- Exit 2 → drift detected → **post to a Teams/Slack channel + open an issue** with the diff.
- Exit 1 → error → page the on-call.

For Bicep with Deployment Stacks, use `az stack sub show` to compare the declared state with the actual `Microsoft.Resources/deployments`. The `denySettings: denyWriteAndDelete` mode prevents most drift at the source.

When drift is found

When drift is found, triage it in the same order every time. The goal is not merely to restore the declared state, but to understand why the declared path was bypassed:

1. **Reproduce** by running `plan` interactively.
2. Was the change legitimate?
 - **Yes** → codify it: open a PR that adopts the change and merge.
 - **No** → revert by running `apply` to restore desired state.
3. **Always investigate why** -- drift indicates a process gap. Maybe a policy is missing, maybe an engineer doesn't know the rules.

Detecting drift is valuable; limiting how much damage a single bad change can cause is equally important. The structural approaches to keeping your blast radius small are the subject of the next section.

Blast radius management

Once drift is visible, you still need to limit how much a single bad change can break. Keep blast radius small by making state granular, applies narrow, and destructive changes hard to execute accidentally; in other words, design the deployment system so the failure domain of an apply matches the smallest operational unit you can realistically own.

1. State-file granularity

State-file granularity is the first containment boundary, which is why [07 state management](#) recommends one state per environment and workload. When a workload plan only has access to its own state, a bad apply cannot accidentally rewrite the connectivity estate or another team's subscription.

2. Bicep Deployment Stacks `denySettings`

```
az stack sub create \
  --deny-settings-mode denyWriteAndDelete \
  --deny-settings-excluded-actions "Microsoft.KeyVault/vaults/secrets/write" \
  --deny-settings-excluded-principals "<sp-app-runtime-id>"
```

Deployment Stacks add a second boundary because a subsequent rogue `apply` or portal click is blocked at the ARM layer rather than merely discovered later by drift detection. Excluded principals should therefore be rare, explicit identities that are allowed to mutate inside the stack for well-understood runtime reasons.

3. Resource locks

Azure Resource Locks (`CanNotDelete` / `ReadOnly`) are a complementary last line of defence, especially for critical resources such as hub virtual networks, ExpressRoute circuits, central Key Vaults, and Log Analytics workspaces. Apply them via IaC so the locks themselves remain code-managed rather than becoming another form of manual drift.

```
resource "azurerm_management_lock" "hub_vnet" {
  name      = "lock-cannotdelete"
  scope     = azurerm_virtual_network.hub.id
  lock_level = "CanNotDelete"
  notes     = "Critical hub network -- break-glass via PIM"
}
```

The tradeoff is that locks make legitimate IaC operations harder too, because a `terraform apply` that needs to *replace* a locked resource fails. Reserve them for genuinely critical, slow-moving resources where the cost of accidental deletion is higher than the friction they add.

4. Soft delete + purge protection

For data-bearing services such as Key Vault and Storage, enable soft delete and purge protection unconditionally in your modules. Recovery beats prevention when the worst happens, and these controls give you time to respond when a destructive operation gets past the earlier boundaries.

Architectural controls limit the structural blast radius, but those controls are only as strong as the identities that hold the keys to the pipeline, which makes RBAC for the IaC system itself the next question to examine. The same containment logic now moves from resources and state files to people, service principals, and privileged roles.

RBAC for the IaC system itself

The identities that operate your IaC system should have only the permissions they need, with continuous integration (CI) identities scoped by environment and human write access treated as break-glass. Day-2 Role-Based Access Control (RBAC) concerns are different from day-1 provisioning because the question is no longer "can the pipeline create the estate?" but "can any identity now change more of the estate than its job requires?"

Identity	Permissions
Pipeline SPN (per env)	Contributor at the env scope; explicit User Access Admin if it manages role assignments
Engineers (read, all envs)	Reader on all subscriptions -- debugging needs visibility
Engineers (write, sandbox only)	Contributor on personal sandbox subs
Engineers (write, prod break-glass)	PIM-elevated, alerting on every elevation
Platform team leads	Owner via PIM only, hours-bounded

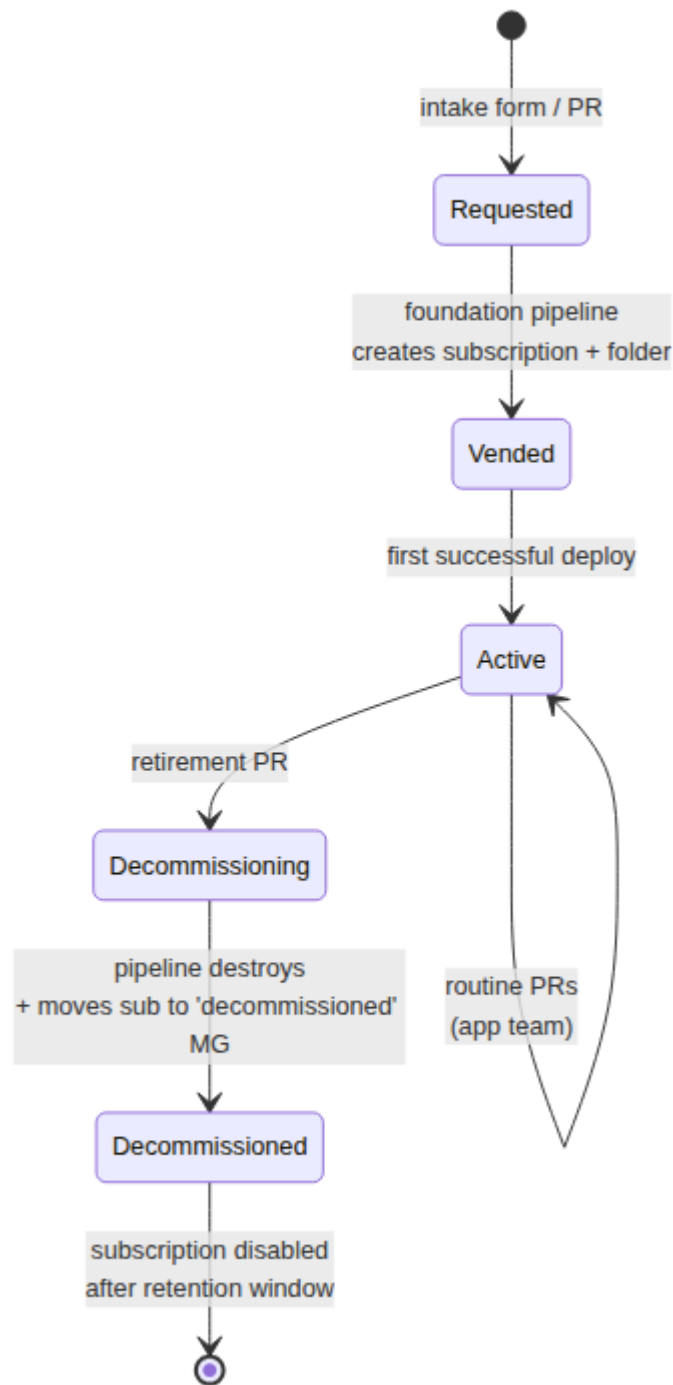
Because role assignments drift just like resources do, use **Azure Resource Graph** queries on a schedule to audit high-privilege grants. The query below is intentionally simple so the weekly diff stays understandable to both platform and security reviewers:

```
AuthorizationResources
| where type =~ "microsoft.authorization/roleassignments"
| where properties.roleDefinitionId endswith "/8e3af657-a8ff-443c-a75c-2fe8c4bcb635"
// Owner
| project subscription = subscriptionId, principalId = properties.principalId, scope =
properties.scope
```

Send the diff against last week to the security team, so permission growth becomes a reviewed change rather than an unpleasant discovery during an incident. Role assignments are a snapshot of *who can do what right now*; the complementary concern is the lifecycle of the resources those identities manage, because landing zones are created, evolve, and should be retired cleanly when they are no longer needed.

Lifecycle of a landing zone

Once the identities are under control, the estate still needs an explicit create-change-retire lifecycle in the repo, including a first-class decommission path. The repository should support each transition below, so a landing zone can move from request to active use to retirement without leaving side channels for teams to improvise under pressure.



Phase	Repo action
Vended	A PR creates a folder under <code>envs/<env>/lz-<name>/</code> from a template
Active	Routine PRs from app team; pipeline deploys
Decommissioned	A PR removes the folder; pipeline destroys; the subscription is moved to a <code>decommissioned</code> MG and disabled

The decommission path must be **first-class** because, without it, your estate accumulates zombie subscriptions forever. A subscription that nobody owns still has cost, data exposure, role assignments, and policy exceptions, so retirement deserves the same engineering attention as onboarding.

Subscription vending

Subscription vending can follow either of two viable patterns, depending on how much automation you already have around intake and billing. The choice affects where subscription creation is reviewed, but it should not change who owns the resulting platform contract:

- **Manual via repo:** A new landing zone PR includes a `subscription.yml` that the foundation pipeline consumes to call the EA / MCA API and create the subscription.
- **Automated via Azure subscription vending API + Service Catalog template:** the landing-zone accelerator / ALZ vending module handles it.

Either way, **the subscription belongs to the foundation/platform repo**, not the workload repo, because creation, policy attachment, budgets, and management-group placement are platform responsibilities. The workload repo only consumes the subscription ID.

A well-defined lifecycle tells you *what* needs to happen at each stage. A runbook tells you *exactly how* -- step by step, copy-pasteable -- so that an operation works the first time it is needed, under pressure, by someone who may not have done it before.

Runbooks (in-repo)

Because the lifecycle only helps if operators can execute it reliably, runbooks should live in the repository and be versioned with the IaC they operate. Every IaC repo benefits from a `/runbooks/` folder with one-pagers for the operations your team actually does:

- `credential-leak.md`
- `state-corruption-recovery.md`
- `force-unlock-state.md`
- `import-existing-resource.md`
- `decommission-landing-zone.md`
- `roll-back-failed-deploy.md`
- `rotate-encryption-key.md`

Each runbook should have the same small set of sections so the person using it does not have to learn a new format during an incident. Consistency matters more than length, because a short runbook that always answers the same questions is easier to trust under pressure:

1. **When to use this** (1 sentence)
2. **Prerequisites** (access, tools)
3. **Steps**, copy-pasteable
4. **Verification** of success
5. **Communication** template (who to notify, where)

These files get used at 2 a.m., so make them findable, terse, and tested rather than comprehensive in the abstract. Runbooks address sudden, high-pressure operational events; cost management addresses the

opposite problem, which is the slow, silent accumulation of spend that nobody notices until the monthly finance review lands in someone's inbox.

Cost & usage management

After you have covered the incident path, make cost and usage signals visible from the repo so landing-zone owners see spend before finance escalation does. Cost is a manageability concern because uncontrolled growth is operational drift expressed in money, so surface the signals where owners already review infrastructure change. That placement turns cost from a monthly surprise into part of the normal PR conversation:

- Tag every resource with `CostCenter` and `Owner` -- enforced via the policy layer.
- Run [Azure Cost Management exports](#) to a storage account, ingest into Log Analytics, and surface per-landing-zone cost in a dashboard.
- For module changes that significantly affect cost (e.g. SKU bump), the PR template includes a "cost impact" field; reviewer asks for an estimate using the [Azure Pricing Calculator](#).
- Set **subscription budgets** in IaC; alert at 50/75/90 % to the landing-zone owner.

Knowing what your infrastructure costs is one signal. Knowing how the system that *deploys* that infrastructure is behaving is another, and the pipeline deserves the same observability treatment as any other production service.

Observability of the IaC pipeline itself

Measure the IaC pipeline itself with DORA-style health metrics because a broken deployment system is a production incident, not merely an inconvenience for the platform team. Treat the pipeline as a service by collecting the same kinds of signals you would expect from any other production dependency:

- Send pipeline metrics to Application Insights or a Log Analytics workspace: run duration per env, success/failure rate, time-to-production for a typical PR.
- Track lead time and change failure rate (DORA metrics) per repo.
- Alert on:
 - Apply failures in prod (page on-call).
 - Drift detection alerts (queue for triage).
 - Unusual SPN sign-ins (security).
 - Pipelines that haven't run in > N days (dead workloads?).

When these signals are missing, day-2 operations degrade into guesswork, and the anti-patterns below are usually the evidence you see first.

Anti-patterns

Most day-2 failures come from invisible ownership, invisible drift, or unbounded permissions; treat the following as design smells. They are not stylistic preferences, but early warnings that the operating model will fail under pressure.

- **✗ No drift detection.** You are flying blind. A misconfiguration can exist for months before someone trips over it.
 - **✗ CODEOWNERS that lists @org/everyone .** Reviews become rubber stamps.
 - **✗ Resource locks that the pipeline SPN bypasses without anyone noticing.** A lock that doesn't lock the deploy identity is theatre.
 - **✗ No decommission process.** Subscriptions accumulate forever and you'll find a "test-sub-2019" still costing \$4 k/month.
 - **✗ Runbooks in a wiki nobody can find at 2 a.m.** Keep them with the code.
 - **✗ Engineers fixing prod by editing in the portal "just this once".** Either it goes through code, or your code is no longer authoritative.
-

Manageability is where the gap between "it worked on day one" and "it still works reliably at year three" lives. The practices in this chapter -- ownership via CODEOWNERS, scheduled drift detection, Deployment Stacks to block drift at the ARM layer, PIM-bounded access, structured landing-zone lifecycles, in-repo runbooks, and pipeline observability -- do not all need to be in place before you ship. Start with drift detection and CODEOWNERS; the rest grows naturally as the estate matures. Chapter 12 turns to the seemingly mundane but genuinely load-bearing question of what you call things -- and what metadata travels with them.

References

Use these references to validate the platform controls, repository governance, and operational metrics described above.

- GitHub, *About code owners*: <https://docs.github.com/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners>
 - Microsoft, *Manage drift in deployment stacks*: <https://learn.microsoft.com/azure/azure-resource-manager/bicep/deployment-stacks#protect-managed-resources-against-deletion>
 - Microsoft, *Azure resource locks*: <https://learn.microsoft.com/azure/azure-resource-manager/management/lock-resources>
 - Microsoft, *Subscription vending*: <https://learn.microsoft.com/azure/architecture/landing-zones/subscription-vending>
 - Microsoft, *PIM*: <https://learn.microsoft.com/entra/id-governance/privileged-identity-management/pim-configure>
 - DORA metrics: <https://dora.dev/>
-

[← 10 Code quality](#) · [Index](#) · [12 Naming & tagging](#) [→](#)

12 · Naming, tagging & metadata conventions

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [Why this gets a whole chapter](#)
- [Naming convention](#)
- [Tagging convention](#)
- [Management group / subscription naming](#)
- [Region strategy](#)
- [Anti-patterns](#)
- [References](#)

***Decision:** what do you call things, and what metadata travels with them? The boring stuff that breaks everything if you skip it.*

[← 11 Manageability](#) · [Index](#) · [13 Documentation](#) →

Adopt one enforced convention: lowercase, dash-separated resource names built from Microsoft Cloud Adoption Framework (CAF) abbreviations, plus a mandatory tag set that is applied by modules and enforced by Azure Policy. Treat names as the human-readable index you can scan during an incident, and treat tags as the machine-readable source of truth for cost, ownership, security, lifecycle, and automation. Define the pattern, abbreviations, allowed tag values, management-group names, subscription names, and region codes once in shared code and policy, because anything that only lives on a standards slide will drift. The historical reason for that discipline is simple: every estate has already tried the informal version, and almost every estate has paid for it later.

How we got here

In the early days of every Azure tenant there's an artisanal naming convention written by the first cloud architect on a whiteboard -- and a second one written by their replacement six months later. By 2018 the average enterprise had three: the documented one, the one used for new resources, and the one used by the legacy migration team. Microsoft's **CAF naming guidance** (2019) and the canonical **resource abbreviation list** finally gave the industry a shared vocabulary, and reusable **naming modules** (the `Azure/naming` Terraform module, the `nianton/azure-naming` Bicep module) made the convention executable rather than aspirational. That history matters because naming is one of the few Azure Landing Zone (ALZ) decisions where a casual exception becomes visible in every portal view, cost export, and incident review for years.

Key terms

CAF (Cloud Adoption Framework) -- Microsoft's comprehensive guidance for cloud strategy, governance, and implementation. Its naming and tagging recommendations are the *de facto* standard for Azure estates.

Resource abbreviations -- a CAF-published list of short prefixes for each Azure resource type (e.g. `rg-` for resource groups, `st` for storage accounts, `kv-` for key vaults).

Modify effect -- an Azure Policy effect that automatically adds or corrects properties on resources (e.g. appending a tag inherited from the resource group) without denying the deployment.

Canonical values -- a fixed, enumerated set of allowed values for a tag or naming segment (e.g. `env` $\in$ `{ prod , nonprod , sandbox }`), enforced by policy to prevent free-text inconsistency.

Data classification -- a tagging practice that labels resources by sensitivity level (e.g. `Public` , `Internal` , `Confidential` , `Restricted`) to drive security automation and access control.

Tagging followed a similar arc: a brief flirtation with "free-text whatever the engineer felt like" gave way to **Azure Policy `modify` effects** that auto-append resource group tags, and to mandatory tag sets enforced by `deny` . The lesson is blunt because the failure mode is so common: **what isn't enforced by code or policy doesn't exist**. Training slides don't enforce; continuous integration (CI), modules, and policy do. That enforcement lesson is the bridge from history to design: before you choose syntax, you need to answer the questions that determine what code and policy must protect.

Decision framework

Because the history points to enforcement rather than persuasion, answer these questions before teams start deploying. The output should be both a standards page and executable code: a naming module, `_shared/abbreviations.tf` or `abbreviations.bicep` , canonical tag values, and Azure Policy assignments.

- 1. What is the canonical naming pattern?** Choose the required components -- workload, environment, region, resource type, and instance -- and their order. This guide's examples use `<resource-abbr>-<workload>-<env>-<region-abbr>-<instance>` with `-` separators because Azure operators scan resource type first; if your organisation prefers workload-first (`<workload>-<env>-<region>-<resourceType>-<instance>`), make that the one allowed order and encode any resource-specific exceptions.
- 2. Which abbreviations are allowed for resource types?** Start with Microsoft's official CAF abbreviation list, then lock any additions or shortened forms in `_shared/abbreviations.tf` or `abbreviations.bicep` so modules and reviews use the same vocabulary.
- 3. What casing is allowed?** Default to lowercase everywhere Azure permits it. Reserve PascalCase only for values that humans read as display names or formal labels, such as management-group display names or tag values that intentionally follow a business taxonomy.
- 4. Which tags are mandatory, and how are they enforced?** Use a small core set -- workload, environment, owner, cost centre, criticality, data classification, and business unit -- then enforce it with

module defaults plus Azure Policy `Modify / DeployIfNotExists` for inheritance and `Audit / Deny` for missing or invalid values.

5. **How are management groups and subscriptions named?** Mirror the management-group path in subscription names so governance, cost, and incident queries can infer the hierarchy without a portal lookup.
6. **How is region encoded?** Pick full Azure location names for policy (`swedencentral`) and a documented short code for names (`swc`), then use that mapping consistently in every module and exception request.
7. **How are exceptions handled?** Require an expiring exemption tag, a reason, and approval from the relevant CODEOWNER; open-ended exceptions become the new convention.

With those answers fixed, the rest of the chapter shows the convention in practice, beginning with why metadata deserves this much attention in the first place.

Why this gets a whole chapter

Those framework answers matter because naming and tagging are the **load-bearing infrastructure of every other process**: cost reporting, access control, automation, decommissioning, disaster recovery, and incident response all rely on consistent metadata. If the convention is weak, you pay a permanent tax every time someone has to open the portal, join exports manually, or guess who owns a resource.

The counterpoint is just as important: an over-engineered convention that nobody can remember will be violated immediately, which means your target is not maximal cleverness but a **rigorous but humane** pattern that humans can read and automation can enforce. That balance explains the debate below and leads directly into the naming pattern that follows it.

The debate -- do you even need a naming convention?

A provocative but defensible position: name every resource with a random identifier (or let Terraform/Bicep generate one), and rely entirely on **tags** and **Azure Resource Graph (ARG)** queries for discoverability, cost reporting, and ownership. After all, the resource's metadata -- subscription, resource group, type, tags -- already tells you everything the name encodes, and unlike names, tags are mutable when your conventions change. Location in the name? The resource has a `location` property. Environment? That's a tag. Why duplicate it in a string with a 24-character limit?

The case for meaningful names: Humans still read names -- in portal URLs, `az` CLI output, cost exports, alert emails, and (critically) in 2 a.m. incident logs where you need to recognise the resource before you have time to query its tags. A name like `kv-platform-prod-swc-01` is instantly parsed by a human; `kv-7f3a2c91` requires a lookup. Community experience on Reddit and in DevOps forums consistently reports that GUID-named estates become unnavigable at scale, especially when tags are inconsistently applied -- which, despite policy enforcement, happens more often than anyone admits.

The case for random/generated names: Naming conventions create a false sense of order. They embed assumptions (region, environment) that become wrong when resources move or conventions evolve. They hit length limits on storage accounts and key vaults, producing ugly truncations. And they tempt engineers into parsing names programmatically instead of using proper metadata -- a brittle pattern that breaks the moment the convention changes. Some teams (particularly those with mature automation and strict tag enforcement) report success with generated names + rich tags.

Where the industry stands (2026): The overwhelming majority of Azure guidance (CAF, Well-Architected Framework (WAF), ALZ accelerators) recommends structured naming conventions. But the strongest argument for conventions is a human factors one, not a technical one: people debug faster when names mean something. If your estate is fully automated and humans rarely read raw resource names, the argument weakens. In practice, most teams adopt a convention and enforce tags -- treating names as a human-readable index and tags as the machine-readable source of truth.

With that stance established, start by making the resource name predictable.

Naming convention

With the human-factor argument settled, make resource names predictable by generating lowercase, dash-separated strings from CAF resource abbreviations, workload, environment, region abbreviation, and instance number. The important constraint is not the exact order -- although this guide uses resource type first -- but that humans never type the finished name into a parameter file and every exception is encoded in the shared naming logic.

Anatomy

```
<resource-abbr>-<workload>-<env>-<region-abbr>-<instance>
```

Examples:

```
vnet-hub-prod-swc-01
rg-platform-connectivity-prod-swc
sa-tflogs-prod-swc-001      (no dashes, lowercase, ≤ 24 chars)
kv-platform-prod-swc-01     (≤ 24 chars, globally unique)
nsg-app01-web-prod-swc-01
```

Rules:

- Lowercase only. Azure is mostly case-insensitive, but tools, Kusto Query Language (KQL), and bash scripts aren't.
- Use `-` separators except where Azure forbids them, such as storage accounts, Azure Container Registry (ACR), and Key Vault.
- **Region abbreviations** (3 letters): pick once and document. Examples: `swc` (Sweden Central), `weu` (West Europe), `nwe` (North Europe), `wus2` (West US 2). Don't invent new ones; consult your standards page.
- **Instance suffix** (`-01` , `-02`) -- even for resources you expect to be singletons. The day you need a second one, your name doesn't fight you.
- **Avoid embedding subscription / tenant info in names.** Tags and the scope hierarchy already convey it.

Canonical resource abbreviations

Use the [Microsoft CAF resource abbreviations list](#) as a starting point. Don't invent your own abbreviations; they aren't worth the bus-factor cost. Lock the approved subset and any shortened forms in `_shared/abbreviations.tf` or `abbreviations.bicep` so Terraform/Bicep code, reviews, and documentation all resolve the same resource type to the same string. That small indirection is what keeps a resource type from becoming `kv` in one module, `keyvault` in another, and `vault` in a review checklist.

Length & character constraints

Some Azure resources have awkward limits, so **plan for the worst** rather than the most readable case:

Resource	Max length	Allowed
Storage account	24	a-z, 0-9
Key Vault	24	a-z, A-Z, 0-9, <code>-</code>
ACR	50	a-z, A-Z, 0-9
Function App	60	a-z, A-Z, 0-9, <code>-</code>
VM (Windows)	15	a-z, A-Z, 0-9, <code>-</code>
Resource Group	90	broad

When names don't fit your convention, define **shortened forms** explicitly in your naming module rather than truncating ad hoc. That way the exception is repeatable, reviewable, and visible to every caller.

Code it, don't print it

The convention only survives if you code it into a **naming module** in `alz-modules`, because a standards page cannot prevent a typo in a parameter file:

```
module "naming" {
  source = "Azure/naming/azurerms"
  version = "0.4.2"
  suffix = ["${var.workload}", var.env, local.region_abbr]
}

resource "azurerms_storage_account" "logs" {
  name = module.naming.storage_account.name_unique
  ...
}
```

For Bicep, the [Azure naming Bicep module](#) is one option, although this community module has not been actively maintained since 2023; Bicep user-defined functions or the [Azure Naming Tool](#) may be safer alternatives in a long-lived estate. Whichever implementation you choose, **never let humans type the name into a parameter file**, because typos become permanent and some resource types are painful to rename.

A consistent naming convention tells you what a resource *is*. A consistent tagging convention tells you who it belongs to, what it costs, and how automation should treat it. The two are complementary, and both require the same discipline.

Tagging convention

The naming module gives you a readable resource index, but it cannot carry the operational facts that change over time; for that, enforce a small mandatory tag set at both resource group and resource level. Use module defaults for consistency, Azure Policy `Modify` or `DeployIfNotExists` where inheritance or remediation is safe, and `Audit` or `Deny` where missing or invalid values should block drift.

Mandatory tags

Every resource and every resource group must carry the core operational set below. Start with workload, environment, owner, cost centre, criticality, data classification, and business unit, then add traceability tags only when they drive automation. Enforce the inherited values through Azure Policy `Modify` or `DeployIfNotExists`, and use `Audit` or `Deny` for the rest so missing or invalid metadata is visible before it becomes operational debt.

Tag key	Example	Notes
CostCenter	CC-12345	Charging code; numeric ideally.
Owner	team-app01@contoso.com	Group address, not a person.
Environment	prod , nonprod , sandbox	Drives a lot of automation.
Workload	app01 , connectivity-hub	The "what".
BusinessUnit	corp , online	The "for whom".
DataClassification	public , internal , confidential , restricted	Drives policy decisions.
Criticality	1 , 2 , 3 , 4	Tier for SLA / DR.
ManagedBy	iac , manual	Detect drift; expect iac everywhere.
Repo	alz-platform	The Git repo of authority.
DeploymentId	<commit-sha> or <run-id>	Trace a resource to a pipeline run.

Optional but useful

Tag key	Example	Use
DeleteAt	2026-12-31	For sandbox / time-bounded resources; cleanup automation.
Project	migration-2026	Programme tracking.
Compliance	pci , gdpr	Regulatory scope.

Inheritance & enforcement

Because the mandatory tags above are only useful when they appear everywhere, do not assume tag inheritance from resource group to resource; Azure does **not** do that automatically, despite many tutorials implying so. Instead, combine policy and module behavior so the platform can fill inherited values while each module remains explicit about what it writes.

- **Policy modify effect** to auto-append resource group tags to resources at creation, with `DeployIfNotExists` where remediation needs a managed identity or a child deployment. Use sparingly -- too many `modify` policies make `terraform plan` noisy.
- **Module defaults** -- every module accepts a `tags` map and merges with its own additions. Do **not** rely on `default_tags` in the AzureRM provider for required tags; module-level merging is more explicit.


```

locals {
  base_tags = {
    Environment      = var.env
    Workload          = var.workload
    Owner             = var.owner
    CostCenter        = var.cost_center
    DataClassification = var.data_classification
    BusinessUnit      = var.business_unit
    Criticality       = var.criticality
    ManagedBy        = "iac"
    Repo              = "alz-platform"
    DeploymentId      = var.deployment_id
  }

  tags = merge(local.base_tags, var.additional_tags)
}

```

Tag reporting

Enforcement still needs feedback, so run a weekly Azure Resource Graph query to find non-compliant resources:

```

Resources
| where tags !has "CostCenter" or tags !has "Owner" or tags !has "Environment"
| project name, type, subscriptionId, resourceGroup, tags

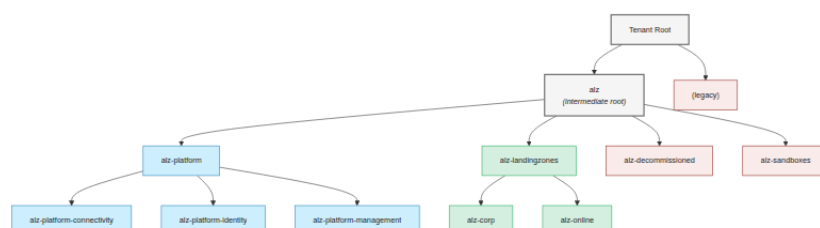
```

Send the count and worst offenders to a dashboard, and use quarterly budget or ownership reviews to make the backlog visible rather than letting tag debt become background noise. Once that feedback loop is in place, resource-level naming and tagging address the workload layer; the same principles apply one level up, where management groups and subscriptions form the structural skeleton of the tenant -- and where an inconsistent naming scheme will haunt your governance queries for years.

Management group / subscription naming

At the structural layer, management-group and subscription names should mirror the ALZ hierarchy so ownership and governance scope are obvious from the string before you open the portal. This is the same principle as resource naming, but the blast radius is larger because these names appear in cost reports, policy assignments, and incident routing.

Because these objects are part of the foundation, bake the tree into foundation code rather than letting it emerge from manual portal operations:



Subscription names mirror the management group path, which keeps cost, governance, and incident queries aligned with the hierarchy:

```
sub-platform-connectivity-prod
sub-corp-app01-prod
sub-corp-app01-nonprod
sub-sandbox-jdoe
```

Document the tree in the foundation repo's `docs/management-groups.md`, and treat changes to it like changes to the management group hierarchy itself rather than cosmetic cleanup. That documented hierarchy establishes *what* things are and *whose* they are; region strategy is a natural extension of the same discipline, because capping the allowed set of deployment locations is one of the simpler, highest-leverage policy decisions you will make.

Region strategy

Once the hierarchy is readable, constrain where resources can land by approving a small region set, using full Azure location names in policy, and using one documented short code in resource names. This keeps policy enforcement precise while preserving concise names.

Pick a small number of **primary regions** -- typically two per geography for disaster recovery (DR) pairing -- and don't deploy outside them without an exception process.

- **Primary:** `swedencentral`
- **Paired:** `northeurope` (DR target for Sweden)
- **Approved exception:** `westeurope` (legacy)

Then bake the allowed list into a policy so the convention is enforced at deployment time:

```
// policies/restrict-regions.bicep
param allowedLocations array = [
  'swedencentral'
  'northeurope'
  'westeurope'
]
```

Use **deny** for the policy assignment in landing zones and **audit** in sandboxes, where experimentation needs more room. Once names, tags, scopes, and regions are encoded this way, the remaining failures are usually recognizable anti-patterns rather than hard design questions.

Anti-patterns

Every anti-pattern below creates metadata that humans or automation will eventually stop trusting, which is why the fix is always enforcement rather than another reminder in a standards deck. Read the list as a set of

failure modes to block in code review, Azure Policy, or module design before they become the convention by accident.

- **✗ Inventing your own resource abbreviations.** Use Microsoft's CAF list; the bus factor is too high otherwise.
- **✗ Free-text tags** (Owner: "John (he sits next to Sara)"). Tags are data; treat them as such.
- **✗ Required tags enforced only by training docs.** People forget. Enforce with policy.
- **✗ Names that include the subscription ID** ("just in case"). The scope is already there.
- **✗ Environment tag values that drift** (Prod , prod , Production). Pick canonical values and Deny anything else with policy.
- **✗ Tagging at the resource level only**, ignoring RG. RG tags are the inheritance source for modify policies.
- **✗ Naming module added "later"**. Names are the hardest thing to refactor -- costs days of pipeline work per resource type.

Naming and tagging may feel like administrative overhead until the day someone asks "which team owns this resource, and what does it cost them?" and the answer cannot be retrieved in under thirty seconds. Enforce the convention from day one, because it is genuinely cheaper to build right than to rename a thousand resources later. The references below provide the authoritative source material for the abbreviations, naming rules, and tooling used here, and the next chapter turns to the documentation that explains *why* these decisions were made so the engineer who joins two years from now understands the system rather than simply inheriting it.

References

- Microsoft, *CAF -- Naming and tagging*: <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/naming-and-tagging>
 - Microsoft, *Resource abbreviations*: <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-abbreviations>
 - Microsoft, *Resource naming rules*: <https://learn.microsoft.com/azure/azure-resource-manager/management/resource-name-rules>
 - Azure/naming Terraform module: <https://registry.terraform.io/modules/Azure/naming/azurerm/latest>
 - nianton/azure-naming Bicep module (*inactive since 2023*): <https://github.com/nianton/azure-naming>
 - Azure Naming Tool (actively maintained alternative): <https://github.com/mspnp/AzureNamingTool>
-

[← 11 Manageability](#) · [Index](#) · [13 Documentation](#) →

13 · Documentation & onboarding

In this chapter:

- [How we got here](#)
- [Decision framework](#)
- [The four kinds of documentation](#)
- [Top-level README.md](#)
- [Architecture Decision Records \(ADRs\)](#)
- [Diagrams as code](#)
- [Auto-generated reference](#)
- [CONTRIBUTING.md](#)
- [CHANGELOG.md](#)
- [Onboarding checklist](#)
- [Architecture overview document](#)
- ["Why" lives in commit messages and ADRs, not in code comments](#)
- [Anti-patterns](#)
- [References](#)

Decision: *what gets written, where, and who keeps it current? Documentation that lives outside the repo decays; documentation in the repo doesn't write itself.*

[← 12 Naming & tagging](#) · [Index](#) · [14 Anti-patterns →](#)

Keep Azure Landing Zone (ALZ) Infrastructure as Code (IaC) documentation small, repo-native, and purpose-specific: tutorials teach, how-tos guide, reference is generated, and Architecture Decision Records (ADRs) preserve why. Code tells you *what* the system does; documentation tells you *why* it does it, *how* to change it safely, and *where* to start when everything is on fire. Most IaC repos skimp on all three -- and then wonder why onboarding takes three months and runbooks get executed incorrectly under pressure. This chapter sets out a disciplined, minimal approach to documentation that lives in the repo, decays slowly, and actually gets used, beginning with how repository documentation escaped from the shared-drive era in the first place.

How we got here

Pre-2010, infrastructure documentation lived in Word documents on a shared drive nobody had write access to. The wiki era (Confluence, SharePoint, MediaWiki) was a step forward in *editability* but a step backward in *trustworthiness* -- there was no diff, no pull request (PR), no link to the code, and the docs decayed silently. Two ideas fixed this. First, **Architecture Decision Records** -- a one-page format proposed by Michael Nygard in 2011 -- gave teams a way to capture *why* a decision was made, immutably and in version

control. Second, the **Diátaxis framework** (Daniele Procida, 2017-2021) explained *why* most documentation is bad: it conflates tutorials, how-tos, reference, and explanation into one unreadable mess. Tooling caught up: **Mermaid** landed in GitHub markdown in 2022, killing the binary-diagram era; **D2** and **Excalidraw** gave teams text-based architecture diagrams; and **terraform-docs** / **PSDocs** automated the reference layer. That history leaves you with a practical design problem rather than a nostalgia lesson: the modern IaC repo's `docs/` folder has to be a small, disciplined library, not a dumping ground, and the decision framework below turns that principle into a set of choices.

Key terms

ADR (Architecture Decision Record) -- a short, immutable document capturing a single architectural decision: the context, the decision itself, and the consequences. Stored in `docs/adr/` and numbered sequentially.

Diátaxis -- a documentation framework by Daniele Procida that classifies all technical content into exactly four types (tutorials, how-to guides, reference, explanation), arguing that mixing types in one document is the most common documentation failure.

Mermaid -- a text-based diagramming language rendered natively by GitHub in markdown code blocks (````mermaid`). Supports flowcharts, sequence diagrams, state diagrams, and more.

D2 -- a modern, text-based diagram scripting language designed for software architecture diagrams, with auto-layout and theming.

Excalidraw -- a virtual whiteboard tool that produces hand-drawn-style diagrams, storable as JSON and embeddable in repos.

PSDocs -- a PowerShell-based documentation generator that produces markdown from Azure resource templates, analogous to `terraform-docs` for Bicep/ARM.

Decision framework

Because the historical failure mode was unmanaged sprawl, decide the documentation system by answering these questions in order. The goal is not "more docs"; it is the smallest set of repo-native documents that explains how to work safely, what the estate contains, and why the hard decisions were made.

1. Which of the four documentation kinds do you owe?

- Tutorials, how-tos, reference, and explanation are different jobs; write only the ones your audience actually needs.

2. Are architectural decisions captured as ADRs?

- Yes for any decision worth defending in a year; no for trivia that code review can settle.

3. How are diagrams produced?

- Prefer Mermaid, D2, Excalidraw JSON, or Draw.io with the XML committed; never accept binary-only diagrams as the source of truth.

4. Is module reference documentation auto-generated?

- Use `terraform-docs`, `PSDocs.Azure`, or Bicep generation on a pre-commit hook or CI check so reference stays aligned with code.

5. Is there an onboarding checklist a new joiner can complete in their first week?

- If not, the team is relying on tribal knowledge and ad hoc pairing.

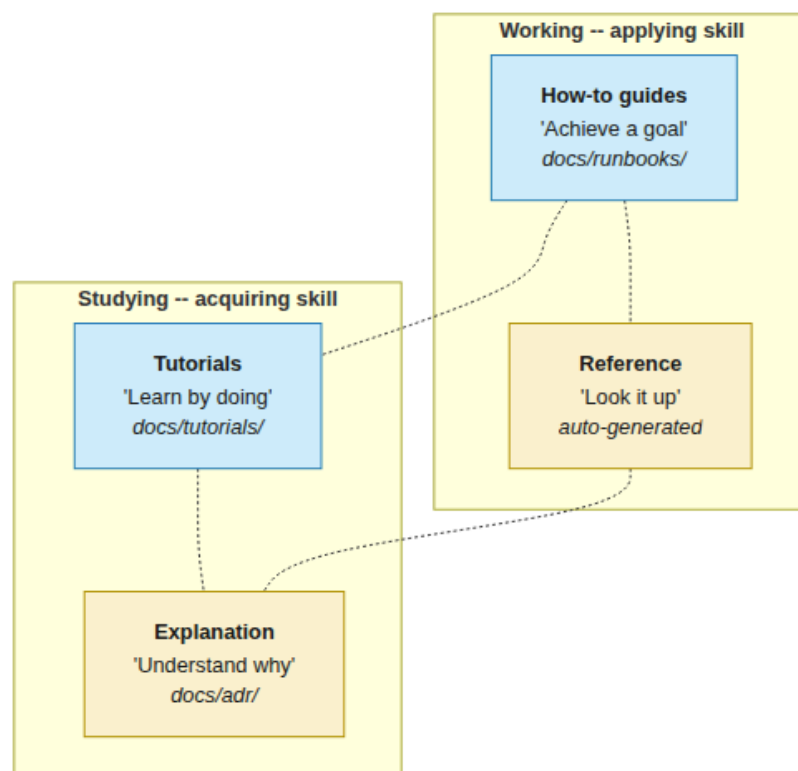
6. How are "why" decisions preserved?

- Put durable rationale in commit messages and ADRs, not in code comments that drift away from the decision context.

The detailed conventions below apply those answers to each document type. Start with the four-kind taxonomy, because it determines whether the next file you write should teach, guide, explain, or simply report facts.

The four kinds of documentation

The first decision is to give every document exactly one Diátaxis job, because mixed-purpose docs become unreadable, unowned, and eventually untrusted. Inspired by the [Diátaxis framework](#), every piece of documentation should be exactly one of these four kinds:



Left column = practical steps · Right column = theoretical understanding

Type	Purpose	Where in the repo
Tutorials	Hand-hold a newcomer to first success	docs/tutorials/
How-to guides	Achieve a specific outcome	docs/runbooks/ , docs/howto/
Reference	Look up exact details	Auto-generated from code
Explanation	Understand <i>why</i>	docs/adr/ , this guide

The taxonomy matters because mixing types in one document is the most common documentation failure: a tutorial that tries to be a reference becomes too long to follow, while a reference that tries to explain every decision stops being searchable. With that distinction established, apply it first to the document every reader encounters before anything else.

Top-level `README.md`

Once you separate tutorials, how-tos, reference, and explanation, the top-level README can stop trying to contain all of them and become what it should have been all along: a one-screen map to the repo. It is the first thing every reader sees, so aim for **one screen**:

```
# alz-platform

Infrastructure-as-Code for Contoso's Azure Landing Zone -- platform layer.

## Quick start

git clone ... && cd alz-platform
make bootstrap
make plan ENV=nonprod WORKLOAD=connectivity

## Where things live

- envs/           Per-environment compositions
- modules/        Local pattern modules (tier 2)
- policies/       Custom Azure Policy definitions
- docs/           Architecture, ADRs, runbooks
- .github/        Pipelines, CODEOWNERS

## Documentation

- [Architecture overview](docs/architecture.md)
- [Architecture Decision Records](docs/adr/)
- [Runbooks](docs/runbooks/)
- [Contributing](CONTRIBUTING.md)

## Owners

Platform engineering · #alz-platform · platform@contoso.com
```

Anything more goes behind a link, because long READMEs go unread precisely when a new contributor most needs a map. A good README is therefore a table of contents for the rest of the documentation, and

the most important item it should point to is the ADR log -- the record of *why* the system is built the way it is, without which every architectural choice becomes oral tradition that disappears when the original architect leaves.

Architecture Decision Records (ADRs)

Because the README can only point to the reasoning, any architectural decision worth defending in a year belongs in an ADR. Every non-trivial decision -- and this guide is full of them -- should be captured as a short, dated, immutable record.

docs/adr/0001-monorepo-or-multirepo.md :

```
# 0001 · Repository topology -- layered few-repo

- Status: Accepted
- Date: 2026-04-12
- Deciders: @jane.platform, @joe.architect
- Tags: topology, governance

## Context
We need to host IaC for a tenant containing ~30 landing zones across 4 BUs.
The platform team owns connectivity/identity/management, app teams own
their landing zones.

## Decision
Adopt three repositories: `alz-foundation`, `alz-platform`, and one
`alz-landingzones-<bu>` per BU. Modules live in a separate `alz-modules`
repo published to ACR.

## Consequences
+ Each repo has a coherent CODEOWNERS and approval policy.
+ Platform releases independent of workload releases.
- Cross-cutting changes require coordination via the modules repo.
- Need a pipeline-templates repo to keep CI consistent.

## Alternatives considered
- Monorepo: rejected for blast-radius and CODEOWNERS complexity at our scale.
- Pure multi-repo: rejected -- discoverability cost too high without a
  developer portal.

## References
- docs/01-repository-topology.md
```

ADRs are **never edited**; if the decision changes, write a new ADR that *supersedes* the old one and link them, so the reader can see both the original constraint and the later correction. Lightweight tools can help you keep that discipline without turning ADRs into a parallel publishing system:

- `adr-tools` -- `adr new "Topology"` .
- `log4brains` -- renders ADRs as a static site.

ADRs capture decisions in prose; some decisions -- particularly around network topology and identity flows -- are better expressed visually. The challenge is keeping those visuals version-controlled and diffable, rather

than letting them become the same undiffable artefacts as the Word documents they replaced, which is why diagram source deserves the same treatment as code.

Diagrams as code

The same rule that makes ADRs trustworthy also applies to diagrams: the source belongs in version-controlled text, and rendered images are generated outputs. Binary `.drawio` / Visio files in Git are an anti-pattern because they are undiffable, unmergeable, and immediately stale, so use diagrams as code instead:

Tool	Best for
Mermaid	Sequence, flowcharts, class -- renders natively in GitHub markdown.
Excalidraw	Sketch-style architecture; export <code>.excalidraw</code> (JSON) + PNG.
D2	Architecture diagrams with great auto-layout.
PlantUML	Sequence/component diagrams; long history.
Draw.io with <code>.drawio.svg</code>	If you must -- the SVG variant is text and renders as PNG on GitHub.

Keep both source (`*.mmd`, `*.d2`, `*.excalidraw`) and rendered PNG/SVG so the README looks right without local tooling, and let continuous integration (CI) re-render the diagram on every PR. In the common case, the diagram can then sit directly in markdown:

```
```mermaid
flowchart LR
 Hub[(Hub VNet)] --- Spoke1[App01]
 Hub --- Spoke2[App02]
 Hub --- ER[ExpressRoute]
```
```

Once diagrams and ADRs are versioned, the remaining documentation risk is the material that changes every time code changes. That is where generated reference documentation becomes non-negotiable.

Auto-generated reference

Reference docs cannot be hand-written reliably, because the exact details they describe are the same details your modules, policies, and pipelines change most often. Generate them and fail CI when they drift:

- **Module READMEs:** `terraform-docs` or `PSDocs.Azure` -- see [10 code quality](#).
- **Pipeline reference:** parse workflow YAML; produce a list of inputs, required secrets, and called workflows.
- **Policy reference:** generate a markdown table from your policy definitions JSON.
- **Naming convention:** generate a table of all resource abbreviations from your naming module.

CI runs the generators and fails the build if the generated output differs from the committed copy; this is **the only way** reference docs stay correct. Once that reference layer can be trusted, the next question is how you help a contributor make a safe first change.

CONTRIBUTING.md

If generated reference tells you what exists, `CONTRIBUTING.md` tells you how to change it without relying on tribal knowledge. It should make the first PR possible by explaining the working agreement in one short file:

- How to set up the dev environment (link to README quick start).
- PR conventions (Conventional Commits, PR template).
- Required checks and how to run them locally.
- Review process (who, when, service-level agreement (SLA)).
- Where to ask questions (channel, office hours).

Keep it under 200 lines, because a contribution guide that needs its own onboarding path has already failed. Once contributors can make changes, the people who consume those changes need an equally trustworthy release history.

CHANGELOG.md

Because consumers need a trustworthy release history, generate changelogs from commits instead of maintaining them by hand. A repo that uses Conventional Commits and tags releases consistently can let `release-please` produce the changelog, and in that model the GitHub Releases page *is* your changelog. For module repos this is non-negotiable, because consumers depend on it; for the platform team itself, the same discipline prepares the next engineer to understand what changed before they touch production.

Onboarding checklist

Release history helps you understand what changed, but a new platform engineer also needs an executable path through the repo in their first week. Onboarding should therefore be a checklist issue, auto-created from a template, that the new joiner can complete while leaving improvements behind for the next person:

Onboarding · @newjoiner

- [] Access requested: Azure (Reader on all subs), GitHub team membership
- [] Devcontainer or local toolchain working: ``make bootstrap``
- [] First successful sandbox plan: ``make plan ENV=sandbox``
- [] Read: README, this guide (docs/), top 5 ADRs
- [] Pair on first PR (assigned: @buddy)
- [] Pager rotation onboarding (assigned: @oncall-lead)
- [] Shadow a deploy to prod (assigned: @release-eng)

The checklist itself is a documented artifact, so improvements should come from new joiners filing PRs against the template rather than from private notes passed between buddies. After the checklist gets someone through the first week, the architecture overview gives them the map they will keep returning to.

Architecture overview document

The onboarding checklist points you toward the system, but `docs/architecture.md` should be the single canonical map of the estate, not a slide deck that drifts separately from code. Keep it focused on the views you actually need when making or reviewing a change:

- Diagram of management group hierarchy.
- Diagram of network topology.
- Diagram of identity model (Entra tenants, managed identities (MIs), federated creds).
- List of subscriptions with owner and purpose.
- List of regions and their role.

Update the overview on every significant change: CI can verify that the diagram source files exist, while you verify that they are current during a quarterly review on the team's calendar. The overview tells you what the estate looks like now; the final documentation habit is to preserve why it looks that way.

"Why" lives in commit messages and ADRs, not in code comments

Because the architecture overview shows the current shape rather than the reasoning behind it, long-lived rationale belongs in commit messages and ADRs; code comments should explain only what the code cannot. Code comments rot, while commit messages and ADRs are immutable, so prefer:

```
# ❌ Comment that explains a workaround
resource "azurerole_assignment" "ra" {
  # Wait 30 seconds because role propagation lags - issue #1234
  ...
}
```

Instead of relying on that comment to survive every future refactor, write the **commit message** that adds the workaround explicitly:

```
fix(identity): wait 30s after role assignment

AzureRM #20871 -- RBAC propagation can take up to 30 seconds. Without
this delay, the next module fails with AuthorizationFailed.

Removable when the upstream issue is closed.
```

Comments belong in code only when they explain *what* the code does in a way the code cannot; *why* belongs in commits and ADRs, where the rationale can be reviewed and recovered later. That discipline is the

capstone of a documentation culture, and the anti-patterns below catalogue the ways that culture most commonly collapses in practice.

Anti-patterns

Once the documentation system stops being repo-native, diffable, generated where appropriate, and focused on intent rather than implementation, it starts to fail in familiar ways. The specific symptoms vary by team, but they usually reduce to one of the patterns below.

- **✗ Wiki-only documentation** (Confluence, SharePoint, Loop). Decays fast, can't be PR-reviewed, can't be searched alongside code.
 - **✗ The "TODO: document this" comment.** Open an issue or write the doc -- TODOs go unread.
 - **✗ Architecture diagrams in proprietary binary formats.** No diff, no merge, no render in PR.
 - **✗ CHANGELOG hand-written and forever stale.** Generate it.
 - **✗ One giant `docs/all.md`.** Split. Use the index.
 - **✗ Documenting *implementation* instead of *design*.** Code is the authoritative implementation; docs explain *why* and *how to use*.
-

Documentation is not a deliverable that ships alongside the code -- it is the medium through which the code communicates its intent to the engineers who will work with it for the next five years. The practices in this chapter -- Diátaxis-informed structure, immutable ADRs, generated reference docs, version-controlled runbooks, and an onboarding checklist that improves with every new joiner -- require no heroic effort. They require the same discipline applied to everything else in this guide: a clear convention, enforced consistently, with the repo as the single source of truth. That discipline is the thread running through all fourteen chapters; the next and final chapter draws it together in a consolidated checklist of everything that can go wrong -- and how to check whether it already has.

References

- Diátaxis: <https://diataxis.fr/>
 - MADR -- Markdown ADR template: <https://adr.github.io/madr/>
 - `adr-tools` : <https://github.com/npryce/adr-tools>
 - GitHub markdown -- Mermaid: <https://github.blog/developer-skills/github/include-diagrams-markdown-files-mermaid/>
 - D2: <https://d2lang.com/>
 - `terraform-docs` : <https://terraform-docs.io/>
-

14 · Anti-patterns & common pitfalls

In this chapter:

- [Decision framework](#)
- [Repository & code structure](#)
- [Tooling & versioning](#)
- [Authentication & secrets](#)
- [Pipelines & CI/CD](#)
- [Modules & registries](#)
- [State management](#)
- [Testing & policy](#)
- [Code quality / DX](#)
- [Manageability / day-2](#)
- [Naming & tagging](#)
- [Documentation](#)
- [Operational](#)

A consolidated checklist of mistakes seen in real ALZ implementations. If you find yourself doing any of these, stop and reconsider.

[← 13 Documentation](#) · [Index](#) · [References →](#)

Use this chapter as a 30-minute health check for any existing Azure Landing Zone (ALZ) repository. The previous chapters gave you the recommended patterns one topic at a time; this final chapter pulls those warnings into a single checklist so you can test an implementation quickly and honestly. Each section maps back to the part of the guide that explains the healthier pattern, and each anti-pattern is phrased as something you might actually find in a real repository rather than as an abstract rule.

That checklist format is deliberate. Anti-patterns are easier to recognise when they are concrete, so this chapter stays more list-heavy than the rest of the book while still giving you enough context to understand why each item is dangerous. No real ALZ implementation scores perfectly on first review; the point is to identify the gaps, prioritise them by risk, and address them through the same pull request process as everything else. Read each item as a question: *Is this us?*

Decision framework

Start with the scorecard before you inspect the individual categories, because the score gives you a useful sense of whether you are dealing with isolated hygiene work or a deeper platform design problem. If you cannot answer Yes to all of these, you have at least one anti-pattern somewhere below; the value of the

exercise is not the number itself, but the conversation it forces about which risks you are accepting deliberately and which ones simply accumulated unnoticed. Score yourself out of 20:

- ☐ State files are split per (env × workload).
- ☐ No long-lived Azure secrets in any CI system.
- ☐ OIDC federation in use, with restrictive `subject` claims.
- ☐ Modules pinned by exact version, never by branch.
- ☐ Reusable pipeline templates shared across repos.
- ☐ Pre-commit hooks installed and run in CI.
- ☐ Plan posted as a PR comment for every PR.
- ☐ Policy gates run at PR time *and* at runtime.
- ☐ CODEOWNERS exist and are reviewed quarterly.
- ☐ Branch protection requires CODEOWNERS + signed commits.
- ☐ Drift detection runs weekly per state file.
- ☐ Required tags enforced via Azure Policy.
- ☐ Module READMEs auto-generated and CI-verified.
- ☐ Devcontainer or `mise / asdf` pinning all toolchain versions.
- ☐ At least 5 ADRs in `docs/adr/`.
- ☐ At least 3 runbooks in `docs/runbooks/`.
- ☐ Decommission process documented and exercised.
- ☐ Naming convention enforced via a naming module.
- ☐ Storage account hosting state is private endpoint + CMK only.
- ☐ State recovery has been tested in the last 6 months.

| Score | Verdict |
|-------|--|
| 18-20 | World class -- write a blog post. |
| 14-17 | Solid -- pick the next two and improve. |
| 10-13 | Functional -- significant risk in 1-2 areas. |
| < 10 | Stop and re-plan before scaling further. |

Working through this list is most useful not as a one-time audit but as a recurring exercise -- quarterly for a mature platform, monthly when the estate is growing quickly. When you find an anti-pattern you are currently living with, resist the temptation to note it privately and move on. Open an issue, assign an owner, and track its remediation alongside everything else the team ships. The self-assessment above gives you a structured framework for that conversation -- a score honest enough to be useful and specific enough to drive action -- and the first place that score usually breaks down is the shape of the repository itself.

Repository & code structure

Repository boundaries and state layout are not cosmetic choices; they decide who can review a change, how much infrastructure a single mistake can touch, and how easy it is for a new engineer to understand where intent lives. The anti-patterns in this section all start from the same assumption, namely that you can tidy structure later once the platform is bigger. In practice, the opposite is true: structure is easiest to fix while the estate is still small, and it becomes expensive precisely when you most need it to be predictable.

- **✗ One mega-repo with one giant Terraform state.** A single repository can be a valid starting point, but a single state file for the whole estate turns every plan into a high-stakes operation because the blast radius is the entire environment. Split state by environment and workload so a failure in one area cannot lock, drift, or destroy unrelated infrastructure. See [01 repository topology](#) and [07 state management](#).
- **✗ One repo per resource group.** This looks like strong isolation until you realise the repository boundary no longer matches any meaningful ownership boundary. You end up with more pipeline glue than infrastructure, cross-resource changes require choreography across many repositories, and reviewers lose the ability to reason about a workload as a whole.
- **✗ Mixing application source code with IaC.** Application code and platform IaC usually move at different cadences, have different reviewers, and carry different security expectations. Keeping them together makes branch protection, CODEOWNERS, and deployment approvals either too strict for developers or too loose for infrastructure.
- **✗ Copy-pasted "modules" folder across many repos.** Copying a module folder feels faster than publishing a shared module, but every copy immediately becomes its own fork with its own bugs, defaults, and undocumented patches. Promote common code to a shared module repository and consume it through a registry so fixes and versioning are explicit.
- **✗ Branch-per-environment** (`dev` , `staging` , `main`) with cherry-picking between them. Environment branches make Git history carry deployment state, which means every hotfix depends on humans remembering which commits moved where. Model environments as folders, workspaces, stacks, or pipeline inputs instead, and let Git branches represent code review rather than runtime topology.
- **✗ Tests, modules, and envs all jumbled together.** A consistent layout is a load-bearing readability feature because reviewers need to know whether they are looking at reusable code, environment composition, or validation logic before they can judge the change. When those concerns sit in one undifferentiated tree, every review starts with archaeology.

Once the repository tells you where responsibility lives, the next question is whether two engineers running the same code with the same inputs will get the same result. That takes you from structure into tooling and versioning, where the same discipline has to apply to every binary, provider, action, and module reference.

Tooling & versioning

A reviewed plan is only meaningful if you can reproduce the tools, providers, actions, and module versions that produced it. The mistakes in this section often hide behind harmless language such as "latest" or "main", but mutable inputs break the link between review and deployment. Pinning is not bureaucracy; it is how you make a future plan comparable to the one someone approved yesterday.

- **✗ Large ARM JSON templates without tooling assistance.** Raw Azure Resource Manager (ARM) JSON is three to five times more verbose than Bicep or Terraform equivalents, which makes pull request reviews harder, merge conflicts more frequent, and resource references error-prone. The VS Code ARM extension and ARM TTK mitigate some of this, but at scale a higher-level language such as Bicep, Terraform, or Pulumi, with modules, loops, and type-checked references, pays for itself quickly. See [02 IaC tooling](#).
- **✗ Floating versions** (`source = "...?ref=main"`, `version = "latest"`). Floating versions make yesterday's reviewed code behave differently tomorrow even when the repository did not change. Pin exact versions or immutable references so upgrades become deliberate pull requests with visible diffs.
- **✗ GitHub Actions referenced by mutable tags** (`@v3`). A tag can move, and when it does, your pipeline changes outside your review process. Pin actions by SHA, use automation to raise update PRs, and treat the workflow runtime as part of the supply chain.
- **✗ Two IaC engines managing the same resource.** When Terraform, Bicep, Pulumi, or portal automation all believe they own the same object, drift becomes a negotiation between tools rather than a controlled change. Assign one authoritative owner per resource and make any handoff explicit.
- **✗ Pulumi adopted by an ops-background team without a software engineering culture.** Pulumi can be excellent when the team is comfortable with application structure, testing, dependency management, and code review discipline. Without that culture, the freedom of a general-purpose language turns infrastructure into spaghetti faster than a declarative tool would.
- **✗ Provider versions unlocked.** Provider minor releases can change defaults, resource schemas, and diff behaviour in ways that rewrite half your plan unexpectedly. Lock provider versions and upgrade them intentionally, with plan output reviewed as a real platform change.

Pinning removes one source of surprise, but a deterministic toolchain still cannot protect you if the identity running it is too powerful or too easy to steal. That is why versioning naturally leads into authentication and secrets, where the same reviewed workflow can become dangerous if its credentials are broad or persistent.

Authentication & secrets

Convenience credentials become estate-wide blast radius when pipeline identity, human access, and secrets are not separated. In an ALZ repository, authentication is part of the architecture: it decides which branch, workflow, environment, or person is allowed to change which scope. The safest pattern is to make short-lived, narrowly scoped, auditable credentials the default path, and to treat any long-lived secret as technical debt with a due date.

- **✗ Long-lived service principal client secret in CI.** A stored client secret turns every workflow runner, repository administrator, and secret export path into a potential route to Azure. Use OpenID Connect (OIDC) federation so the pipeline receives a short-lived token only when the expected repository, branch, and workflow context match.
- **✗ One SPN with Owner @ tenant root for "convenience".** A tenant-root Owner service principal can change almost anything, so a compromised workflow becomes a tenant-wide incident. Scope identities to the smallest management group, subscription, or resource group they need, and separate plan, apply, and break-glass permissions where possible.

- **✗ Federated credential subject `repo:org/repo:*`**. Broad subject claims defeat the point of federation because any branch, tag, or workflow context in the repository can potentially mint a token. Bind the subject to the specific branch, environment, or reusable workflow that should deploy, and review those claims with the same care as role assignments.
- **✗ Same identity used by humans and pipelines**. Shared identities make audit trails ambiguous, which means you cannot tell whether a change came from a reviewed workflow or a person at a terminal. Give pipelines their own identities, require humans to use named accounts, and make break-glass activity visible.
- **✗ Secrets stored in `terraform.tfvars` "just for now"**. Terraform variable files have a habit of being committed, copied into artifacts, or reflected into state, and state files can leak sensitive values even when the variable file is later cleaned up. Store secrets in Key Vault or another managed secret store, and pass references rather than values wherever the platform allows it. See [07 state management](#).
- **✗ Adding `--allow-secret` to bypass scanners**. Secret-scanner bypasses are rarely temporary because the pressure that created them usually returns. If the scanner is wrong, fix the rule or document the exception narrowly; if it is right, rotate the credential and remove the secret instead of teaching the pipeline to ignore it.

After identity is narrowed and secrets are out of the repository, the next risk is the path those identities use to make changes. Pipelines should turn the controls above into the default route, not into optional ceremony, because a safe credential still needs a safe delivery process around it.

Pipelines & CI/CD

The continuous integration and continuous delivery (CI/CD) path is where repository structure, pinned tooling, and scoped identity either come together as a safe delivery path or fall apart into manual exceptions. A good ALZ pipeline preserves the reviewed plan, applies only after the right gates have passed, reuses hardened workflow templates, and keeps watching for drift after deployment. The anti-patterns below weaken that chain at different points, but they all have the same effect: they make it harder to prove what changed, who approved it, and whether production still matches code.

- **✗ apply directly without an explicit plan artifact**. If the apply job recalculates the plan without preserving the reviewed artifact, the change applied to Azure might not be the change reviewers saw. Store the plan, bind approval to that artifact, and make the apply consume it rather than silently planning again.
- **✗ `continue-on-error: true` to "make the build green" while investigating**. A temporary bypass in CI becomes permanent because the visible signal turns green and the pressure to fix the underlying failure disappears. If a control is noisy, tune it; if it is broken, fail loudly until it is repaired.
- **✗ `pull_request_target` with checkout of PR code from forks**. `pull_request_target` runs with elevated permissions from the base repository, so checking out untrusted fork code inside that context creates a privilege-escalation path. Use safer pull request events for untrusted code, or separate metadata processing from code execution.
- **✗ `cancel-in-progress: true` on the deploy concurrency group**. Cancelling a plan job is usually fine, but cancelling an in-flight apply can leave locks, partial deployments, or inconsistent external state.

Protect deployment concurrency so applies serialize without being interrupted midway.

- **✗ One pipeline that deploys all environments serially.** A single serial pipeline makes every environment wait for every other one and encourages broad permissions because the workflow must touch everything. Use environment-specific jobs, approvals, and gates so non-production remains fast while production stays controlled.
- **✗ Per-repo bespoke workflows.** Custom workflows feel locally efficient until small differences in flags, permissions, and gates create divergent security postures across repositories. Put the common logic in reusable workflows or shared templates, and let repositories provide parameters rather than copy-pasted YAML.
- **✗ No drift detection.** A pipeline that only runs on pull requests assumes Azure never changes outside Git, which is not a safe assumption in a real estate. Schedule drift detection per state file or deployment stack, route findings to an owner, and treat untriaged drift as an operational incident.

Pipelines can enforce safe movement through environments, but they still depend on the building blocks they deploy. Those building blocks are the modules and registries that either concentrate enterprise standards or spread duplication across the estate, so the next set of mistakes moves from workflow mechanics into reusable design.

Modules & registries

Modules are supposed to convert repeated design decisions into reusable, versioned platform primitives. They fail when duplication, branch pins, and over-flexible wrappers turn that leverage into hidden coupling. The healthiest module strategy is opinionated enough to enforce enterprise defaults, small enough to remain understandable, and versioned enough that consumers can upgrade deliberately.

- **✗ Inlined modules duplicated between landing zones.** "We'll DRY it later" rarely survives contact with delivery pressure because every duplicate gains its own local assumptions. Extract shared logic before the third copy, publish it, and make consumers pin a version so fixes have a route back to every workload.
- **✗ A "kitchen sink" module** with 80 boolean toggles. A module with dozens of feature flags usually represents several different products hiding behind one interface. Split it into smaller modules with clear responsibilities, and compose them at the landing-zone layer where the architecture is visible.
- **✗ Calling AVM resource modules directly from landing-zone code.** Azure Verified Modules (AVM) are useful building blocks, but direct consumption leaves no place to enforce your naming, tagging, diagnostics, policy, and security defaults consistently. Wrap AVM where you need enterprise opinion, and make the wrapper the supported path for landing-zone teams.
- **✗ Wrapping AVM with a module that adds nothing.** A wrapper that simply passes every input through creates another API surface without adding governance value. If you are not constraining defaults, simplifying inputs, or enforcing standards, call AVM directly and avoid owning a pointless abstraction.
- **✗ Releasing a major version without a `MIGRATION.md`.** A breaking module release without migration guidance pushes every consumer into reverse engineering. Document what changed, why it changed, the

safest upgrade path, and any state moves or import steps before you ask teams to adopt the new major version.

- **✗ Pinning to a branch name (`?ref=main`).** Branch pins make module consumption depend on whatever happened to be on that branch at plan time. Publish immutable versions or tags, then let automated dependency updates raise visible PRs when consumers should move.

A registry gives you controlled distribution, but the applied infrastructure still depends on state that records what already exists. That makes state management the next place where small shortcuts can turn into large outages, because the module version tells you what should happen while state tells the tool what already did happen.

State management

State is a protected production dependency, not a local cache or an implementation detail. It contains the map between code and deployed resources, often includes sensitive outputs, and determines what the next apply believes it is allowed to change. Treat it as isolated, remote, locked, recoverable, and never hand-edited, because state failure is one of the few IaC problems that can turn a small mistake into a platform-wide recovery exercise.

- **✗ State backend in the same subscription as the resources it manages.** A bad apply, accidental deletion, or subscription-level lockout can damage the very storage account you need to recover. Keep state in a separate, tightly controlled subscription with private access, strong backup, and permissions that are independent from the workloads being managed.
- **✗ State on engineer laptops.** Local state means the truth about infrastructure lives on a device that can be lost, overwritten, or unavailable when someone else needs to deploy. Use remote state even for development so locking, backup, and audit behaviour match the rest of the estate.
- **✗ `terraform_remote_state` cross-references that leak producer secrets to consumers.** Remote state exposes outputs from one stack to another, and those outputs often include values the consumer does not need to see. Publish only the required contract through a safer channel such as App Configuration, Key Vault references, or a deliberately shaped output module.
- **✗ Editing state files by hand.** Manual state edits bypass the tool's safety checks and can create a mismatch that only appears during the next destructive plan. Use supported commands such as `import`, `state mv`, or `deployment-stack` operations, and peer-review the recovery procedure before touching production state.
- **✗ Running `force-unlock` reflexively when a lock appears.** A lock is a signal that another operation might still be changing infrastructure. Investigate the running job, confirm it is dead, and record why you are unlocking before you remove the protection that prevents concurrent writes.
- **✗ Bicep prod deployments without Deployment Stacks.** Without Deployment Stacks, Bicep has weaker managed-resource tracking and drift protection for production scopes. Use stacks where they fit so Azure understands the intended resource set and can help you detect or control unmanaged changes.

Once state is protected, you still need confidence that proposed changes are safe before they reach it and that runtime controls catch anything code review misses. Testing and policy provide those two layers, but only when they are placed where teams cannot easily bypass them or learn to ignore them.

Testing & policy

Controls that run too late, too slowly, or only in one layer train teams to work around them. The point of testing and policy in an ALZ repository is not to create a perfect gate at one moment; it is to create a feedback ladder, with fast checks in the pull request, stronger validation before deployment, and Azure Policy enforcing the non-negotiables at runtime. The anti-patterns below either collapse that ladder into one slow gate or leave a gap large enough for portal changes to walk through.

- **✗ All policy lives at PR time.** Pull request checks only evaluate code that passes through the repository. Anyone with portal access, another deployment tool, or a legacy script can bypass them unless Azure Policy or equivalent runtime control enforces the same constraints in the platform.
- **✗ All policy lives in Azure Policy.** Azure Policy is essential, but if developers only discover violations after a long plan or failed deployment, they will see governance as a late surprise. Move checks that can be evaluated statically into pre-commit, CI, PSRule, Checkov, or equivalent tools so feedback arrives while the change is still cheap.
- **✗ `soft_fail: true` on Checkov to "fix later".** A soft-failing scanner teaches the organisation that the signal is optional. Either fail the build on rules you care about, narrow the ruleset to reduce noise, or remove the check until you are ready to enforce it.
- **✗ Integration tests against a shared sub with hard-coded names.** Shared subscriptions and fixed names make parallel pull requests collide, which turns valid changes into flaky failures. Create isolated test scopes, generate unique names, and clean them up automatically so tests can run concurrently.
- **✗ 45-minute test suites.** Slow tests are avoided, skipped, or moved out of the developer path, which means they stop protecting the work they were meant to protect. Split fast PR checks from deeper scheduled or pre-release tests, and parallelise anything that must remain in the critical path.
- **✗ Custom rules in Rego when a built-in PSRule rule exists.** Custom policy code has maintenance cost, documentation cost, and interpretation cost. Prefer built-in rules when they express the control you need, and reserve custom rules for genuinely local requirements.

Good controls should feel like part of the development workflow rather than a separate compliance ritual. That makes developer experience the next practical concern, because the easiest safe path is the one teams will actually use when delivery pressure rises.

Code quality / DX

Developer experience guardrails should be pinned, automated, and hard to ignore. In infrastructure repositories, developer experience is not a luxury layer on top of governance; it is how you make the governed path faster than the unsafe path. The common failure mode is to document a good practice without making it executable, which leaves every engineer to rebuild the toolchain by memory.

- **✗ README that says "install Terraform 1.x"** without pinning. A vague version range guarantees different engineers and runners will eventually plan with different binaries. Pin exact versions through `.terraform-version`, `mise`, `asdf`, a devcontainer, or the pipeline image, and make the README point to that source of truth.

- **✗ Linters that produce warnings nobody reads.** Warning-only linters create noise without changing behaviour because the build still passes and the pull request still merges. Decide which rules matter enough to fail, tune the rest, and remove checks that nobody is prepared to act on.
- **✗ Module READMEs written by hand.** Hand-written module inputs and outputs drift as soon as the interface changes under delivery pressure. Generate README content from the module source, fail CI when generated docs are stale, and reserve hand-written prose for intent and examples.
- **✗ `make apply` that works against prod from a laptop.** A local command that can change production bypasses the audit trail, approval gates, and identity controls you built into the pipeline. Keep local commands limited to validation, formatting, planning in safe scopes, or ephemeral development environments.
- **✗ Devcontainer that nobody uses.** A devcontainer is only valuable if it is the normal way to get the right tools, not an optional artefact that drifts behind the README. Make it easy, current, and used in CI where possible, or choose another toolchain-pinning mechanism that the team will actually adopt.

A smooth developer path gets changes into the platform safely, but the platform also needs owners after the merge. Day-2 manageability is where IaC stops being a deployment technique and becomes an operating model, with named humans responsible for the resources code creates.

Manageability / day-2

Day-2 ownership fails when reviews, runbooks, locks, drift, and decommissioning have no accountable owner. The first apply is only the beginning of an ALZ lifecycle: resources need change review, exceptions need expiry, subscriptions need retirement, and production incidents need responders who can find the right runbook at 2 a.m. The anti-patterns here are dangerous because they often look like administration rather than architecture until something goes wrong.

- **✗ CODEOWNERS = `@org/everyone`**. A review requirement that names everyone effectively names no one, because responsibility diffuses across the whole organisation. Map paths to the smallest accountable team that can make an informed decision, and review those mappings as teams and repositories change.
- **✗ Resource locks that the pipeline SPN bypasses without anyone noticing.** Locks become theatre when the automation identity can remove or ignore them without visibility. If locks are part of your control model, test how the pipeline behaves against them, document the break-glass path, and alert on lock removal.
- **✗ No decommission process.** Subscriptions, role assignments, private endpoints, and diagnostic settings accumulate when retirement is not a defined workflow. Document how a workload leaves the estate, who approves it, how data is retained, and how state is cleaned up.
- **✗ Runbooks in a wiki nobody can find at 2 a.m.** Incident documentation that lives outside the repository is hard to review, hard to version, and often hard to locate under pressure. Keep runbooks with the code, link them from alerts, and test them during drills.
- **✗ Engineers fixing prod by editing in the portal "just this once".** A portal fix may solve the immediate incident, but it also makes code stop being authoritative unless the change is reconciled

immediately. Capture emergency changes as follow-up PRs, or the next apply may undo them without understanding why they existed.

- **✗ Drift alerts that nobody triages.** An alert without an owner or service-level agreement becomes background noise. Route drift findings to a team, define how quickly they must be classified, and close the loop by either reconciling code or removing unauthorized changes.

Operations also depends on the metadata you attach to resources, because responders and cost owners cannot manage what they cannot identify. That brings the checklist to naming and tagging, where small inconsistencies become large reporting and automation problems.

Naming & tagging

Names and tags are governed data, not free text or conventions that live only in training. They drive ownership, cost allocation, policy targeting, automation, and incident response, which means inconsistency becomes an operational defect rather than a cosmetic annoyance. The safest pattern is to encode naming and tagging rules in modules and policy so the estate stays queryable as it grows.

- **✗ Inventing your own resource abbreviations.** Local abbreviations feel harmless until they collide with Microsoft Cloud Adoption Framework (CAF) guidance, Azure constraints, or another team's convention. Start from Microsoft's CAF list, document any deliberate exceptions, and keep the naming module aligned with that source.
- **✗ Free-text tags** (Owner: "John (he sits next to Sara)"). Tags are data that automation and reporting need to parse. Use stable identifiers such as team names, cost centres, service IDs, or distribution lists, and keep human commentary in documentation or issue trackers.
- **✗ Environment values that drift** (Prod , prod , Production). Multiple spellings of the same environment break queries, policy filters, and cost reports. Define canonical values, enforce them with policy or module validation, and migrate old values rather than teaching every report to handle every variant.
- **✗ Required tags enforced only by training docs.** Training explains the rule, but it does not stop a deployment that omits the tag. Enforce required tags in modules and Azure Policy so missing metadata fails close to the point of creation.
- **✗ Naming module added "later".** Names are among the hardest things to refactor because they often appear in DNS, diagnostics, access policies, state, and downstream integrations. Build the naming module early, even if the first version is small, so the estate grows around one contract.

Consistent metadata makes the estate easier to operate, but the reasoning behind the estate still needs to be captured somewhere durable. Documentation is where those design decisions either remain reviewable or decay into folklore, so the next anti-patterns focus on where knowledge lives and how it changes.

Documentation

Documentation that cannot be reviewed, diffed, generated, or tied to design decisions decays immediately. In an IaC repository, docs are not a separate publishing exercise; they are part of the same change system as the

code, which means pull requests should update decisions, diagrams, examples, and runbooks when the platform changes. The anti-patterns below all move knowledge out of that reviewable path.

- **✗ Wiki-only documentation** (Confluence, SharePoint). A wiki can be useful for broad collaboration, but if it is the only source of platform documentation, design knowledge sits outside code review and drifts from the implementation. Keep authoritative ALZ docs with the repository, and link outward only for supplementary material.
- **✗ Architecture diagrams in proprietary binary formats.** Binary diagrams cannot be meaningfully diffed, merged, or reviewed in a pull request. Prefer text-based diagrams, generated diagrams, or formats with a reviewable source so architecture changes travel with the code they describe.
- **✗ TODO comments instead of issues.** TODO comments hide future work in files that nobody scans during planning. Convert them into issues with owners, priorities, and links back to the code so the work can be triaged instead of rediscovered by accident.
- **✗ One giant `docs/all.md`.** A single documentation dump becomes hard to navigate and impossible to review coherently. Split docs by decision area, runbook, or module so readers can find the context they need and reviewers can assess one topic at a time.
- **✗ Documenting implementation rather than design.** Code already shows what is deployed; documentation should explain why the design exists, which tradeoffs were accepted, and what would cause you to revisit the decision. If the prose merely restates resource arguments, it will age worse than the code.

By this point the checklist has covered the repository, the delivery path, the deployed state, and the knowledge around it. The final group asks whether the operating model is ready for the moments when the platform is under pressure, because the best repository design still fails if nobody can respond, measure flow, or retire what is no longer needed.

Operational

Operational readiness requires named responders, measured flow, retirement paths, and practiced failure drills. These are not afterthoughts to add once the ALZ is stable; they are the mechanisms that tell you whether the platform is healthy, whether delivery is improving, and whether the team can recover when a high-impact failure occurs. The last anti-patterns therefore focus on the human and procedural side of running the estate.

- **✗ Pager configured but no on-call rota.** An alerting tool without a named responder is just delayed email. Define the rota, escalation path, and ownership model before you rely on paging as an operational control.
- **✗ No SLO on time-to-production for a typical PR.** Without a service-level objective for delivery flow, you cannot tell whether gates are protecting the platform or quietly making teams bypass it. Measure how long a normal change takes from pull request to production, then improve the bottlenecks deliberately.
- **✗ No mechanism to retire old subscriptions.** Old subscriptions keep consuming cost, policy assignments, role assignments, and operational attention long after the workload has gone away. Give

retirement the same workflow discipline as creation: approval, dependency checks, data retention, state cleanup, and final deletion.

- **✗ No fire drills** for state recovery, credential leak, full-estate outage. The first time you test recovery should not be in production during an incident. Practice the scenarios that would hurt most, record what failed, and turn the lessons into issues, runbook updates, and pipeline improvements.

A strong score at the top of this chapter does not mean you are finished; it means the platform has habits that let it keep improving. Re-run the checklist after major platform changes, after incidents, and before scaling to new business units, because anti-patterns usually return quietly through exceptions that were supposed to be temporary.

[← 13 Documentation](#) · [Index](#) · [References](#) [→](#)

References & further reading

In this chapter:

- [Microsoft -- Cloud Adoption Framework](#)
- [Microsoft -- Bicep & Deployment Stacks](#)
- [Microsoft -- Identity & Security](#)
- [Microsoft -- Governance & Policy](#)
- [Azure Verified Modules](#)
- [ALZ accelerators](#)
- [Terraform / OpenTofu](#)
- [GitHub](#)
- [Azure DevOps](#)
- [Policy / scanning tools](#)
- [Developer experience](#)
- [Documentation](#)
- [Methodology / metrics](#)
- [Community video resources](#)

[← 14 Anti-patterns](#) · [Index](#)

A consolidated list of the external references cited across this guide.

Microsoft -- Cloud Adoption Framework

- CAF Landing Zones overview -- <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/>
- CAF Landing Zone implementation options -- <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/landing-zone/implementation-options>
- CAF Naming and tagging -- <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/naming-and-tagging>
- CAF Resource abbreviations -- <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-abbreviations>
- CAF Environments -- <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/considerations/environments>
- Subscription vending -- <https://learn.microsoft.com/azure/architecture/landing-zones/subscription-vending>

Microsoft -- Bicep & Deployment Stacks

- Bicep documentation -- <https://learn.microsoft.com/azure/azure-resource-manager/bicep/>
- Deployment stacks -- <https://learn.microsoft.com/azure/azure-resource-manager/bicep/deployment-stacks>
- Private module registry -- <https://learn.microsoft.com/azure/azure-resource-manager/bicep/private-module-registry>
- What-if -- <https://learn.microsoft.com/azure/azure-resource-manager/templates/deploy-what-if>

Microsoft -- Identity & Security

- Workload identity federation -- <https://learn.microsoft.com/entra/workload-id/workload-identity-federation>
- Federated identity credentials on managed identities -- <https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/how-manage-federated-identity-credentials>
- Privileged Identity Management -- <https://learn.microsoft.com/entra/id-governance/privileged-identity-management/pim-configure>
- Defender for DevOps -- <https://learn.microsoft.com/azure/defender-for-cloud/defender-for-devops-introduction>
- Storage security baseline -- <https://learn.microsoft.com/security/benchmark/azure/baselines/storage-security-baseline>

Microsoft -- Governance & Policy

- Azure Policy as code -- <https://learn.microsoft.com/azure/governance/policy/concepts/policy-as-code>
- Azure Resource Locks -- <https://learn.microsoft.com/azure/azure-resource-manager/management/lock-resources>
- Resource naming rules -- <https://learn.microsoft.com/azure/azure-resource-manager/management/resource-name-rules>

Azure Verified Modules

- AVM home -- <https://aka.ms/avm>
- AVM Bicep specs -- <https://github.com/Azure/bicep-registry-modules>
- AVM Terraform template -- <https://github.com/Azure/terraform-azurerm-avm-template>

ALZ accelerators

- ALZ-Bicep (Classic -- entering extended support) -- <https://github.com/Azure/ALZ-Bicep>
- Terraform CAF/ESLZ module (Classic -- entering extended support, archived Aug 2026) -- <https://github.com/Azure/terraform-azurerm-caf-enterprise-scale>

- ALZ accelerator (AVM-based, current) -- <https://azure.github.io/Azure-Landing-Zones/accelerator/>
- ALZ Terraform migration guide -- <https://aka.ms/alz/tf/migrate>
- ALZ PowerShell module -- <https://github.com/Azure/ALZ-PowerShell-Module>
- ALZ Portal accelerator -- <https://aka.ms/alz/portal>

Terraform / OpenTofu

- AzureRM provider -- <https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs>
- AzAPI provider -- <https://registry.terraform.io/providers/Azure/azapi/latest/docs>
- Backend `azurerm` -- <https://developer.hashicorp.com/terraform/language/settings/backends/azurerm>
- Recommended practices -- <https://developer.hashicorp.com/terraform/cloud-docs/recommended-practices>
- `terraform test` -- <https://developer.hashicorp.com/terraform/language/tests>
- `import` blocks -- <https://developer.hashicorp.com/terraform/language/import>
- Terratest -- <https://terratest.gruntwork.io/>
- OpenTofu -- <https://opentofu.org/>

GitHub

- Reusable workflows -- <https://docs.github.com/actions/using-workflows/reusing-workflows>
- Required workflows -- <https://docs.github.com/actions/using-workflows/required-workflows>
- Environments -- <https://docs.github.com/actions/deployment/targeting-different-environments/using-environments-for-deployment>
- Configuring OIDC in Azure -- <https://docs.github.com/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-azure>
- Security hardening for Actions -- <https://docs.github.com/actions/security-guides/security-hardening-for-github-actions>
- Secret scanning -- <https://docs.github.com/code-security/secret-scanning>
- Artifact attestations -- <https://docs.github.com/actions/security-guides/using-artifact-attestations-to-establish-provenance-for-builds>
- CODEOWNERS -- <https://docs.github.com/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners>

Azure DevOps

- Workload identity federation for service connections -- <https://learn.microsoft.com/azure/devops/pipelines/library/connect-to-azure>
- Branch policies -- <https://learn.microsoft.com/azure/devops/repos/git/branch-policies>
- YAML templates -- <https://learn.microsoft.com/azure/devops/pipelines/process/templates>

Policy / scanning tools

- PSRule for Azure -- <https://azure.github.io/PSRule.Rules.Azure/>
- Checkov -- <https://www.checkov.io/>
- tfsec / Trivy -- <https://aquasecurity.github.io/tfsec/>
- Conftest (OPA/Rego) -- <https://www.conftest.dev/>
- OPA / Rego -- <https://www.openpolicyagent.org/docs/latest/policy-language/>
- Sentinel -- <https://developer.hashicorp.com/sentinel>

Developer experience

- pre-commit -- <https://pre-commit.com/>
- pre-commit-terraform -- <https://github.com/antonbabenko/pre-commit-terraform>
- terraform-docs -- <https://terraform-docs.io/>
- tflint -- <https://github.com/terraform-linters/tflint>
- Devcontainers -- <https://containers.dev/>
- mise -- <https://mise.jdx.dev/>
- release-please -- <https://github.com/googleapis/release-please>
- Conventional Commits -- <https://www.conventionalcommits.org/>
- SemVer -- <https://semver.org/>
- actionlint -- <https://github.com/rhysd/actionlint>
- zizmor -- <https://github.com/woodruffw/zizmor>
- gitleaks -- <https://github.com/gitleaks/gitleaks>
- Sigstore gitsign -- <https://github.com/sigstore/gitsign>
- Renovate -- <https://docs.renovatebot.com/>

Documentation

- Diátaxis -- <https://diataxis.fr/>
- MADR -- Markdown ADR template -- <https://adr.github.io/madr/>
- adr-tools -- <https://github.com/npryce/adr-tools>
- GitHub Mermaid -- <https://github.blog/developer-skills/github/include-diagrams-markdown-files-mermaid/>
- D2 -- <https://d2lang.com/>

Methodology / metrics

- DORA metrics -- <https://dora.dev/>
- SLSA -- <https://slsa.dev/>

- Trunk-based development -- <https://trunkbaseddevelopment.com/>

Community video resources

- **ALZ Weekly Questions** -- YouTube playlist with best practices and Q&A from the ALZ product team: <https://www.youtube.com/playlist?list=PLukmLwi08-bTb4jpVTGdgFi5adbzj54jj>
 - Week 1: How to use AVM in ALZ, Bicep or Terraform? -- https://www.youtube.com/watch?v=ry39tWr_SXc
 - Week 2: When to use SLZ over ALZ? + alzlbttool -- <https://www.youtube.com/watch?v=r8h7F6IJIqw>
 - Week 3: How to Stay Current with ALZ Azure Policies -- https://www.youtube.com/watch?v=ddcVKS_MKkk
 - Week 4: Using EPAC for Azure Policy in ALZ -- https://www.youtube.com/watch?v=x1I_XhC6GtA
 - Week 5: Multi-Region ALZ Expansion -- https://www.youtube.com/watch?v=nii_LF65zyE
 - Week 6: Understanding ALZ Bicep File Structure -- <https://www.youtube.com/watch?v=sPA3YWkQ-4s>
 - Week 7: Migrating from CAF-Enterprise-Scale to AVM -- <https://www.youtube.com/watch?v=DSBWjQIVpSs>
 - Week 9: Community ALZ Projects + Right-Sized ALZ -- <https://www.youtube.com/watch?v=MznCQbT-EZw>
 - Week 10: Subscription Vending: Repo Structure, Security & Multi-Tenant -- <https://www.youtube.com/watch?v=11PmT0t6TUI>
 - Week 11: Azure Migrate Agent Preview -- <https://www.youtube.com/watch?v=kFS9lNPuXxM>
 - Week 12: Private DNS Zones: Centralized vs Decentralized -- <https://www.youtube.com/watch?v=WnrKlflIiwo>
 - ALZ for SMB -- <https://www.youtube.com/watch?v=cyLhLJEYIkU>
 - Azure Migrate Agent Deep Dive -- <https://www.youtube.com/watch?v=ODaFlsja308>
-

[← 14 Anti-patterns](#) · [Index](#)